

# Quantum Phynance

Sanjiv Das

w/Claude (Sonnet-4.6 and Opus-4.7)

[First unedited draft, this is an experiment  
in technical book generation]

# Contents

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>List of figures</b>                                               | <b>11</b> |
| <b>Foreword</b>                                                      | <b>12</b> |
| <b>1 What does quantum computing mean for finance?</b>               | <b>13</b> |
| 1.1 Why quantum computing, and why now? . . . . .                    | 14        |
| 1.2 The landscape of quantum finance . . . . .                       | 15        |
| 1.3 Quantum Monte Carlo and derivative pricing                       | 16        |
| 1.4 Portfolio optimization . . . . .                                 | 19        |
| 1.5 Risk management and credit risk . . . . .                        | 21        |
| 1.6 Quantum machine learning in finance . . . . .                    | 22        |
| 1.7 Algorithmic trading and market microstructure                    | 24        |
| 1.8 Post-quantum cryptography and financial security . . . . .       | 25        |
| 1.9 Current hardware, limitations, and the NISQ constraint . . . . . | 26        |
| 1.10 Schaden's quantum finance: a different tradition . . . . .      | 28        |
| 1.11 Prospects and the road ahead . . . . .                          | 29        |
| <b>2 Mathematical Preliminaries</b>                                  | <b>31</b> |

|      |                                                                              |    |
|------|------------------------------------------------------------------------------|----|
| 2.1  | Vector Spaces . . . . .                                                      | 32 |
| 2.2  | Span and Linear Independence . . . . .                                       | 32 |
| 2.3  | Basis and Dimension . . . . .                                                | 33 |
| 2.4  | Inner (Scalar) Product . . . . .                                             | 33 |
| 2.5  | Norms . . . . .                                                              | 34 |
| 2.6  | Hilbert Spaces . . . . .                                                     | 34 |
| 2.7  | Orthogonality and Orthonormal Bases . . .                                    | 36 |
| 2.8  | Gram–Schmidt Orthogonalization . . . . .                                     | 36 |
| 2.9  | Linear Operators . . . . .                                                   | 37 |
| 2.10 | Eigenvalues and Eigenvectors . . . . .                                       | 37 |
| 2.11 | Nonsingular Matrices . . . . .                                               | 38 |
| 2.12 | Similar Matrices . . . . .                                                   | 38 |
| 2.13 | Hermitian Operators . . . . .                                                | 38 |
| 2.14 | Unitary Operators . . . . .                                                  | 39 |
| 2.15 | Spectral Decomposition . . . . .                                             | 39 |
| 2.16 | Tensor Product of Vector Spaces . . . . .                                    | 40 |
| 2.17 | Tensor Product of Operators . . . . .                                        | 41 |
| 2.18 | Jordan Canonical Form . . . . .                                              | 41 |
| 2.19 | Matrix Norms . . . . .                                                       | 42 |
| 2.20 | Quadratic Forms . . . . .                                                    | 43 |
| 2.21 | Functions of Matrices . . . . .                                              | 44 |
| 2.22 | $L^2$ Function Spaces . . . . .                                              | 45 |
| 2.23 | Fourier Series . . . . .                                                     | 46 |
| 2.24 | Eigenfunctions . . . . .                                                     | 46 |
| 2.25 | Hadamard Operator . . . . .                                                  | 47 |
| 2.26 | Adjoint, Self-Adjoint, and Unitary Operators<br>on Function Spaces . . . . . | 47 |
| 2.27 | Matrix Representations of Operators . . . .                                  | 48 |

|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| 2.28     | Coordinate Transformations . . . . .                                 | 49        |
| <b>3</b> | <b>Two Worlds, One Question</b>                                      | <b>50</b> |
| 3.1      | The classical model: bits, gates, Turing . . .                       | 51        |
| 3.2      | The quantum model: amplitudes, not probabilities . . . . .           | 52        |
| 3.3      | Two qubits, four amplitudes, and the exponential staircase . . . . . | 53        |
| 3.4      | What classical machines cannot do (efficiently)                      | 54        |
| 3.5      | Reversibility, unitarity, and information conservation . . . . .     | 55        |
| 3.6      | Probability and randomness, classical and quantum . . . . .          | 56        |
| 3.7      | Hybrid computation: the practical near term                          | 57        |
| 3.8      | What the contrast really teaches . . . . .                           | 57        |
| <b>4</b> | <b>A Map of Quantum Computing</b>                                    | <b>59</b> |
| 4.1      | The four postulates, said plainly . . . . .                          | 60        |
| 4.2      | Qubits, gates, and circuits . . . . .                                | 60        |
| 4.3      | Where does the speed come from? . . . . .                            | 62        |
| 4.4      | The algorithmic landscape . . . . .                                  | 63        |
| 4.5      | The hardware reality check . . . . .                                 | 65        |
| 4.6      | What this book covers, and in what order .                           | 66        |
| <b>5</b> | <b>Where Quantum States Live</b>                                     | <b>68</b> |
| 5.1      | Why a complex Hilbert space? . . . . .                               | 69        |
| 5.2      | States as rays, not vectors . . . . .                                | 70        |
| 5.3      | Observables as Hermitian operators . . . . .                         | 70        |
| 5.4      | Composite systems and tensor products . .                            | 72        |
| 5.5      | Density operators and mixed states . . . . .                         | 73        |

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| 5.6      | The geometry of distinguishability . . . . .                             | 73        |
| 5.7      | Why finite-dimensional Hilbert spaces suffice<br>for computing . . . . . | 74        |
| 5.8      | What to carry into the next chapters . . . . .                           | 75        |
| <b>6</b> | <b>Dirac's Brackets</b>                                                  | <b>77</b> |
| 6.1      | Kets and bras . . . . .                                                  | 78        |
| 6.2      | Inner products: bra-ket . . . . .                                        | 79        |
| 6.3      | Outer products: ket-bras as operators . . . . .                          | 80        |
| 6.4      | The resolution of the identity . . . . .                                 | 80        |
| 6.5      | Operators in spectral form . . . . .                                     | 81        |
| 6.6      | Tensor products in bra-ket . . . . .                                     | 82        |
| 6.7      | Expectation values, in three notations . . . . .                         | 83        |
| 6.8      | A small grammar of bra-ket . . . . .                                     | 83        |
| <b>7</b> | <b>The Qubit</b>                                                         | <b>85</b> |
| 7.1      | What a qubit is . . . . .                                                | 86        |
| 7.2      | Why two dimensions, and not three or four? . . . . .                     | 86        |
| 7.3      | The Bloch sphere . . . . .                                               | 87        |
| 7.4      | Single-qubit gates as rotations . . . . .                                | 89        |
| 7.5      | Measurement and what it does . . . . .                                   | 90        |
| 7.6      | Multi-qubit registers . . . . .                                          | 91        |
| 7.7      | Physical realizations . . . . .                                          | 91        |
| 7.8      | A pocket reference . . . . .                                             | 93        |
| <b>8</b> | <b>Superposition</b>                                                     | <b>94</b> |
| 8.1      | The principle, plainly . . . . .                                         | 94        |
| 8.2      | Where does superposition come from? . . . . .                            | 95        |
| 8.3      | Quantum parallelism: powerful but tricky . . . . .                       | 96        |

|           |                                                                    |            |
|-----------|--------------------------------------------------------------------|------------|
| 8.4       | Examples in finance: data loading and amplitude encoding . . . . . | 97         |
| 8.5       | Measurement collapse and the basis choice . . . . .                | 98         |
| 8.6       | Coherent superposition versus statistical mixture . . . . .        | 99         |
| 8.7       | What superposition is, and what it isn't . . . . .                 | 100        |
| <b>9</b>  | <b>Entanglement</b>                                                | <b>102</b> |
| 9.1       | Entanglement, defined . . . . .                                    | 103        |
| 9.2       | The Bell states . . . . .                                          | 103        |
| 9.3       | Why Einstein objected . . . . .                                    | 104        |
| 9.4       | Quantifying entanglement . . . . .                                 | 105        |
| 9.5       | Entanglement as a computational resource . . . . .                 | 106        |
| 9.6       | Entanglement in financial algorithms . . . . .                     | 107        |
| 9.7       | Entanglement in cryptography . . . . .                             | 108        |
| 9.8       | What to remember . . . . .                                         | 109        |
| <b>10</b> | <b>Interference</b>                                                | <b>111</b> |
| 10.1      | What interference is . . . . .                                     | 111        |
| 10.2      | Constructive and destructive amplification . . . . .               | 113        |
| 10.3      | Coherence: the prerequisite for interference . . . . .             | 114        |
| 10.4      | The phase kickback trick . . . . .                                 | 115        |
| 10.5      | Interference in financial algorithms . . . . .                     | 116        |
| 10.6      | The classical view of interference, contrasted . . . . .           | 117        |
| 10.7      | Putting the three pillars together . . . . .                       | 118        |
| <b>11</b> | <b>Schrödinger's Equation</b>                                      | <b>120</b> |
| 11.1      | The equation . . . . .                                             | 120        |
| 11.2      | Eigenstates and stationary states . . . . .                        | 121        |

|           |                                                              |            |
|-----------|--------------------------------------------------------------|------------|
| 11.3      | Continuous-variable Schrödinger: the wave function . . . . . | 122        |
| 11.4      | Time-dependent Hamiltonians: pulses and gates . . . . .      | 123        |
| 11.5      | The path integral perspective . . . . .                      | 124        |
| 11.6      | What the Schrödinger equation forces on us . . . . .         | 125        |
| 11.7      | When the Schrödinger equation fails . . . . .                | 126        |
| 11.8      | Take-aways . . . . .                                         | 127        |
| <b>12</b> | <b>Hardware in the Real World</b>                            | <b>129</b> |
| 12.1      | DiVincenzo's criteria . . . . .                              | 130        |
| 12.2      | Superconducting qubits . . . . .                             | 130        |
| 12.3      | Trapped ions . . . . .                                       | 132        |
| 12.4      | Neutral atoms . . . . .                                      | 133        |
| 12.5      | Photonic qubits . . . . .                                    | 134        |
| 12.6      | Other platforms . . . . .                                    | 135        |
| 12.7      | Connectivity, qubit count, and effective scale . . . . .     | 136        |
| 12.8      | Control electronics and cryogenics . . . . .                 | 137        |
| 12.9      | The road ahead . . . . .                                     | 138        |
| <b>13</b> | <b>Decoherence</b>                                           | <b>140</b> |
| 13.1      | Two pictures of decoherence . . . . .                        | 141        |
| 13.2      | $T_1$ and $T_2$ . . . . .                                    | 141        |
| 13.3      | Quantum operations and noise channels . . . . .              | 143        |
| 13.4      | The Lindblad master equation . . . . .                       | 144        |
| 13.5      | Sources of decoherence in real hardware . . . . .            | 144        |
| 13.6      | Decoherence as the loss of interference . . . . .            | 146        |
| 13.7      | The classical limit as decoherence-driven . . . . .          | 146        |
| 13.8      | What it all means for algorithm design . . . . .             | 147        |

|           |                                                                     |            |
|-----------|---------------------------------------------------------------------|------------|
| <b>14</b> | <b>Saving Qubits from Themselves</b>                                | <b>149</b> |
| 14.1      | Why classical strategies don't work . . . . .                       | 150        |
| 14.2      | The three-qubit codes . . . . .                                     | 150        |
| 14.3      | Shor's nine-qubit code . . . . .                                    | 151        |
| 14.4      | Stabilizer formalism and CSS codes . . . . .                        | 152        |
| 14.5      | The surface code . . . . .                                          | 153        |
| 14.6      | The threshold theorem and fault tolerance .                         | 154        |
| 14.7      | Where the field stands in 2026 . . . . .                            | 155        |
| 14.8      | Beyond surface codes: high-rate codes and<br>biased noise . . . . . | 156        |
| 14.9      | The QEC takeaway . . . . .                                          | 157        |
| <b>15</b> | <b>Living in the NISQ Era</b>                                       | <b>159</b> |
| 15.1      | What NISQ means, precisely . . . . .                                | 160        |
| 15.2      | Variational algorithms: the workhorse . . .                         | 160        |
| 15.3      | Error mitigation, not correction . . . . .                          | 162        |
| 15.4      | Where NISQ wins, where it doesn't . . . . .                         | 163        |
| 15.5      | The benchmarking landscape . . . . .                                | 164        |
| 15.6      | The practical NISQ recipe for finance . . . .                       | 165        |
| 15.7      | What ends the NISQ era? . . . . .                                   | 166        |
| <b>16</b> | <b>Programming Quanta with Qiskit</b>                               | <b>168</b> |
| 16.1      | What Qiskit is . . . . .                                            | 169        |
| 16.2      | A minimal Qiskit circuit . . . . .                                  | 170        |
| 16.3      | Transpilation: from algorithm to chip . . . .                       | 171        |
| 16.4      | Primitives: sampling and estimation . . . . .                       | 172        |
| 16.5      | Qiskit Finance . . . . .                                            | 173        |
| 16.6      | Hybrid programs: classical-quantum loops .                          | 174        |
| 16.7      | Simulators: development before deployment                           | 175        |



|           |                                                                |            |
|-----------|----------------------------------------------------------------|------------|
| 16.8      | Where Qiskit is going . . . . .                                | 176        |
| 16.9      | Why this matters for the financial reader . .                  | 177        |
| <b>17</b> | <b>RSA</b>                                                     | <b>179</b> |
| 17.1      | Public-key cryptography in one paragraph .                     | 180        |
| 17.2      | The RSA construction . . . . .                                 | 180        |
| 17.3      | Where the security comes from . . . . .                        | 181        |
| 17.4      | RSA in the financial system . . . . .                          | 182        |
| 17.5      | Variants and accelerators . . . . .                            | 184        |
| 17.6      | Pollard rho and other classical factoring techniques . . . . . | 185        |
| 17.7      | The harvest-now–decrypt-later threat . . .                     | 186        |
| 17.8      | The post-quantum migration . . . . .                           | 186        |
| <b>18</b> | <b>Elliptic Curves</b>                                         | <b>188</b> |
| 18.1      | What an elliptic curve is . . . . .                            | 189        |
| 18.2      | Scalar multiplication and the discrete log problem . . . . .   | 190        |
| 18.3      | ECDH key exchange and ECDSA signatures                         | 191        |
| 18.4      | Curve choices and standards . . . . .                          | 192        |
| 18.5      | Why ECC is the modern default . . . . .                        | 193        |
| 18.6      | Pairings and identity-based cryptography .                     | 195        |
| 18.7      | Shor’s algorithm hits ECC at least as hard as RSA . . . . .    | 195        |
| 18.8      | The post-quantum landscape for ECC replacements . . . . .      | 197        |
| 18.9      | The unifying lesson . . . . .                                  | 198        |
| <b>19</b> | <b>Shor’s Algorithm</b>                                        | <b>199</b> |
| 19.1      | Factoring as period finding . . . . .                          | 200        |

|           |                                                                       |            |
|-----------|-----------------------------------------------------------------------|------------|
| 19.2      | The quantum period-finding subroutine . .                             | 201        |
| 19.3      | Why classical algorithms cannot match this                            | 203        |
| 19.4      | From factoring to discrete logarithms . . . .                         | 204        |
| 19.5      | Has Shor's algorithm been run? . . . . .                              | 205        |
| 19.6      | Resource estimates for cryptographically rel-<br>evant Shor . . . . . | 206        |
| 19.7      | The strategic implications . . . . .                                  | 207        |
| 19.8      | The book in one circuit . . . . .                                     | 208        |
| <b>20</b> | <b>Where Now?</b>                                                     | <b>210</b> |
| 20.1      | What we know, and where the limits are . .                            | 211        |
| 20.2      | An agenda for the next five years . . . . .                           | 212        |
| 20.3      | The hopes . . . . .                                                   | 214        |
| 20.4      | The fears . . . . .                                                   | 216        |
| 20.5      | A balanced posture . . . . .                                          | 219        |
| 20.6      | Closing . . . . .                                                     | 220        |

# *List of Figures*

# Foreword

Thanks to Alex Zecevic for letting me sit in on his quantum computing course, which motivated me to work on this project, after Brian Granger introduced me to quantum computing.

Thanks also to Claude models Sonnet-4.6 and Opus-4.7 for assistance in developing this book. See these notes on how the book was prepared: <https://github.com/srdas/quantum-finance/blob/main/notes/instructions.md>. All errors are my own. This is a draft and not meant for wide circulation but as a proof of concept that AI can be used to help write technical books and support pedagogy.

More to come.

## *Chapter 1*

# *What does quantum computing mean for finance?*

## A SURVEY OF OPPORTUNITIES, ALGORITHMS, AND THE ROAD AHEAD

Finance has always been hungry for computation. From the Black–Scholes formula to modern Monte Carlo risk engines, the history of quantitative finance is largely a story of harnessing more computing power to solve harder problems. Quantum computing represents a potential step-change in that story—not a modest acceleration, but a qualitatively different kind of computation that may unlock problems currently beyond practical reach.

This chapter surveys what quantum computing means for finance practitioners, theorists, and regulators. We draw on a growing body of literature spanning quantum algorithms, financial applications, and empirical demonstrations on real quantum hardware. Our aim is to give the reader a clear-eyed picture: what quantum advantage means and where it genuinely applies, what the current state of hardware permits, and what the promising directions are for the decade ahead.

## 1.1 *Why quantum computing, and why now?*

Classical computers process information as bits—binary digits that are either 0 or 1. Quantum computers exploit the principles of quantum mechanics to process qubits, which can exist in superpositions of 0 and 1 simultaneously. Combined with entanglement—correlations between qubits with no classical analogue—and interference (the ability to amplify correct answers and suppress wrong ones), quantum computers can in principle solve certain classes of problems exponentially faster than any classical machine.

The key word is *certain*. Quantum speedups are not universal. They arise for specific algorithmic structures: unstructured search (Grover’s algorithm gives a quadratic speedup), linear systems (the HHL algorithm), quantum simulation of physical systems, and—critically for finance—sampling and Monte Carlo integration. Most financial bottlenecks fall squarely into the problems where quantum advantage has been theoretically established [Egger et al., 2020; Herman et al., 2023;

[Orús et al., 2019](#)].

Why now? The answer is Preskill's NISQ era [[Preskill, 2018](#)]. Noisy Intermediate-Scale Quantum (NISQ) devices—quantum processors with 50 to a few hundred qubits, subject to noise and without full error correction—are already available through cloud platforms from IBM, Google, and others. These machines cannot yet tackle large-scale financial problems with proven quantum advantage, but they are large enough to run proof-of-concept demonstrations, to benchmark quantum algorithms against classical baselines, and to identify where quantum techniques will first become practically relevant. The financial sector has accordingly begun sustained investment in quantum research [[Auer et al., 2024](#); [Soller et al.; Soller](#)].

## 1.2 *The landscape of quantum finance*

The literature on quantum computing for finance has expanded rapidly. Comprehensive surveys [[Herman et al., 2022, 2023](#); [Orús et al., 2019](#); [Egger et al., 2020](#); [Bunescu and Vartei, 2024](#); [Gong et al., 2026](#); [Zhou, 2026](#)] identify three main domains where quantum techniques offer the strongest theoretical case for advantage: simulation and sampling (quantum Monte Carlo), optimization (portfolio construction, resource allocation), and machine learning (pattern recognition, prediction). These three pillars map directly onto the core computational problems of the financial industry.

**Remark 1.1.** *The survey by [Herman et al. \[2023\]](#) in Nature Reviews Physics provides an especially clear roadmap. It is deliberately addressed to physicists—describing both the classical financial techniques and the quantum alternatives—and is a recommended starting point for readers new to the field.*

Alongside these opportunities, quantum computing introduces a significant risk: the ability of future quantum computers to break widely-used public-key cryptography. Financial systems rely on cryptographic protocols to secure transactions, authenticate identities, and protect sensitive data. The harvest now, decrypt later threat—in which adversaries collect encrypted data today to decrypt it once quantum computers arrive—means that post-quantum security is already a strategic necessity, not a distant concern [[Auer et al., 2024](#); [Gong et al., 2026](#)].

We treat these topics in turn.

### *1.3 Quantum Monte Carlo and derivative pricing*

Monte Carlo simulation is the backbone of computational finance. Pricing over-the-counter derivatives, computing value-at-risk (VaR), running scenario analyses for stress testing, and calibrating stochastic models all rely on drawing large numbers of random samples and averaging a payoff or loss function. Classical Monte Carlo converges at a rate proportional



to  $1/\sqrt{N}$ , where  $N$  is the number of samples. Doubling precision requires quadrupling the sample count—an expensive proposition for complex, high-dimensional problems.

Quantum amplitude estimation [Egger et al., 2020], which underlies quantum Monte Carlo, achieves a quadratic speedup: it achieves the same precision with  $\sqrt{N}$  samples, or equivalently, it squares the convergence rate. For financial applications that spend significant compute budget on Monte Carlo, this is potentially transformative. The catch is that quantum amplitude estimation requires coherent quantum circuits of substantial depth, which is difficult on current NISQ hardware.

Stamatopoulos et al. [2020] demonstrated a complete methodology for pricing options on a gate-based quantum computer. Using quantum amplitude estimation, they priced European calls and Asian options, constructing quantum circuits that encode the payoff distribution and estimate the expected payoff directly. The method provides a quadratic speedup over classical Monte Carlo in the number of oracle calls.

For interest-rate derivatives, Martin et al. [2021] adapted the quantum principal component analysis algorithm to the Heath–Jarrow–Morton framework. The HJM model describes forward rate dynamics through volatility factors; in its multi-factor version, the number of stochastic factors drives computational cost. Quantum principal component analysis can reduce the effective dimensionality, and the authors demon-

strated this on IBM's five-qubit quantum computer for  $2 \times 2$  and  $3 \times 3$  covariance matrices estimated from historical data.

[Chen et al. \[2025\]](#) provided a rigorous complexity analysis of a quantum Monte Carlo algorithm for high-dimensional Black–Scholes PDEs. For payoff functions that are continuous and piecewise affine (covering most practical payoffs), the computational complexity is bounded polynomially in both the dimension  $d$  of the PDE and the reciprocal of the prescribed accuracy  $\varepsilon$ . This establishes that quantum Monte Carlo can escape the curse of dimensionality for a broad class of option pricing problems.

[Matsakos and Nield \[2024\]](#) applied quantum Monte Carlo to financial risk analytics at a broader scale, addressing scenario generation for equity, rate, and credit risk factors simultaneously. Their work targets the computationally expensive risk measurement workflows used in regulatory capital calculations and risk management, where hundreds of thousands of Monte Carlo paths are routinely needed.

[Blanchet et al. \[2025\]](#) provide a particularly accessible bridge between classical stochastic simulation and its quantum counterpart. Starting from Grover's algorithm for unstructured search, they build intuition through amplitude estimation toward Monte Carlo integration in finance, with hands-on Python/Qiskit implementations.

## 1.4 *Portfolio optimization*

Portfolio optimization is one of the most natural applications of quantum computing to finance. Markowitz mean–variance optimization is a quadratic program with linear constraints. At the scale of institutional portfolios—thousands of assets, cardinality constraints, transaction cost penalties, regulatory requirements—the problem becomes combinatorial, entering the territory of NP-hard problems for which quantum annealers and variational quantum algorithms offer potential advantages.

The standard quantum approach maps portfolio selection to a Quadratic Unconstrained Binary Optimization (QUBO) problem [Orús et al., 2019; Egger et al., 2020]. Each asset is represented by a binary variable indicating inclusion or exclusion. The objective and constraints are encoded in a Hamiltonian, and the ground state of that Hamiltonian corresponds to the optimal portfolio. Two main quantum approaches then apply: quantum annealing (used by D-Wave hardware) and QAOA (Quantum Approximate Optimization Algorithm), a gate-based variational method.

Morapakula et al. [2026] present an end-to-end portfolio optimization pipeline combining continuous mean-variance and Sharpe-ratio formulations with a QUBO-based discrete asset selection stage solved on D-Wave’s hybrid quantum annealing solver, followed by classical convex optimization for portfolio weights and quarterly rebalancing. Tested on Indian equity

markets, the hybrid pipeline achieves competitive returns relative to benchmark indices. They are careful to frame the contribution as demonstrating feasibility and integration rather than claiming quantum advantage—an important distinction at the current stage of hardware maturity.

[Thomassin et al. \[2025\]](#) address a fundamental challenge in quantum portfolio optimization: how to handle inequality constraints within a QAOA framework. Their approach embeds slack variables directly into the problem Hamiltonian as ancilla qubits, converting inequality constraints into a QUBO formulation. This slack-ancilla scheme enables QAOA to find the optimal portfolio consistently in simulation, whereas standard penalty-based QAOA fails. They also derive a quantum uncertainty principle for the simultaneous precision of portfolio risk and return—a striking formal analogy between quantum mechanics and financial optimization.

[Zaman et al. \[2024\]](#) propose a scalable framework, PO-QA, for systematically studying how quantum circuit parameters (rotation blocks, repetition depth, entanglement patterns) affect QAOA and VQE (Variational Quantum Eigensolver) performance on portfolio optimization. Convergence to the classical eigensolver solution is used as the benchmark, providing practical guidance on circuit design for finance. The framework was accepted at the 2024 IEEE International Conference on Quantum Computing and Engineering.

[Chou et al. \[2026\]](#) take a different approach: quantum-in-

spired algorithms, classical algorithms that borrow ideas from quantum mechanics (tensor networks, quantum-like state evolution) without requiring quantum hardware. Applied to G7 currency portfolio management, they demonstrate that quantum-inspired optimization can already outperform standard classical methods on real financial data, offering a near-term path to value even before fault-tolerant quantum computers arrive.

## 1.5 *Risk management and credit risk*

Risk management encompasses a broad set of computational challenges: credit risk modeling, value-at-risk estimation, conditional value-at-risk, counterparty exposure, and stress testing. Many of these reduce to tail-probability estimation problems, which are notoriously expensive classically because the rare events that matter most require enormous simulation samples.

[Dri et al. \[2022\]](#) address credit risk analysis directly. A quantum algorithm for credit risk with quadratic speedup over classical methods had been established in prior work; they extend it to handle more realistic credit models. Their contributions include: a richer default probability model that accounts for multiple systemic risk factors (not just a single factor), and a flexible loss given default parameter that accepts real-valued inputs rather than integers. The latter is critical for using real financial data in benchmarking. While the enhanced circuits are deeper and wider than the original,

they map a path toward a practically useful quantum credit risk tool as hardware matures.

[Matsakos and Nield \[2024\]](#) address the same domain from a simulation angle. Their quantum Monte Carlo framework for financial risk analytics generates scenarios jointly for equity prices, interest rates, and credit spreads, enabling holistic risk measurement. This is relevant for derivative counterparty risk, regulatory stress testing, and internal model validation.

[Zhou \[2026\]](#) review the evidence that quantum Monte Carlo algorithms can reduce sample-size requirements by up to a factor of four compared to classical methods, with applications specifically to VaR and CVaR. They also survey the challenges: scalability, data quality, integration with legacy classical systems, and regulatory compliance—all of which must be solved before quantum risk tools can be deployed in production.

## 1.6 *Quantum machine learning in finance*

Machine learning has transformed quantitative finance over the past decade, and quantum machine learning (QML) promises to extend that transformation by encoding data in quantum states that classical computers cannot efficiently represent [[Biamonte et al., 2017](#)]. The key intuition is that quantum systems naturally generate probability distributions that are hard to sample classically; if financial data has a similar complex structure, quantum models may learn it more

efficiently.

[Biamonte et al. \[2017\]](#) provide the foundational survey of QML. They identify building blocks including quantum principal component analysis, quantum support vector machines, and quantum neural networks, all of which have potential speedups over their classical counterparts under specific conditions on the data-loading problem. The caveat—sometimes called the input problem—is that efficiently loading classical data into quantum states is itself non-trivial and may limit practical speedups.

In finance, QML applications include return prediction, volatility forecasting, fraud detection, and portfolio construction using quantum-enhanced feature spaces. [Herman et al. \[2023\]](#) provide a careful assessment: while the theoretical case for QML speedups is strong, the practical advantage on financial data depends heavily on the structure of that data and the efficiency of quantum data encoding.

[Samal \[2024\]](#) focus specifically on the intersection of quantum computing and Bayesian machine learning in finance. Bayesian methods are natural for financial modeling—they handle uncertainty explicitly and update beliefs as new data arrives—but Bayesian inference is computationally expensive. Quantum approaches to Bayesian inference may offer speedups for posterior sampling, relevant to risk model calibration and regime detection.

## 1.7 *Algorithmic trading and market microstructure*

The use of quantum computing in trading represents one of the most commercially sensitive application areas. [Ciceri et al. \[2025\]](#) provide a rare empirical demonstration using real production-scale data. They study fill probability estimation for institutional algorithmic bond trading—the problem of predicting, for a given trade inquiry in the corporate bond market, the probability that the trade will actually execute at the quoted price.

Using IBM Heron quantum processors, they apply quantum feature transformations to intraday trade event data, then feed these transformed features into classical machine learning models. The striking finding is a relative gain of up to 34% in out-of-sample test scores for models with access to quantum hardware-transformed data compared to models using either the original data or noiseless quantum simulation. They conjecture that the inherent noise in current quantum hardware contributes to this improvement—the hardware noise acts as a form of stochastic regularization that improves generalization. This empirical result is both intriguing and controversial, and invites further study.

[Huang](#) provides a hands-on introduction to quantum finance through the lens of financial technology education, covering amplitude estimation, option pricing circuits, and portfolio optimization with Qiskit.



## 1.8 *Post-quantum cryptography and financial security*

The threat quantum computing poses to financial security is distinct from the opportunity it offers for financial computation. Shor's algorithm can factor large integers and solve the discrete logarithm problem in polynomial time, breaking RSA and elliptic-curve cryptography—the protocols that secure interbank settlements, electronic payments, digital signatures on securities, and encrypted communications throughout the financial system.

This threat is not immediate: Shor's algorithm requires thousands of error-corrected logical qubits, which will take years or decades to achieve. However, the *harvest now, decrypt later* attack vector is operational today. Adversaries can record encrypted financial communications now and decrypt them once powerful quantum computers arrive. For data that must remain confidential for years—strategic positions, client information, regulatory filings—this is an active risk.

[Auer et al. \[2024\]](#), writing for the Bank for International Settlements, lay out both the opportunities and risks of quantum computing for the financial system. They note that central banks, including the Banque de France, Deutsche Bundesbank, and the BIS Innovation Hub, have launched Project Leap to address post-quantum migration. The National Institute of Standards and Technology (NIST) has standardized post-quantum cryptographic algorithms, and financial regu-

lators are increasingly requiring institutions to assess their cryptographic exposure.

Gong et al. [2026] make the point directly: “Post-quantum cryptography is already strategically necessary because financial infrastructures must migrate before fault-tolerant attacks arrive.” The migration challenge is substantial—it requires identifying all cryptographic dependencies across complex IT infrastructure, updating protocols, and coordinating with counterparties and market infrastructure operators.

## 1.9 *Current hardware, limitations, and the NISQ constraint*

Every practical claim about quantum advantage in finance must be assessed against the reality of current hardware. NISQ devices suffer from:

- **Qubit noise.** Gate errors, measurement errors, and decoherence limit the depth of circuits that can execute reliably. Typical two-qubit gate error rates are on the order of 0.1%–1%, which means circuits with thousands of gates accumulate significant error.
- **Limited qubit count.** Current devices have tens to a few hundred qubits. Financial problems of practical size—pricing a realistic exotic option, optimizing a large portfolio—require far more qubits than are available today.

- **Connectivity constraints.** Qubits are not all directly connected; two-qubit gates can only be applied to physically adjacent qubits, requiring costly SWAP operations for non-adjacent interactions.
- **No quantum memory.** Unlike classical RAM, quantum states cannot be easily stored and retrieved. All computation must happen within a single coherence time.

[Preskill \[2018\]](#) was prescient in describing this era: NISQ devices “will be useful tools for exploring many-body quantum physics, and may have other useful applications, but the 100-qubit quantum computer will not change the world right away.” Seven years later, that assessment remains accurate. The 1000+ qubit devices now available are still far from the millions of physical qubits needed for fault-tolerant operation at the scale required for transformative financial applications.

The implication is that near-term quantum finance must be pursued through hybrid quantum-classical workflows [[Morapakula et al., 2026](#); [Gong et al., 2026](#)]. In these architectures, a classical optimizer drives the quantum circuit parameters, the quantum device evaluates a cost function or generates samples, and the result is fed back classically. This division of labor plays to the strengths of both paradigms while mitigating the limitations of current quantum hardware.

The IBM Quantum platform and Qiskit framework [[Pathak et al., 2025](#); [noa](#)] have been central to financial demonstrations. Qiskit provides circuit design, simulation, error miti-

gation, and access to real hardware, making it the dominant toolkit for quantum finance research. IBM's Qiskit Finance module includes implementations of quantum amplitude estimation for option pricing, QAOA for portfolio optimization, and quantum data loading utilities.

### 1.10 *Schaden's quantum finance: a different tradition*

The term “quantum finance” predates the quantum computing era. Schaden [2002] used quantum *theory*—not quantum *computing*—to model financial markets. In Schaden's formulation, the Hilbert space of market states consists of all possible configurations of investor holdings; the temporal evolution of an isolated market is unitary; and quantum operators represent basic financial transactions. The lognormal price distribution of classical finance emerges as the equilibrium of this quantum market model.

This tradition of applying quantum *formalism* to finance (path integrals for option pricing, Feynman–Kac formula interpretations, quantum field theory models of volatility) is distinct from the quantum *computing* approach. Both traditions use the mathematics of quantum mechanics, but for different purposes: one to model market dynamics, the other to speed up computation. This book is primarily concerned with the latter—quantum computing for finance—but the former tradition provides useful conceptual tools and is worth knowing.

## 1.11 *Prospects and the road ahead*

The consensus emerging from the literature is carefully optimistic. Quantum computing for finance is neither imminent revolution nor distant fantasy—it is a maturing research area with credible near-term applications and compelling long-term prospects.

The strongest near-term cases are [[Gong et al., 2026](#); [Herman et al., 2023](#); [Egger et al., 2020](#)]:

- **Hybrid optimization** for portfolio selection with cardinality or combinatorial constraints, where quantum annealing or QAOA provides meaningful advantages on NISQ hardware for moderate-size problems.
- **Quantum amplitude estimation** for Monte Carlo integration, once circuit depth and qubit count allow reliable execution of the required circuits. Error mitigation techniques are making this practical at small scales today.
- **Quantum feature transformations** for machine learning, where quantum hardware noise may actually help by providing a richer feature space than noiseless simulation—as suggested by the bond trading results of [Ciceri et al. \[2025\]](#).
- **Post-quantum security migration**, which is already underway in the financial sector and does not wait for quantum advantage.

Longer-term, fault-tolerant quantum computers will enable

the full theoretical speedups of quantum amplitude estimation (quadratic over Monte Carlo), quantum linear algebra (potentially exponential for structured linear systems), and quantum simulation of financial stochastic processes. The timeline is uncertain—perhaps 10–20 years for systems large enough to demonstrate unambiguous advantage on production-scale financial problems—but the trajectory of hardware improvement makes the direction clear.

**Remark 1.2.** *For finance professionals, the actionable near-term steps are: (1) build internal quantum computing expertise, (2) audit cryptographic infrastructure for post-quantum migration, (3) partner with quantum hardware providers to run proof-of-concept experiments on actual business problems, and (4) track the literature—the field is moving quickly and the competitive landscape will shift as hardware matures.*

The remaining chapters of this book develop the mathematical and algorithmic foundations needed to understand quantum finance at a technical level. We begin with the mathematical preliminaries—vector spaces, Hilbert spaces, linear operators—that underlie quantum mechanics and quantum information. From there we build toward quantum circuits, quantum algorithms, and their financial applications. The goal is to equip the reader to read the primary literature, implement quantum finance algorithms in Qiskit, and evaluate claims of quantum advantage with appropriate rigor.



## *Chapter 2*

# *Mathematical Preliminaries*

This chapter reviews the core mathematical structures used throughout the book. We proceed from the foundational notion of a vector space through inner products, linear operators, and spectral theory, culminating in tensor products and the Jordan canonical form. The mathematical topics covered here are essential for understanding the algebraic and geometric aspects of quantum information theory, including state spaces, quantum channels, and the behavior of quantum systems under various transformations. This is a minimal set of math I learned in the course I took on quantum and parallel computing by Alex Zecevic, as I ploughed through his book for many hours, see [Zecevic \[2022\]](#).

## 2.1 Vector Spaces

A *vector space* over a field  $\mathbb{F}$  (typically  $\mathbb{R}$  or  $\mathbb{C}$ ) is a set  $V$  equipped with two operations—vector addition and scalar multiplication—satisfying the standard axioms of commutativity, associativity, distributivity, and the existence of an additive identity  $\mathbf{0}$  and additive inverses.

**Example 2.1.**  $\mathbb{R}^n$  with component-wise addition and scalar multiplication is the canonical real vector space.  $\mathbb{C}^n$  is its complex analogue, central to quantum mechanics.

**Example 2.2.** The set of  $m \times n$  real matrices  $\mathbb{R}^{m \times n}$  forms a vector space of dimension  $mn$ .

## 2.2 Span and Linear Independence

Given vectors  $\{v_1, \dots, v_k\} \subset V$ , their *span* is

$$\text{span}\{v_1, \dots, v_k\} = \left\{ \sum_{i=1}^k \alpha_i v_i \mid \alpha_i \in \mathbb{F} \right\}.$$

The set  $\{v_1, \dots, v_k\}$  is *linearly independent* if

$$\sum_{i=1}^k \alpha_i v_i = \mathbf{0} \implies \alpha_i = 0 \forall i.$$

Otherwise it is linearly dependent.



**Example 2.3.** In  $\mathbb{R}^3$ , the vectors  $(1,0,0)$ ,  $(0,1,0)$ ,  $(0,0,1)$  are linearly independent. Adding  $(1,1,0)$  makes the set dependent since  $(1,1,0) = (1,0,0) + (0,1,0)$ .

## 2.3 Basis and Dimension

A *basis* of  $V$  is a linearly independent set that spans  $V$ . Every basis of a finite-dimensional space has the same cardinality, called the *dimension*  $\dim V$ .

Every vector  $v \in V$  has a unique representation  $v = \sum_i \alpha_i e_i$  with respect to a basis  $\{e_i\}$ ; the scalars  $\alpha_i$  are its *coordinates*.

**Example 2.4.** The standard basis of  $\mathbb{C}^2$  is  $\{e_1 = (1,0)^\top, e_2 = (0,1)^\top\}$ . In quantum information these are denoted  $|0\rangle$  and  $|1\rangle$ .

## 2.4 Inner (Scalar) Product

An *inner product* on  $V$  is a map  $\langle \cdot, \cdot \rangle : V \times V \rightarrow \mathbb{F}$  satisfying:

1. **Conjugate symmetry:**  $\langle u, v \rangle = \overline{\langle v, u \rangle}$ .
2. **Linearity in second argument:**  $\langle u, \alpha v + \beta w \rangle = \alpha \langle u, v \rangle + \beta \langle u, w \rangle$ .
3. **Positive definiteness:**  $\langle v, v \rangle \geq 0$ , with equality iff  $v = 0$ .

A vector space equipped with an inner product is called an *inner product space* (or *pre-Hilbert space*).

**Example 2.5.** On  $\mathbb{C}^n$ :  $\langle u, v \rangle = u^\dagger v = \sum_{i=1}^n \bar{u}_i v_i$ , where  $u^\dagger$  denotes the conjugate transpose (Dirac notation:  $\langle u|v \rangle$ ).

## 2.5 Norms

The inner product induces the *Euclidean* (or  $\ell^2$ ) norm

$$\|v\|_2 = \sqrt{\langle v, v \rangle}.$$

More generally, for  $v = (v_1, \dots, v_n) \in \mathbb{R}^n$ :

- **$\ell^p$  norm:**  $\|v\|_p = \left( \sum_{i=1}^n |v_i|^p \right)^{1/p}$ ,  $p \geq 1$ .
- **Infinity (supremum) norm:**  $\|v\|_\infty = \max_{1 \leq i \leq n} |v_i|$ .
- **Weighted infinity norm:**  $\|v\|_{w,\infty} = \max_{1 \leq i \leq n} \frac{|v_i|}{w_i}$ , with weights  $w_i > 0$ .

**Example 2.6.** For  $v = (3, -4, 1) \in \mathbb{R}^3$ :  $\|v\|_2 = \sqrt{26}$ ,  $\|v\|_\infty = 4$ . With weights  $w = (1, 2, 1)$ :  $\|v\|_{w,\infty} = \max\{\frac{3}{1}, \frac{4}{2}, \frac{1}{1}\} = 3$ .

## 2.6 Hilbert Spaces

An inner product space  $(\mathcal{H}, \langle \cdot, \cdot \rangle)$  is a *Hilbert space* if it is *complete*: every Cauchy sequence  $\{v_n\} \subset \mathcal{H}$  (satisfying  $\|v_m - v_n\| \rightarrow 0$  as  $m, n \rightarrow \infty$ ) converges to a limit in  $\mathcal{H}$ . Completeness ensures that limits of convergent infinite series remain in the space, which is essential for spectral expansions and Fourier analysis.

The inner product induces the norm  $\|v\| = \sqrt{\langle v, v \rangle}$  (Section 2.5), and the following two inequalities hold in every

inner product space:

- **Cauchy–Schwarz:**  $|\langle u, v \rangle| \leq \|u\| \|v\|$ , with equality iff  $u$  and  $v$  are linearly dependent.
- **Parallelogram law:**  $\|u + v\|^2 + \|u - v\|^2 = 2(\|u\|^2 + \|v\|^2)$ .

A Hilbert space is *separable* if it admits a countable orthonormal basis  $\{e_i\}$  such that every  $v \in \mathcal{H}$  has the expansion  $v = \sum_i \langle e_i, v \rangle e_i$ . All Hilbert spaces arising in quantum mechanics and this book are separable. The *Riesz representation theorem* states that every bounded linear functional  $\varphi : \mathcal{H} \rightarrow \mathbb{F}$  has the form  $\varphi(v) = \langle u, v \rangle$  for a unique  $u \in \mathcal{H}$  (in Dirac notation: every bra  $\langle u|$  corresponds to a unique ket  $|u\rangle$ ).

**Example 2.7.**  $\mathbb{C}^n$  with inner product  $\langle u, v \rangle = u^\dagger v$  is a finite-dimensional Hilbert space; completeness is automatic in finite dimensions. The state space of an  $n$ -level quantum system is  $\mathbb{C}^n$ .

**Example 2.8.**  $L^2[a, b]$  (square-integrable functions with  $\langle f, g \rangle = \int_a^b f^*(x) g(x) dx$ ) is an infinite-dimensional separable Hilbert space. It is the natural setting for wave functions and Fourier analysis; see Section 2.22.

## 2.7 Orthogonality and Orthonormal Bases

Two vectors  $u, v$  are *orthogonal*, written  $u \perp v$ , if  $\langle u, v \rangle = 0$ . A set  $\{e_1, \dots, e_n\}$  is *orthonormal* if

$$\langle e_i, e_j \rangle = \delta_{ij} = \begin{cases} 1 & i = j, \\ 0 & i \neq j. \end{cases}$$

An orthonormal basis (ONB) simplifies coordinate extraction:  $\alpha_i = \langle e_i, v \rangle$ .

**Example 2.9.** The standard basis  $\{e_i\}$  of  $\mathbb{R}^n$  is orthonormal under the Euclidean inner product. The Bell basis  $\{|\Phi^\pm\rangle, |\Psi^\pm\rangle\}$  is an ONB for  $\mathbb{C}^4$  used in quantum information.

## 2.8 Gram–Schmidt Orthogonalization

Given a linearly independent set  $\{v_1, \dots, v_k\}$ , the *Gram–Schmidt process* produces an orthonormal set  $\{e_1, \dots, e_k\}$  spanning the same subspace:

$$u_j = v_j - \sum_{i=1}^{j-1} \langle e_i, v_j \rangle e_i, \quad e_j = \frac{u_j}{\|u_j\|}.$$

**Example 2.10.** Starting from  $v_1 = (1, 1, 0)^\top$ ,  $v_2 = (1, 0, 1)^\top$  in  $\mathbb{R}^3$ :

$$e_1 = \frac{v_1}{\|v_1\|} = \frac{1}{\sqrt{2}}(1, 1, 0)^\top, \quad u_2 = v_2 - \langle e_1, v_2 \rangle e_1 = \frac{1}{2}(1, -1, 2)^\top.$$

## 2.9 Linear Operators

A map  $A : V \rightarrow W$  is a *linear operator* if  $A(\alpha u + \beta v) = \alpha Au + \beta Av$ . When  $V = W$  we call  $A$  a *linear operator on  $V$* . With respect to bases,  $A$  is represented by a matrix; composition of operators corresponds to matrix multiplication.

Key properties:

- **Kernel:**  $\ker A = \{v \in V \mid Av = \mathbf{0}\}$ .
- **Image:**  $\operatorname{im} A = \{Av \mid v \in V\}$ .
- **Rank–nullity theorem:**  $\dim \ker A + \dim \operatorname{im} A = \dim V$ .

## 2.10 Eigenvalues and Eigenvectors

A scalar  $\lambda \in \mathbb{F}$  is an *eigenvalue* of  $A$  if there exists a nonzero  $v \in V$  such that

$$Av = \lambda v.$$

Such  $v$  is an *eigenvector*. Eigenvalues are roots of the *characteristic polynomial*  $p(\lambda) = \det(A - \lambda I)$ .

**Example 2.11.** For  $A = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$ ,  $p(\lambda) = (\lambda - 2)(\lambda - 3)$ , giving eigenvalues  $\lambda_1 = 2$ ,  $\lambda_2 = 3$  with eigenvectors  $(1, 0)^\top$  and  $(1, 1)^\top$  respectively.

## 2.11 Nonsingular Matrices

A square matrix  $A \in \mathbb{F}^{n \times n}$  is *nonsingular* (invertible) if any of the following equivalent conditions hold:

- $\det A \neq 0$ .
- $\ker A = \{\mathbf{0}\}$  (zero is not an eigenvalue).
- $A$  has full rank  $n$ .
- There exists  $A^{-1}$  such that  $AA^{-1} = A^{-1}A = I$ .

## 2.12 Similar Matrices

Matrices  $A$  and  $B$  are *similar* if there exists a nonsingular  $P$  such that  $B = P^{-1}AP$ . Similar matrices represent the same linear operator in different bases and share eigenvalues, determinant, trace, and characteristic polynomial.

**Example 2.12.** If  $A$  is diagonalizable with eigenvalues  $\lambda_i$  and eigenvector matrix  $P$ , then  $P^{-1}AP = \text{diag}(\lambda_1, \dots, \lambda_n)$ .

## 2.13 Hermitian Operators

An operator  $A$  on an inner product space is *Hermitian* (self-adjoint) if  $A = A^\dagger$ , i.e.,  $\langle u, Av \rangle = \langle Au, v \rangle$  for all  $u, v$ . Key properties:

- All eigenvalues are real.

- Eigenvectors for distinct eigenvalues are orthogonal.
- Every Hermitian operator is diagonalizable by a unitary matrix.

**Example 2.13.** The Pauli matrices  $\sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ ,  $\sigma_y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$ ,  $\sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$  are Hermitian with eigenvalues  $\pm 1$ .

## 2.14 Unitary Operators

An operator  $U$  is *unitary* if  $U^\dagger U = U U^\dagger = I$ , equivalently if it preserves inner products:  $\langle Uu, Uv \rangle = \langle u, v \rangle$ . Unitary operators map ONBs to ONBs, have  $|\det U| = 1$ , and all eigenvalues lie on the unit circle in  $\mathbb{C}$ .

**Example 2.14.** The Hadamard gate  $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$  is unitary and Hermitian ( $H = H^\dagger = H^{-1}$ ), and maps the computational basis to an equal superposition.

## 2.15 Spectral Decomposition

The *spectral theorem* states that every Hermitian (or, over  $\mathbb{C}$ , every normal:  $AA^\dagger = A^\dagger A$ ) operator  $A$  admits the decomposition

$$A = \sum_i \lambda_i |e_i\rangle \langle e_i| = \sum_i \lambda_i P_i,$$

where  $\lambda_i$  are eigenvalues,  $|e_i\rangle$  are corresponding orthonormal eigenvectors, and  $P_i = |e_i\rangle\langle e_i|$  are rank-1 orthogonal projectors satisfying  $P_i P_j = \delta_{ij} P_i$  and  $\sum_i P_i = I$ .

This decomposition allows the definition of functions of operators:  $f(A) = \sum_i f(\lambda_i) P_i$ .

**Example 2.15.** For  $A = \sigma_z$  with eigenvalues  $\pm 1$  and eigenvectors  $|0\rangle, |1\rangle$ :

$$\sigma_z = |0\rangle\langle 0| - |1\rangle\langle 1|, \quad e^{i\theta\sigma_z} = e^{i\theta}|0\rangle\langle 0| + e^{-i\theta}|1\rangle\langle 1|.$$

## 2.16 Tensor Product of Vector Spaces

Given vector spaces  $V$  (dim  $m$ ) and  $W$  (dim  $n$ ), their *tensor product*  $V \otimes W$  is a vector space of dimension  $mn$  whose elements are (equivalence classes of) formal sums of *simple tensors*  $v \otimes w$ , with bilinearity:

$$(\alpha v) \otimes w = v \otimes (\alpha w) = \alpha(v \otimes w), \quad (v_1 + v_2) \otimes w = v_1 \otimes w + v_2 \otimes w.$$

If  $\{e_i\}$  is a basis of  $V$  and  $\{f_j\}$  is a basis of  $W$ , then  $\{e_i \otimes f_j\}$  is a basis of  $V \otimes W$ .

**Example 2.16.** For  $\mathbb{C}^2 \otimes \mathbb{C}^2 \cong \mathbb{C}^4$ , the computational basis is  $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$  where  $|ij\rangle = |i\rangle \otimes |j\rangle$ . An entangled state such as  $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$  cannot be written as a simple tensor  $v \otimes w$ .



## 2.17 Tensor Product of Operators

Given  $A: V \rightarrow V$  and  $B: W \rightarrow W$ , their tensor product  $A \otimes B: V \otimes W \rightarrow V \otimes W$  is defined by

$$(A \otimes B)(v \otimes w) = (Av) \otimes (Bw),$$

extended by linearity. In matrix form,  $A \otimes B$  is the *Kronecker product*: if  $A = (a_{ij})$ , then

$$A \otimes B = \begin{pmatrix} a_{11}B & \cdots & a_{1n}B \\ \vdots & \ddots & \vdots \\ a_{m1}B & \cdots & a_{mn}B \end{pmatrix}.$$

**Example 2.17.**  $\sigma_z \otimes I_2 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}$  measures the  $z$ -spin of the first qubit in a two-qubit system.

## 2.18 Jordan Canonical Form

Not every matrix is diagonalizable. Every  $A \in \mathbb{C}^{n \times n}$  is similar to its *Jordan canonical form*

$$J = P^{-1}AP = \text{diag}(J_{n_1}(\lambda_1), \dots, J_{n_k}(\lambda_k)),$$

where each *Jordan block* is

$$J_m(\lambda) = \begin{pmatrix} \lambda & 1 & & \\ & \lambda & \ddots & \\ & & \ddots & 1 \\ & & & \lambda \end{pmatrix} \in \mathbb{C}^{m \times m}.$$

The number and sizes of Jordan blocks for eigenvalue  $\lambda$  are determined by the dimensions of  $\ker(A - \lambda I)^k$  for  $k = 1, 2, \dots$ . The Jordan form is unique up to reordering of blocks.

**Example 2.18.** The matrix  $A = \begin{pmatrix} 3 & 1 \\ 0 & 3 \end{pmatrix}$  is already in Jordan form with a single  $2 \times 2$  block  $J_2(3)$ . It has eigenvalue  $\lambda = 3$  (algebraic multiplicity 2, geometric multiplicity 1) and is not diagonalizable.

The Jordan form is essential for computing matrix exponentials and analyzing the long-run behavior of non-diagonalizable systems, relevant in the study of quantum channels and decoherence.

## 2.19 Matrix Norms

Every matrix  $A \in \mathbb{F}^{m \times n}$  induces an *operator norm*  $\|A\|_{\text{op}} = \max_{\|x\|=1} \|Ax\|$ . The induced norms most relevant here are:

- **Spectral (2-)norm:**  $\|A\|_2 = \sqrt{\rho(A^\top A)}$ , where  $\rho(A^\top A) = \max_i \lambda_i(A^\top A)$  is the *spectral radius* of  $A^\top A$ .

• **Induced infinity norm:**  $\|A\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|$  (maximum absolute row sum).

• **Weighted infinity norm:**  $\|A\|_{\omega, \infty} = \max_{1 \leq i \leq m} \frac{1}{\omega_i} \sum_{j=1}^n |a_{ij}| \omega_j$ ,  
with weights  $\omega_j > 0$ .

All matrix norms satisfy  $\|A\| \geq 0$ ,  $\|cA\| = |c|\|A\|$ , the triangle inequality, and submultiplicativity  $\|AB\| \leq \|A\|\|B\|$ .

**Example 2.19.** For  $A = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 4 & 5 \\ 1 & 2 & 3 \end{pmatrix}$ , the eigenvalues of  $A^\top A$  are approximately 0.310, 13.776, 45.914, giving  $\|A\|_2 = \sqrt{45.914} \approx 6.776$ . The induced infinity norm is  $\|A\|_\infty = \max\{3, 9, 6\} = 9$ . With weights  $\omega = (1, 0.5, 0.4)$ :

$$\|A\|_{\omega, \infty} = \max\left\{\frac{1 \cdot 1 + 0 \cdot 0.5 + 2 \cdot 0.4}{1}, \frac{0 \cdot 1 + 4 \cdot 0.5 + 5 \cdot 0.4}{0.5}, \frac{1 \cdot 1 + 2 \cdot 0.5 + 3 \cdot 0.4}{0.4}\right\}$$

## 2.20 Quadratic Forms

Given a symmetric matrix  $A \in \mathbb{R}^{n \times n}$ , the associated *quadratic form* is  $q(x) = x^\top Ax$ . The matrix  $A$  is *positive definite* if  $q(x) > 0$  for all  $x \neq \mathbf{0}$ , and *positive semi-definite* if  $q(x) \geq 0$  for all  $x$ .

Via the spectral decomposition  $A = T\Lambda T^\top$  (with  $T$  ortho-

nal), the substitution  $y = T^\top x$  gives

$$q(x) = x^\top A x = y^\top \Lambda y = \sum_{i=1}^n \lambda_i y_i^2.$$

Hence  $A$  is positive definite if and only if all eigenvalues  $\lambda_i > 0$ . Choosing  $x = x_k$  (the  $k$ -th eigenvector) gives  $q(x_k) = \lambda_k \|x_k\|_2^2$ , so any  $\lambda_k \leq 0$  immediately violates positive definiteness.

**Example 2.20.**  $A = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$  has characteristic polynomial  $\lambda^2 - 5\lambda + 5 = 0$ , giving  $\lambda_{1,2} = \frac{5 \pm \sqrt{5}}{2} \approx 1.382, 3.618$ . Both are positive, confirming  $q(x) = 2x_1^2 + 2x_1x_2 + 3x_2^2 > 0$  for all  $x \neq 0$ .

## 2.21 Functions of Matrices

For  $f$  with Taylor series  $f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(0)}{k!} x^k$ , the *matrix function*  $f(A)$  is defined by

$$f(A) = \sum_{k=0}^{\infty} \frac{f^{(k)}(0)}{k!} A^k.$$

For a diagonalizable  $A = T \Lambda T^{-1}$ , this simplifies to

$$f(A) = T f(\Lambda) T^{-1}, \quad f(\Lambda) = \text{diag}(f(\lambda_1), \dots, f(\lambda_n)),$$

since  $A^k x_i = \lambda_i^k x_i$  implies  $f(A)x_i = f(\lambda_i)x_i$  for each eigenvector  $x_i$ . For a Hermitian  $A = \sum_i \lambda_i P_i$ , the spectral form is  $f(A) = \sum_i f(\lambda_i) P_i$ , consistent with Section 2.15.

**Example 2.21.** Let  $A = \begin{pmatrix} 2 & 2 \\ 2 & 4 \end{pmatrix}$  and  $f(x) = e^{-x} \cos x$ . The eigenvalues are  $\lambda_{1,2} = 3 \mp \sqrt{5}$  ( $\approx 0.764, 5.236$ ). Computing:

$$f(\Lambda) \approx \begin{pmatrix} 0.336 & 0 \\ 0 & 0.003 \end{pmatrix}, \quad f(A) = T f(\Lambda) T^{-1} \approx \begin{pmatrix} 0.244 & \\ & 0.149 \end{pmatrix}$$

## 2.22 $L^2$ Function Spaces

The space  $L^2[a, b]$  consists of complex-valued functions  $f: [a, b] \rightarrow \mathbb{C}$  that are square-integrable,  $\int_a^b |f(x)|^2 dx < \infty$ . It is an infinite-dimensional *Hilbert space* with inner product

$$\langle f, g \rangle = \int_a^b f^*(x) g(x) dx,$$

satisfying the same axioms as the finite-dimensional case: conjugate symmetry  $\langle f, g \rangle = \overline{\langle g, f \rangle}$ , linearity in the second argument, and positive definiteness. A countable set  $\{\phi_i\}$  is an *orthonormal system* if  $\langle \phi_i, \phi_j \rangle = \delta_{ij}$ . When it spans  $L^2[a, b]$ , every  $f$  has the expansion

$$f = \sum_{i=1}^{\infty} \langle \phi_i, f \rangle \phi_i,$$

the infinite-dimensional analogue of coordinate extraction via an ONB.

## 2.23 Fourier Series

A  $T$ -periodic function satisfies  $f(t+T) = f(t)$ . With  $\omega = 2\pi/T$ , the *Fourier series* is

$$f(t) = a_0 + \sum_{n=1}^{\infty} a_n \cos(n\omega t) + \sum_{n=1}^{\infty} b_n \sin(n\omega t).$$

The functions  $\{1, \cos(n\omega t), \sin(n\omega t)\}_{n \geq 1}$  are orthogonal in  $L^2[0, T]$ , giving

$$a_0 = \frac{1}{T} \int_0^T f(t) dt, \quad a_n = \frac{2}{T} \int_0^T f(t) \cos(n\omega t) dt, \quad b_n = \frac{2}{T} \int_0^T f(t) \sin(n\omega t) dt.$$

For even functions ( $f(t) = f(-t)$ ), all  $b_n = 0$ . The Fourier basis plays the same structural role in  $L^2[0, T]$  as the monomial basis  $\{1, x, x^2, \dots\}$  in a Taylor expansion, but orthogonality makes coefficient extraction direct.

## 2.24 Eigenfunctions

A linear operator  $\hat{A}$  on a Hilbert space has an *eigenfunction*  $\phi_i$  with *eigenvalue*  $\lambda_i$  if  $\hat{A}\phi_i = \lambda_i\phi_i$ . When  $\hat{A}$  possesses an orthonormal set of eigenfunctions  $\{\phi_i\}$  spanning the space, the action on any  $f = \sum_i \beta_i \phi_i$  is computed term-by-term:

$$\hat{A}f = \sum_i \lambda_i \beta_i \phi_i, \quad \beta_i = \langle \phi_i, f \rangle.$$

**Example 2.22.** Let  $\{\psi_0, \psi_1\}$  be an orthonormal basis and define the bit-flip operator  $\hat{X}$  by  $\hat{X}\psi_0 = \psi_1$ ,  $\hat{X}\psi_1 = \psi_0$ . For

$\psi = \alpha_0 \psi_0 + \alpha_1 \psi_1$ , the eigenfunction equation  $\hat{X}\psi = \lambda\psi$  gives  $\lambda^2 = 1$ , so  $\lambda_1 = 1$  and  $\lambda_2 = -1$ . The normalized eigenfunctions are

$$\psi_A = \frac{1}{\sqrt{2}}(\psi_0 + \psi_1), \quad \psi_B = \frac{1}{\sqrt{2}}(\psi_0 - \psi_1).$$

## 2.25 Hadamard Operator

The Hadamard operator  $\hat{H}$  acts on the orthonormal basis  $\{\psi_0, \psi_1\}$  by

$$\hat{H}\psi_0 = \frac{1}{\sqrt{2}}(\psi_0 + \psi_1), \quad \hat{H}\psi_1 = \frac{1}{\sqrt{2}}(\psi_0 - \psi_1).$$

It is both self-adjoint and unitary ( $\hat{H} = \hat{H}^\dagger = \hat{H}^{-1}$ ). The characteristic equation is  $\lambda^2 - 1 = 0$ , giving  $\lambda_1 = 1$  and  $\lambda_2 = -1$ , with normalized eigenfunctions

$$\psi_A = \cos \frac{\pi}{8} \psi_0 + \sin \frac{\pi}{8} \psi_1 \approx 0.9239 \psi_0 + 0.3827 \psi_1, \quad \psi_B = -\sin \frac{\pi}{8} \psi_0 + \cos \frac{\pi}{8} \psi_1 \approx -0.3827 \psi_0 + 0.9239 \psi_1$$

## 2.26 Adjoint, Self-Adjoint, and Unitary Operators on Function Spaces

For a linear operator  $\hat{A}$  on a Hilbert space  $\mathcal{H}$ , the adjoint  $\hat{A}^\dagger$  is the unique operator satisfying

$$\langle \hat{A}f, g \rangle = \langle f, \hat{A}^\dagger g \rangle \quad \forall f, g \in \mathcal{H}.$$

- $\hat{A}$  is *self-adjoint* (Hermitian) if  $\hat{A} = \hat{A}^\dagger$ : eigenvalues are real and eigenfunctions for distinct eigenvalues are orthogonal.
- $\hat{U}$  is *unitary* if  $\hat{U}^\dagger \hat{U} = \hat{U} \hat{U}^\dagger = \hat{I}$ , equivalently  $\langle \hat{U}f, \hat{U}g \rangle = \langle f, g \rangle$ . All eigenvalues satisfy  $|\lambda_k| = 1$ .

The unit-circle property follows directly: if  $\hat{U}\xi_k = \lambda_k \xi_k$ , then

$$\langle \xi_k, \xi_k \rangle = \langle \hat{U}\xi_k, \hat{U}\xi_k \rangle = |\lambda_k|^2 \langle \xi_k, \xi_k \rangle \implies |\lambda_k| = 1.$$

These properties mirror exactly those of Hermitian and unitary matrices, confirming that the matrix and functional-space theories are instances of the same abstract framework.

## 2.27 Matrix Representations of Operators

Given operator  $\hat{A}$  on a space with orthonormal basis  $\{\psi_j\}_{j=0}^{n-1}$ , its *matrix representation*  $A$  has entries

$$A_{jk} = \langle \psi_j, \hat{A}\psi_k \rangle,$$

so that  $\hat{A}\psi_k = \sum_j A_{jk}\psi_j$ . The coordinates transform as  $\rho = A\alpha$ : if  $f = \sum_k \alpha_k \psi_k$  then  $\hat{A}f = \sum_j \rho_j \psi_j$  with  $\rho_j = \sum_k A_{jk}\alpha_k$ .

**Example 2.23.** In basis  $\{\psi_0, \psi_1\}$ , the operators  $\hat{X}$  and  $\hat{H}$  have matrix representations

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \sigma_x, \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix},$$

matching the Pauli matrix  $\sigma_x$  and the Hadamard gate from Sections 2.13 and 2.14.



## 2.28 Coordinate Transformations

Let  $\{\psi_j\}$  and  $\{\xi_k\}$  be two orthonormal bases. The *change-of-basis matrix*  $S$  with entries

$$S_{jk} = \langle \psi_j, \xi_k \rangle$$

encodes the  $\psi$ -coordinates of each new basis vector as columns. If  $f$  has coordinates  $\alpha$  in the  $\psi$ -basis and  $\beta$  in the  $\xi$ -basis, then  $\alpha = S\beta$ . Since both bases are orthonormal,  $S$  is unitary ( $S^{-1} = S^\dagger$ ), and the matrix representation of  $\hat{A}$  transforms as

$$A_\xi = S^{-1} A_\psi S = S^\dagger A_\psi S,$$

making  $A_\xi$  and  $A_\psi$  similar matrices with identical eigenvalues.

**Example 2.24.** *The Hadamard operator has matrix  $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$  in the  $\{\psi_0, \psi_1\}$  basis. In its eigenbasis  $\{\psi_A, \psi_B\}$  the representation is  $H_{\text{eig}} = \text{diag}(1, -1)$ . The change-of-basis matrix  $S$  with columns equal to the  $\psi$ -coordinates of  $\psi_A$  and  $\psi_B$  satisfies  $S^\dagger H S = \text{diag}(1, -1)$ :*

$$S = \begin{pmatrix} \cos \frac{\pi}{8} & -\sin \frac{\pi}{8} \\ \sin \frac{\pi}{8} & \cos \frac{\pi}{8} \end{pmatrix} \approx \begin{pmatrix} 0.9239 & -0.3827 \\ 0.3827 & 0.9239 \end{pmatrix}.$$

## *Chapter 3*

# *Two Worlds, One Question*

### CLASSICAL BITS, QUANTUM AMPLITUDES, AND WHAT REALLY CHANGES

Every computation we have ever performed—on an abacus, a slide rule, a Cray supercomputer, or a smartphone—obeys the rules of classical physics. Switches click, electrons flow, transistors gate. The arithmetic underneath is binary, deterministic, and local. Quantum computation breaks each of those assumptions in a precise, mathematical way. This chapter draws the contrast carefully, because most misunderstandings about quantum computing trace back to mixing the two worlds together.

### 3.1 *The classical model: bits, gates, Turing*

A classical bit is a variable that takes the value 0 or 1. A classical computer is, in the abstract, a Turing machine: a finite-state controller reading and writing on an unbounded tape [Shannon, 1948]. Two facts make this model what it is:

- **Definiteness.** At every moment, every bit has a single, observable value. There is no ambiguity, no parallel reality, no probability before measurement (excepting the trivial case where we are simply uncertain about what value was written).
- **Locality.** The state of bit  $A$  has no effect on bit  $B$  unless a wire, signal, or instruction couples them. Information must be transported to be felt elsewhere.

Classical logic is built from the standard gates AND, OR, NOT, NAND—and any function on  $n$  bits can be assembled from them. Gates are typically irreversible:  $\text{AND}(1,0) = 0$  and  $\text{AND}(0,1) = 0$  produce the same output from different inputs, so you cannot recover the input from the output. Landauer showed this irreversibility has a thermodynamic cost: erasing one bit dissipates at least  $k_B T \ln 2$  joules of heat [Landauer, 1961]. Bennett later showed that one can, in principle, compute reversibly and pay no such bill [Bennett, 1973]. That observation matters more than it first appears—quantum computation is *intrinsically* reversible, so it sidesteps Landauer’s tax entirely.

## 3.2 The quantum model: amplitudes, not probabilities

A qubit is the quantum analogue of a bit. Where a bit is a number, a qubit is a unit vector in a two-dimensional complex Hilbert space (Chapter on Hilbert spaces). We write a qubit's state as

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1.$$

The complex numbers  $\alpha, \beta$  are amplitudes. When we measure the qubit in the standard basis we obtain 0 with probability  $|\alpha|^2$  and 1 with probability  $|\beta|^2$ . The Born rule [Born, 1926] provides this bridge between the abstract amplitude and the observed probability.

But—and this is the entire point—between preparation and measurement, the system is described by amplitudes, not probabilities. Amplitudes can be negative or complex; probabilities cannot. Amplitudes can cancel; probabilities cannot. The capacity for *cancellation* is the engine of quantum interference, and it is what classical models cannot replicate without paying an exponential cost.

**Remark 3.1.** *A common, misleading line is “a qubit can be both 0 and 1 at the same time.” Better: a qubit is described by a complex unit vector whose components determine the statistics of measurement outcomes. Before measurement, neither outcome is realised; after measurement, exactly one is. The richness lies in what happens between.*

### 3.3 Two qubits, four amplitudes, and the exponential staircase

Two classical bits have four possible joint states—00, 01, 10, 11. Two qubits are described by four *amplitudes*:

$$|\Psi\rangle = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle, \quad \sum |\alpha_{ij}|^2 =$$

An  $n$ -qubit state lives in a space of dimension  $2^n$  and is therefore specified by  $2^n$  complex numbers. Three qubits give 8 amplitudes; ten qubits give 1024; fifty qubits give over a quadrillion. Simulating a 50-qubit pure state on a classical computer already pushes the limits of supercomputing memory [Arute et al., 2019].

This exponential scaling is not a parlour trick. It is the precise resource that quantum algorithms exploit. A quantum computer with  $n$  qubits can, in some sense, manipulate  $2^n$  amplitudes in parallel via a single unitary operation. The catch—one of the deep catches of the field—is that we cannot *read out* all  $2^n$  amplitudes. Measurement collapses the state into a single classical outcome, drawn from a probability distribution determined by the amplitudes. So the algorithmic art is to arrange amplitudes so that, upon measurement, the answer we want falls out with high probability.

**Example 3.1.** *For three classical bits, we can represent any specific configuration with three switches. For three qubits, a general state requires  $2^3 = 8$  complex amplitudes, subject to one normalization constraint and one global phase ambiguity—*

leaving  $2 \cdot 8 - 2 = 14$  real degrees of freedom. The state space is genuinely vast compared with the eight classical configurations.

### 3.4 What classical machines cannot do (efficiently)

It is tempting to say that quantum computers are simply faster classical computers. They are not. What separates them is a small list of computational structures where classical methods require exponential resources and quantum methods do not. The most consequential examples are:

- **Factoring large integers.** Shor's algorithm [Shor, 1997] factors an  $n$ -bit integer in time polynomial in  $n$ , whereas the best classical algorithms (the number field sieve, [Lenstra et al., 1993]) require sub-exponential but super-polynomial time. This is the threat to RSA cryptography (Chapter on RSA).
- **Unstructured search.** Grover's algorithm [Grover, 1996] searches an unsorted database of  $N$  items in  $O(\sqrt{N})$  steps, a quadratic improvement over classical  $O(N)$ .
- **Simulation of quantum systems.** Simulating a system of  $n$  interacting quantum particles on a classical computer requires manipulating a state space of dimension exponential in  $n$ . Feynman pointed this out in 1982 [Feynman, 1982]: nature is quantum, so to simulate it efficiently we should build quantum computers.

- **Sampling and integration.** Quantum amplitude estimation provides a quadratic speedup over classical Monte Carlo [Egger et al., 2020], which is precisely the bottleneck of derivative pricing and risk simulation in finance.

For most everyday computing tasks—web browsing, word processing, training large language models—quantum computers offer no advantage. Quantum speedups are surgical, not universal.

### *3.5 Reversibility, unitarity, and information conservation*

All allowed transformations of a closed quantum system are unitary (Section 2.14). A unitary  $U$  satisfies  $U^\dagger U = I$  and preserves the inner product, so it preserves information. Quantum gates are therefore reversible by construction: every gate  $U$  has an inverse  $U^\dagger$ , and applying both in sequence returns the original state.

This is a strict change from classical computation. There is no quantum AND gate that maps  $(\alpha, \beta) \rightarrow \alpha \wedge \beta$  in the naive sense, because such a map would lose information and could not be unitary. Instead, quantum versions of irreversible classical gates require ancillary qubits to preserve a record of the input. The Toffoli gate (controlled-controlled-NOT) is the canonical universal reversible gate. Together with single-qubit unitaries and CNOT, it spans the entire space of quantum computation [Barenco et al., 1995].

**Remark 3.2.** *The fact that classical computation can be embedded inside quantum computation is what makes the latter strictly more powerful (or equally powerful) on every problem. Quantum cannot lose; it can only fail to gain, depending on the problem structure.*

### 3.6 Probability and randomness, classical and quantum

Classical probability is, in spirit, a tool for reasoning under ignorance. A coin lying under your hand is heads or tails with certainty; you simply do not know which. Quantum probability is different. Until measured, a qubit in superposition is not secretly 0 or secretly 1. It is genuinely in a state described by amplitudes, with no underlying definite value waiting to be revealed. The experimental evidence for this view is decisive: violations of Bell's inequalities [Bell, 1964; Aspect et al., 1982] rule out the existence of any local hidden-variable model that would reproduce quantum statistics.

This has practical consequences. A classical Monte Carlo simulation generates samples from a distribution; we average them and obtain an estimate of an expected value. A quantum amplitude estimation does something subtler: the quantum circuit *encodes* the desired expectation as the amplitude of a particular outcome, and amplitude estimation reads that amplitude back with precision that scales as  $1/N$  rather than the classical  $1/\sqrt{N}$ . The improvement is paid for in coherence, not in samples.



### 3.7 *Hybrid computation: the practical near term*

For the foreseeable future—certainly the entire NISQ era [Preskill, 2018]—useful quantum computation will be hybrid quantum–classical. A classical machine prepares data, drives the quantum circuit’s parameters, and post-processes measurement results; the quantum machine does the part where its peculiar resources matter. Variational quantum algorithms [Cerezo et al., 2021] are the canonical example: a classical optimizer adjusts a parameterized quantum circuit to minimize a cost function evaluated on the quantum device.

In finance, this hybrid pattern is already standard. Portfolio optimization runs as continuous mean–variance on the classical side, with a discrete QUBO solved on the quantum side and weights recombined classically [Morapakula et al., 2026]. Quantum machine learning models load classical financial data into quantum states, transform them via parameterized circuits, and read out classical features for downstream classical models [Ciceri et al., 2025]. The two worlds do not compete; they cooperate, with quantum hardware as the accelerator for the small set of operations where it has a genuine edge.

### 3.8 *What the contrast really teaches*

The deepest difference between classical and quantum computation is not speed. It is what counts as a *state*. Classical

states are points in a discrete configuration space. Quantum states are unit vectors in a continuous Hilbert space, evolving by unitary maps and projected to classical outcomes only at the moment of measurement. Everything else in this book—superposition, entanglement, interference, error correction, Shor’s algorithm—follows from this single shift in mathematical setting.

Keep this contrast in mind as we proceed. When a chapter says “the algorithm uses interference to amplify the right answer,” it is leveraging amplitude cancellation, which has no classical counterpart. When we say “the entangled state cannot be written as a product,” we are pointing to a correlation structure absent from classical probability distributions. The mathematics of Chapter on Mathematical Preliminaries is the formalism for these statements; the chapters that follow show what the formalism makes possible.



## *Chapter 4*

# *A Map of Quantum Computing*

### POSTULATES, GATES, CIRCUITS, AND WHERE THE SPEED COMES FROM

Quantum computing is a small set of physical ideas dressed in the formalism of linear algebra. Once you accept that states are unit vectors, evolution is unitary, and measurement collapses, the rest of the discipline—qubits, circuits, algorithms—is largely bookkeeping. This chapter lays out that bookkeeping at survey level, so the reader has a map of the territory before we work through each region in detail. The exposition draws on the canonical treatment of [Nielsen and Chuang \[2010\]](#), and on the historical motivations of [Feynman \[1982\]](#) and [Deutsch \[1985\]](#).

## 4.1 The four postulates, said plainly

Quantum mechanics has many possible formulations; for computation we need only four working postulates.

1. **State space.** The state of an isolated quantum system is a unit vector in a complex Hilbert space  $\mathcal{H}$  (Section 2.6).
2. **Evolution.** The state of a closed quantum system evolves by a unitary transformation  $|\psi'\rangle = U|\psi\rangle$ , where  $U^\dagger U = I$  (Section 2.14).
3. **Measurement.** A measurement is described by a collection of operators  $\{M_m\}$  acting on  $\mathcal{H}$ , satisfying  $\sum_m M_m^\dagger M_m = I$ . The probability of outcome  $m$  is  $\langle\psi|M_m^\dagger M_m|\psi\rangle$ ; conditional on observing  $m$ , the post-measurement state is  $M_m|\psi\rangle/\sqrt{\langle\psi|M_m^\dagger M_m|\psi\rangle}$ .
4. **Composition.** The state space of a composite system is the tensor product of the component state spaces.

The last postulate is what makes the state space exponentially large. Two qubits live in  $\mathbb{C}^2 \otimes \mathbb{C}^2 = \mathbb{C}^4$ , three qubits in  $\mathbb{C}^8$ , and so on. The whole discipline of quantum computing is the study of how to do useful work inside this enormous space while only being able to read out a single classical bit-string at the end.

## 4.2 Qubits, gates, and circuits

A qubit is a quantum system with a two-dimensional state space, conventionally with computational basis  $\{|0\rangle, |1\rangle\}$ . A

general qubit state is

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1.$$

A quantum gate acting on  $n$  qubits is a unitary  $U \in \mathbb{C}^{2^n \times 2^n}$ . The single-qubit Pauli gates and the Hadamard gate were already introduced in Sections 2.13 and 2.14. The two-qubit CNOT (controlled-NOT) flips the target qubit if and only if the control is  $|1\rangle$ :

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

A celebrated result of [Barenco et al. \[1995\]](#) establishes that single-qubit gates together with CNOT form a universal gate set: any  $n$ -qubit unitary can be decomposed (to arbitrary precision) into a finite sequence drawn from this set. The architectural implication is profound: a quantum computer need only implement a handful of gate types reliably to be programmable for any computation.

A quantum circuit is a sequence of gates applied to a register of qubits, typically followed by measurement. We read circuits left-to-right, like classical wiring diagrams—but with the understanding that “wires” carry quantum states evolving by tensor-product unitaries, not classical bits.

**Example 4.1.** *The simplest non-trivial quantum circuit takes a single qubit in state  $|0\rangle$ , applies a Hadamard gate, and measures.*

*The Hadamard produces  $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ , and measurement returns 0 or 1 each with probability  $\frac{1}{2}$ . This is a perfect random bit generator—something a classical computer cannot produce without external entropy.*

### 4.3 *Where does the speed come from?*

This is the question every reader asks, and the honest answer has three parts.

**Superposition** gives a quantum register access to  $2^n$  amplitudes at once. But superposition alone is not enough—a uniform superposition is equivalent in measurement statistics to drawing a random bit-string, no faster than a classical random sampler.

**Interference** is what allows correct answers to be amplified and wrong answers to cancel. Quantum amplitudes can be negative or complex, so two computational paths arriving at the same answer with opposite phases destroy each other. Algorithms are choreographies of constructive and destructive interference, designed to concentrate amplitude on the desired outcome before measurement.

**Entanglement** encodes correlations that have no classical analogue, allowing certain joint computations to proceed without classical communication overhead. Without entanglement, an  $n$ -qubit register decomposes as a product of single-qubit states, and quantum mechanics offers nothing

exponential over a collection of independent classical random variables.

**Remark 4.1.** *A useful slogan: superposition gives the parallelism, entanglement gives the structure, interference gives the answer. Drop any one of the three and the speedup disappears. The chapters that follow on each pillar develop the details.*

The deep theorem behind all of this is that quantum computation can be simulated classically with exponential overhead in the number of qubits, and not less. This is the formal content of the Church–Turing–Deutsch principle introduced by Deutsch [1985]: every physically realizable computation can be efficiently simulated by a universal quantum computer, while not every quantum computation can be efficiently simulated classically.

## 4.4 *The algorithmic landscape*

Of the few dozen quantum algorithms developed over four decades, a small handful matter most for our purposes. We list them here as a road map; later chapters and the introductory survey treat each in depth.

- **Shor’s algorithm** [Shor, 1997] factors integers in polynomial time, breaking RSA and ECC if run on a sufficiently large fault-tolerant quantum computer. It is the headline existential

threat to current public-key infrastructure (Chapter on Shor's algorithm).

- **Grover's algorithm** [Grover, 1996] searches an unsorted space of  $N$  items in  $O(\sqrt{N})$  queries to an oracle. It generalizes to amplitude amplification and is the seed of several quantum Monte Carlo techniques used in option pricing [Stamatopoulos et al., 2020].
- **Quantum amplitude estimation** provides a quadratic speedup over classical Monte Carlo for expectation values. It is the algorithmic core of quantum risk and derivative pricing [Egger et al., 2020; Matsakos and Nield, 2024].
- **HHL** [Harrow et al., 2009] solves systems of linear equations  $Ax = b$  in time polylogarithmic in the matrix dimension, under technical conditions on  $A$  and on data loading. It underlies several quantum machine-learning subroutines.
- **Variational quantum algorithms** (VQE, QAOA) [Cerezo et al., 2021; Farhi et al., 2014; Peruzzo et al., 2014] use a classical optimizer to tune a parameterized quantum circuit, looking for ground states of Hamiltonians that encode optimization or chemistry problems. These are the workhorses of NISQ-era applications, including portfolio optimization.
- **Quantum simulation**—the original motivation [Feynman, 1982]—uses a quantum computer to model physical systems whose Hilbert spaces are too large to simulate classically.



Quantum machine learning sits across many of these categories [[Biamonte et al., 2017](#)]. The picture to keep in mind is that quantum computing offers a small toolbox of *primitives*—search, expectation estimation, eigenvalue finding, linear-system solving—and that financial applications are largely about composing those primitives over a hybrid classical-quantum stack.

## 4.5 *The hardware reality check*

A description of quantum computing that does not mention hardware is misleading. Real quantum computers are noisy, finite, and slow. Single-qubit gates take nanoseconds to microseconds, with error rates near 0.1%. Two-qubit gates are an order of magnitude noisier. Coherence times—how long a qubit can hold an arbitrary state before relaxing—are measured in tens to hundreds of microseconds for current superconducting hardware. A circuit that takes longer than the coherence time will deliver garbage [[Preskill, 2018](#)].

The implication for algorithms is severe. Shor’s algorithm for a 2048-bit RSA key would require millions of logical qubits and trillions of gates with error rate close to  $10^{-15}$ —far beyond any current device [[Nielsen and Chuang, 2010](#)]. The path forward involves quantum error correction (Chapter on Error Correction), which encodes one logical qubit in many physical qubits and uses the redundancy to detect and correct errors below a threshold rate. Surface codes [[Fowler et al., 2012](#); [Kitaev, 2003](#)] are the leading approach.

For the next several years, we will not have fault-tolerant devices. What we have is the NISQ era [[Preskill, 2018](#)]*—*Noisy Intermediate-Scale Quantum hardware, treated in detail in its own chapter. NISQ devices are useful for testing algorithm structures, demonstrating quantum effects, and exploring hybrid workflows on small instances. They are not yet useful for transformative production-scale finance.

## 4.6 *What this book covers, and in what order*

The remainder of the book moves outward from foundations to applications:

1. Hilbert space and the bra–ket notation, the language in which everything else is written.
2. Qubits, superposition, entanglement, and interference—the four conceptual pillars.
3. The Schrödinger equation, which governs unitary evolution and underlies hardware modelling.
4. Hardware platforms, decoherence, error correction, and the NISQ constraint.
5. Qiskit, the software framework that bridges algorithm and hardware.
6. RSA, elliptic-curve cryptography, and Shor’s algorithm—the negative side of the quantum advantage, with direct implications for financial security.

Throughout, we keep the financial perspective in view. The mathematics of quantum mechanics is intricate but not ex-

otic; what makes it powerful for finance is the alignment between its primitives (sampling, expectation, search, eigenvalue finding) and the bottlenecks of computational finance (Monte Carlo, optimization, machine learning). The opening chapter laid out that landscape; the remaining chapters build the machinery to engage with it.



## *Chapter 5*

# *Where Quantum States Live*

### HILBERT SPACE AS THE STAGE OF QUANTUM MECHANICS

Hilbert space is the mathematical home of quantum mechanics. Section [2.6](#) introduced it formally; this chapter walks around the same object from a physics-and-computing point of view. The aim is to translate definitions into intuition: which vectors count as states, what counts as an observable, how composite systems are assembled, and why the geometry of unit vectors is the right setting for quantum probability.

## 5.1 Why a complex Hilbert space?

The motivation for choosing a complex Hilbert space as the state space of a quantum system goes back to [Dirac \[1958\]](#) and [Schrödinger \[1926\]](#). Three properties are essential.

- **Linearity.** Quantum dynamics is linear: if  $|\psi_1\rangle$  and  $|\psi_2\rangle$  are valid states, so is any linear combination. This linearity is what makes superposition possible.
- **Complex amplitudes.** Real numbers cannot account for interference of wave functions with relative phase. Complex amplitudes are the minimal extension that supports both the magnitude (probability) and the phase (interference) of quantum states.
- **Inner product.** An inner product allows us to measure angles, compute overlaps, and define orthogonality. These notions become *probabilities*, *transition amplitudes*, and *distinguishability of states*.

A Hilbert space  $\mathcal{H}$  is thus a complex vector space equipped with a positive-definite inner product, complete with respect to the induced norm. For quantum computing we work almost exclusively in finite-dimensional spaces  $\mathbb{C}^d$ , where the completeness condition is automatic. The infinite-dimensional case ( $L^2$ , Section 2.22) arises only for continuous-variable systems such as a particle's position–momentum space.

## 5.2 States as rays, not vectors

Strictly speaking, a quantum state is not a unit vector but a ray: an equivalence class of vectors differing by a global phase,  $|\psi\rangle \sim e^{i\theta}|\psi\rangle$ . A global phase is physically unobservable because the Born rule depends only on  $|\langle\phi|\psi\rangle|^2$  for any reference  $|\phi\rangle$ . The global phase is invisible.

Relative phases are very visible. The states

$$\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \quad \text{and} \quad \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

differ by a relative sign and are orthogonal:  $\langle+|- \rangle = 0$ . Measuring in the  $|0\rangle, |1\rangle$  basis gives identical statistics (probability  $\frac{1}{2}$  each), but measuring after a Hadamard gate distinguishes them with certainty. Quantum information is encoded in relative, not absolute, phases.

**Remark 5.1.** *In practice we work with unit vectors and remember that physically meaningful operations preserve normalization. The space of physical pure states is the complex projective space  $\mathbb{CP}^{d-1}$ , but the linear formalism of  $\mathcal{H}$  is what we compute with.*

## 5.3 Observables as Hermitian operators

Every measurable quantity (energy, spin component, position) corresponds to a Hermitian operator  $A$  on  $\mathcal{H}$  (Section 2.13). The possible outcomes of measuring  $A$  are its eigenvalues  $\{\lambda_i\}$ , which are real by Hermiticity. The spectral decomposi-

tion (Section 2.15)

$$A = \sum_i \lambda_i P_i, \quad P_i = |\lambda_i\rangle\langle\lambda_i|$$

gives the probability rule: the probability of obtaining  $\lambda_i$  in state  $|\psi\rangle$  is

$$p(\lambda_i) = \langle\psi|P_i|\psi\rangle = |\langle\lambda_i|\psi\rangle|^2.$$

This is the Born rule [Born, 1926] in operator form, and it is the bridge between Hilbert-space geometry and laboratory statistics.

**Example 5.1.** *The Pauli  $\sigma_z$  operator measures the third spin component. Its eigenvalues are  $\pm 1$  with eigenstates  $|0\rangle, |1\rangle$ . For a qubit in state  $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ , the probabilities of obtaining  $\pm 1$  are  $|\alpha|^2, |\beta|^2$ , respectively. After measurement, the state collapses to  $|0\rangle$  or  $|1\rangle$  accordingly.*

The expectation value of  $A$  in state  $|\psi\rangle$  is

$$\langle A \rangle_\psi = \langle\psi|A|\psi\rangle = \sum_i \lambda_i p(\lambda_i).$$

This is the quantum analogue of the classical expected value  $\mathbb{E}[X]$ . In finance, expectations are the central quantity computed by Monte Carlo simulation; the quantum analogue is computed by amplitude estimation, which works directly with  $\langle A \rangle_\psi$  encoded as an amplitude.

## 5.4 Composite systems and tensor products

The state space of a composite system is the tensor product of the components' state spaces. If subsystems  $A$  and  $B$  have spaces  $\mathcal{H}_A$  and  $\mathcal{H}_B$ , the composite space is  $\mathcal{H}_A \otimes \mathcal{H}_B$ , with dimension  $\dim(\mathcal{H}_A) \cdot \dim(\mathcal{H}_B)$ .

This is the source of quantum's exponential scaling. For  $n$  qubits, the joint space is  $(\mathbb{C}^2)^{\otimes n} = \mathbb{C}^{2^n}$ . A general state is

$$|\Psi\rangle = \sum_{x \in \{0,1\}^n} c_x |x\rangle,$$

where  $|x\rangle = |x_1 x_2 \cdots x_n\rangle$  runs over all  $2^n$  classical bit-strings and  $\sum_x |c_x|^2 = 1$ . The number of complex amplitudes needed to specify the state grows exponentially in the number of qubits.

Importantly, not every state in  $\mathcal{H}_A \otimes \mathcal{H}_B$  factorizes as a product  $|\phi\rangle_A \otimes |\chi\rangle_B$ . States that fail to factorize are entangled (Chapter on Entanglement). The existence of non-factorizable states is the formal underpinning of every distinctive quantum phenomenon, from EPR correlations to teleportation to algorithmic speedup.

**Example 5.2.** *The Bell state  $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$  is entangled: there are no  $\alpha_0, \alpha_1, \beta_0, \beta_1$  such that  $|\Phi^+\rangle = (\alpha_0|0\rangle + \alpha_1|1\rangle) \otimes (\beta_0|0\rangle + \beta_1|1\rangle)$ . By contrast,  $\frac{1}{\sqrt{2}}(|00\rangle + |10\rangle) = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |0\rangle$  factorizes; it is a product state.*



## 5.5 *Density operators and mixed states*

Pure states ( $|\psi\rangle$ ) describe systems with complete quantum information. When information is incomplete—because the system has interacted with an environment, or because we are studying a subsystem of a larger entangled state—we use a density operator  $\rho$ :

$$\rho = \sum_i p_i |\psi_i\rangle\langle\psi_i|, \quad p_i \geq 0, \quad \sum_i p_i = 1.$$

This is a positive semi-definite Hermitian operator of unit trace. Pure states correspond to rank-1 density operators; mixed states have rank greater than one. The Born rule generalizes:  $\text{Tr}(P_i \rho)$  is the probability of outcome  $i$ , and  $\text{Tr}(A \rho)$  is the expectation value of observable  $A$ .

Density operators are essential for treating decoherence (Chapter on Decoherence) and for tracking the state of a subsystem within a larger entangled state via the partial trace. They will appear again whenever we discuss noise, error channels, or open-system dynamics.

## 5.6 *The geometry of distinguishability*

Two quantum states are perfectly distinguishable by some measurement if and only if they are orthogonal,  $\langle\phi|\psi\rangle = 0$ . States that are not orthogonal cannot be perfectly distinguished by any single measurement: no procedure can decide with certainty which of two non-orthogonal states a single

copy was prepared in. This is the formal core of the no-cloning theorem—an unknown quantum state cannot be copied—and of the security guarantees of quantum cryptography [Bennett and Brassard, 2014].

For mixed states, the natural distinguishability measure is the trace distance

$$D(\rho, \sigma) = \frac{1}{2} \text{Tr}|\rho - \sigma|,$$

which equals the maximum total-variation distance of measurement outcome distributions over all possible measurements. The closely related fidelity is  $F(\rho, \sigma) = \text{Tr}(\sqrt{\sqrt{\rho}\sigma\sqrt{\rho}})$ , with  $F = 1$  exactly when  $\rho = \sigma$ . These quantities are the standard yardsticks for benchmarking how closely a noisy quantum device implements an intended pure state—a notion we will use repeatedly when discussing hardware fidelity.

## 5.7 Why finite-dimensional Hilbert spaces suffice for computing

Quantum mechanics writ large requires infinite-dimensional Hilbert spaces—a particle’s position has a continuum of values, an oscillator has countably many energy levels. Quantum *computing*, by contrast, deals almost exclusively with  $\mathbb{C}^{2^n}$ . The reason is pragmatic: digital quantum computation discretises every degree of freedom into two-level systems (qubits) for the same reason classical computation reduces every signal to bits. The exponentially large but *finite* space

$\mathbb{C}^{2^n}$  already contains enough room to encode universal computation.

Continuous-variable quantum computing—using infinite-dimensional bosonic modes as the basic unit—exists and is interesting, particularly for quantum simulation of physics. But for finance applications and the algorithms in this book, finite-dimensional Hilbert spaces are the natural and sufficient setting.

## 5.8 *What to carry into the next chapters*

The take-aways from this chapter, used everywhere from here on:

- Quantum states are unit vectors (or rays) in a complex Hilbert space; global phase is unobservable, relative phase is everything.
- Observables are Hermitian operators; possible outcomes are eigenvalues, probabilities follow the Born rule.
- Composite systems live in tensor products; entanglement is the failure of factorization.
- Density operators describe mixed states and partial information.
- Distinguishability is geometric: orthogonal states are perfectly distinguishable, non-orthogonal ones are not.

The next chapter introduces bra-ket notation—Dirac’s compact symbolic language for everything we have just written down—which makes quantum calculations remarkably readable once the eye is trained.



## Chapter 6

# *Dirac's Brackets*

### THE NOTATION THAT MADE QUANTUM MECHANICS READABLE

In 1939 Paul Dirac introduced a notation so well suited to quantum mechanics that almost every physicist has used it ever since [Dirac, 1958]. Bra-ket notation is not a different theory; it is the same linear algebra of the Hilbert-space and operator chapters written in a way that hides nothing essential and shows the structure of expressions at a glance. Once it clicks, the formalism feels almost typographic—each symbol points to its meaning.

## 6.1 Kets and bras

A column vector in  $\mathbb{C}^d$  is written as a ket:

$$|\psi\rangle = \begin{pmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_d \end{pmatrix}.$$

The symbol inside the brackets labels the state, not its components:  $|0\rangle, |1\rangle, |\psi\rangle, |+\rangle$  are all kets. The corresponding bra is the conjugate transpose:

$$\langle\psi| = |\psi\rangle^\dagger = (\bar{\alpha}_1, \bar{\alpha}_2, \dots, \bar{\alpha}_d).$$

Bras act on kets to produce complex numbers (inner products) and on operators to produce new bras. The map  $|\psi\rangle \mapsto \langle\psi|$  is anti-linear:  $|\alpha\psi + \beta\phi\rangle \mapsto \bar{\alpha}\langle\psi| + \bar{\beta}\langle\phi|$ . The Riesz representation theorem (Section 2.6) guarantees a one-to-one correspondence between kets and bras in any Hilbert space.

**Example 6.1.** *The computational basis vectors of a qubit are written*

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad \langle 0| = (1, 0), \quad \langle 1| = (0, 1).$$

*Any qubit state is  $\alpha|0\rangle + \beta|1\rangle$  for complex  $\alpha, \beta$  with  $|\alpha|^2 + |\beta|^2 = 1$ .*

## 6.2 Inner products: bra-keting

The inner product of two states is written by joining bra and ket:

$$\langle\phi|\psi\rangle = \sum_i \bar{\phi}_i \psi_i \in \mathbb{C}.$$

The notation is wonderfully suggestive: “ $\langle\phi$  on  $\psi\rangle$ .” Several identities follow at sight:

- $\langle\phi|\psi\rangle = \overline{\langle\psi|\phi\rangle}$  (conjugate symmetry).
- $\langle\psi|\psi\rangle = \|\psi\|^2 \geq 0$ , with equality iff  $|\psi\rangle = 0$ .
- Linearity in the right slot:  $\langle\phi|(\alpha|\psi\rangle + \beta|\chi\rangle) = \alpha\langle\phi|\psi\rangle + \beta\langle\phi|\chi\rangle$ .
- Anti-linearity in the left slot:  $(\langle\phi|\alpha + \langle\chi|\beta)|\psi\rangle$  has  $\bar{\alpha}, \bar{\beta}$  on the outside.

The Cauchy–Schwarz inequality reads  $|\langle\phi|\psi\rangle| \leq \|\phi\| \cdot \|\psi\|$ .

**Example 6.2.** For  $|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$  and  $|\phi\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ , the inner product is

$$\langle\phi|\psi\rangle = \frac{1}{2}(\langle 0|0\rangle - \langle 0|1\rangle + \langle 1|0\rangle - \langle 1|1\rangle) = \frac{1}{2}(1 - 0 + 0 - 1) = 0$$

*The states are orthogonal: a measurement in the  $\{|\phi\rangle, |\psi\rangle\}$  basis distinguishes them perfectly.*

## 6.3 Outer products: ket-bras as operators

Joining a ket and a bra in the opposite order yields an outer product, which is a matrix—an operator:

$$|\psi\rangle\langle\phi| = \begin{pmatrix} \psi_1 \\ \psi_2 \\ \vdots \\ \psi_d \end{pmatrix} (\bar{\phi}_1, \bar{\phi}_2, \dots, \bar{\phi}_d) = (\psi_i \bar{\phi}_j)_{ij}.$$

This operator acts on a ket  $|\chi\rangle$  by first computing the bracket  $\langle\phi|\chi\rangle$  (a scalar) and then producing  $|\psi\rangle\langle\phi|\chi\rangle = \langle\phi|\chi\rangle|\psi\rangle$ . The right-hand side is a ket; the operator sends  $|\chi\rangle$  to  $|\psi\rangle$  scaled by the overlap  $\langle\phi|\chi\rangle$ .

A particularly important case is the projector onto a unit vector  $|\psi\rangle$ :

$$P_\psi = |\psi\rangle\langle\psi|.$$

It satisfies  $P_\psi^2 = P_\psi$  and  $P_\psi^\dagger = P_\psi$ . The action  $P_\psi|\chi\rangle = \langle\psi|\chi\rangle|\psi\rangle$  projects  $|\chi\rangle$  onto the line through  $|\psi\rangle$ .

**Example 6.3.**  $\sigma_z = |0\rangle\langle 0| - |1\rangle\langle 1|$ ,  $\sigma_x = |0\rangle\langle 1| + |1\rangle\langle 0|$ , and  $I = |0\rangle\langle 0| + |1\rangle\langle 1|$ . The Hadamard gate is  $H = \frac{1}{\sqrt{2}}(|+\rangle\langle 0| + |-\rangle\langle 1|) = \frac{1}{\sqrt{2}}(|0\rangle\langle 0| + |0\rangle\langle 1| + |1\rangle\langle 0| - |1\rangle\langle 1|)$ .

## 6.4 The resolution of the identity

If  $\{|e_i\rangle\}$  is an orthonormal basis, then

$$\sum_i |e_i\rangle\langle e_i| = I.$$



This identity is called the resolution of the identity or the completeness relation. It is the bra-ket form of the statement that a vector equals the sum of its projections onto basis vectors:

$$|\psi\rangle = I|\psi\rangle = \sum_i |e_i\rangle \langle e_i|\psi\rangle = \sum_i \langle e_i|\psi\rangle |e_i\rangle.$$

The coefficients  $\langle e_i|\psi\rangle$  are the components in the chosen basis.

The completeness relation is the workhorse of bra-ket computation. It allows the insertion of  $I = \sum_i |e_i\rangle \langle e_i|$  at any point in an expression—an algebraic move equivalent to “inserting basis vectors and summing”—without changing the result. It is the same trick used to derive matrix representations of operators in Section 2.27.

## 6.5 Operators in spectral form

A Hermitian operator  $A$  with eigenvalues  $\lambda_i$  and eigenvectors  $|\lambda_i\rangle$  has the bra-ket spectral form (Section 2.15)

$$A = \sum_i \lambda_i |\lambda_i\rangle \langle \lambda_i|.$$

This form makes functions of  $A$  transparent:

$$f(A) = \sum_i f(\lambda_i) |\lambda_i\rangle \langle \lambda_i|.$$

For the Pauli  $\sigma_z$  with eigenvalues  $\pm 1$  and eigenvectors  $|0\rangle, |1\rangle$ :

$$e^{i\theta\sigma_z} = e^{i\theta}|0\rangle\langle 0| + e^{-i\theta}|1\rangle\langle 1| = \cos\theta I + i\sin\theta\sigma_z.$$

Every unitary applied to a qubit can be written in such closed form; the Schrödinger evolution of a qubit under a time-independent Hamiltonian is just  $\exp(-iHt/\hbar)$ , expanded on the eigenbasis of  $H$ .

## 6.6 *Tensor products in bra-ket*

For composite systems, the tensor product symbol is often suppressed:

$$|\psi\rangle_A \otimes |\phi\rangle_B = |\psi\rangle|\phi\rangle = |\psi, \phi\rangle = |\psi\phi\rangle.$$

For two qubits, the computational basis is  $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$ . Inner products factorize across tensor products:

$$(\langle\psi_1| \otimes \langle\phi_1|)(|\psi_2\rangle \otimes |\phi_2\rangle) = \langle\psi_1|\psi_2\rangle \langle\phi_1|\phi_2\rangle.$$

Operators acting on different subsystems commute under the tensor product:

$$(A \otimes I)(I \otimes B) = (I \otimes B)(A \otimes I) = A \otimes B.$$

**Example 6.4.** For the Bell state  $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ , the inner product with  $|01\rangle$  is

$$\langle 01|\Phi^+\rangle = \frac{1}{\sqrt{2}}(\langle 01|00\rangle + \langle 01|11\rangle) = \frac{1}{\sqrt{2}}(0 + 0) = 0.$$

The amplitude of  $|01\rangle$  in  $|\Phi^+\rangle$  is zero—measurement never gives 01.

## 6.7 Expectation values, in three notations

The expectation value of an observable  $A$  in a state  $|\psi\rangle$  is written

$$\langle A \rangle_\psi = \langle \psi | A | \psi \rangle.$$

Reading inside-out:  $A|\psi\rangle$  is a ket; bra'ing with  $\langle\psi|$  gives a scalar. For a mixed state  $\rho = \sum_i p_i |\psi_i\rangle\langle\psi_i|$ , the expectation generalises to

$$\langle A \rangle_\rho = \text{Tr}(\rho A) = \sum_i p_i \langle \psi_i | A | \psi_i \rangle.$$

The cyclic property of the trace,  $\text{Tr}(AB) = \text{Tr}(BA)$ , gives a third (and very useful) view:  $\langle A \rangle_\rho = \text{Tr}(A\rho)$ .

In financial applications,  $A$  is often a payoff operator: the expected payoff is  $\text{Tr}(\rho A)$  for an appropriately prepared  $\rho$ . Quantum amplitude estimation reads this expectation value with quadratically fewer queries than the classical  $1/\sqrt{N}$  Monte Carlo rate [Egger et al., 2020].

## 6.8 A small grammar of bra-ket

For quick reference, the standard moves of the formalism are:

- $\langle\phi|\psi\rangle$ : a scalar (inner product).
- $|\psi\rangle\langle\phi|$ : an operator (outer product).
- $\langle\psi|A|\phi\rangle$ : a scalar (matrix element of  $A$ ).

- $A|\psi\rangle$ : a ket (operator acting on state).
- $\langle\psi|A$ : a bra (operator transposed onto bra).
- $\sum_i |e_i\rangle\langle e_i| = I$ : completeness, may be inserted anywhere.
- $A = \sum_i \lambda_i |\lambda_i\rangle\langle\lambda_i|$ : spectral decomposition.

**Remark 6.1.** *A useful rule of thumb when reading bra-ket: parse from inside outward. Identify scalars first (bra-kets), then collapse them. Operators acting on kets are kets, operators acting from the right on bras are bras. With practice, expressions become almost diagrammatic.*

The notation will return on every page of every chapter that follows. The investment in fluency pays off immediately: the same expressions that look intimidating in matrix form ( $U\rho U^\dagger$ ,  $\text{Tr}_B(\rho_{AB})$ ,  $\sum_x \langle x|U|\psi\rangle|x\rangle$ ) read at a glance once the eye is trained.



## Chapter 7

# *The Qubit*

### INFORMATION'S QUANTUM ATOM AND ITS BLOCH-SPHERE GEOMETRY

If the classical bit is a coin lying flat on a table—heads or tails—the qubit is a coin spinning in the air, described by a direction in three-dimensional space. This chapter develops the qubit carefully: its mathematical state, its geometric picture on the Bloch sphere, the gates that move it around, and the measurement that pins it down. Everything afterwards—multi-qubit registers, entanglement, algorithms—is built from this one object.

## 7.1 What a qubit is

A qubit is a quantum system with a two-dimensional Hilbert space  $\mathcal{H} = \mathbb{C}^2$ . Its standard basis vectors are

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

called the computational basis. A general qubit state is

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1.$$

The complex numbers  $\alpha, \beta$  are the amplitudes. By the Born rule, measuring in the computational basis returns 0 with probability  $|\alpha|^2$  and 1 with probability  $|\beta|^2$ , after which the state collapses to  $|0\rangle$  or  $|1\rangle$  accordingly.

**Example 7.1.** *Three useful single-qubit states appear repeatedly:*

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \quad |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle), \quad |i\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle)$$

*Each is a uniform superposition in some sense, but they are mutually distinct and detected by different measurements (the  $\sigma_x$ ,  $\sigma_y$ , and  $\sigma_z$  bases, respectively).*

## 7.2 Why two dimensions, and not three or four?

The qubit's two-dimensionality is a design choice for digital quantum computation, not a law of nature. Many physical

systems have more than two relevant energy levels—an atom has infinitely many—but for computation we isolate two of them and ignore the rest. The reasons mirror the choice of binary classical bits:

- **Universality.** Two-level systems plus a small set of gates suffice for universal quantum computation [Barenco et al., 1995]; higher-dimensional “qudits” offer no theoretical capability that qubits lack.
- **Engineering.** Two-level subspaces are easier to isolate. A transmon superconducting qubit, for instance, is a slightly anharmonic oscillator; the lowest two energy levels are well-separated from the third, allowing operations to remain in the qubit subspace [Koch et al., 2007].
- **Modularity.** An  $n$ -qubit register has  $2^n$  basis states, mirroring the  $2^n$  bit-strings of  $n$  classical bits and allowing direct compatibility with classical algorithm structures.

Higher-dimensional “qutrits” (three-level) and “qudits” ( $d$ -level) are studied for niche applications, but the dominant paradigm is the qubit.

## 7.3 *The Bloch sphere*

Up to global phase, every qubit state can be written

$$|\psi\rangle = \cos\frac{\theta}{2}|0\rangle + e^{i\varphi}\sin\frac{\theta}{2}|1\rangle, \quad \theta \in [0, \pi], \varphi \in [0, 2\pi).$$

This parameterization maps every pure qubit state to a point on the surface of a unit sphere—the Bloch sphere—with polar angle  $\theta$  and azimuthal angle  $\varphi$ . The six cardinal points correspond to:

- North pole ( $\theta = 0$ ):  $|0\rangle$ .
- South pole ( $\theta = \pi$ ):  $|1\rangle$ .
- $+x$  axis ( $\theta = \pi/2, \varphi = 0$ ):  $|+\rangle$ .
- $-x$  axis ( $\theta = \pi/2, \varphi = \pi$ ):  $|-\rangle$ .
- $+y$  axis ( $\theta = \pi/2, \varphi = \pi/2$ ):  $|i\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle)$ .
- $-y$  axis ( $\theta = \pi/2, \varphi = 3\pi/2$ ):  $|-i\rangle = \frac{1}{\sqrt{2}}(|0\rangle - i|1\rangle)$ .

The Bloch sphere is a wonderfully concrete picture. Antipodal points are orthogonal states. The expectation values of the Pauli operators are the Cartesian coordinates of the state's Bloch vector:

$$\langle\sigma_x\rangle = \sin\theta\cos\varphi, \quad \langle\sigma_y\rangle = \sin\theta\sin\varphi, \quad \langle\sigma_z\rangle = \cos\theta.$$

A mixed state lives inside the ball: its Bloch vector  $\vec{r}$  satisfies  $\|\vec{r}\| < 1$ , with the centre of the ball representing the maximally mixed state  $\rho = I/2$ .

**Remark 7.1.** *The Bloch sphere does not generalize cleanly to many qubits. An  $n$ -qubit pure state has  $2 \cdot 2^n - 2$  real degrees of freedom (after normalization and global phase), so even two*



qubits inhabit a six-dimensional space. The Bloch picture is genuinely a single-qubit aid. Use it accordingly.

## 7.4 Single-qubit gates as rotations

A single-qubit unitary  $U$  acts on the Bloch sphere as a rigid rotation. This is the geometric content of the fact that  $SU(2)$  is the double cover of  $SO(3)$ : every  $U \in SU(2)$  corresponds to a rotation in three-dimensional space.

The standard single-qubit gates are:

$$X = \sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \sigma_y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

each a  $\pi$ -rotation about its respective Bloch axis. The Hadamard gate  $H$  is a  $\pi$ -rotation about the axis  $(\hat{x} + \hat{z})/\sqrt{2}$ , swapping the  $X$  and  $Z$  axes; it maps  $|0\rangle$  to  $|+\rangle$  and  $|1\rangle$  to  $|-\rangle$ . Continuous rotations about an axis  $\hat{n}$  by angle  $\theta$  are

$$R_{\hat{n}}(\theta) = e^{-i\theta \hat{n} \cdot \vec{\sigma}/2} = \cos \frac{\theta}{2} I - i \sin \frac{\theta}{2} (\hat{n} \cdot \vec{\sigma}).$$

The phase gate  $S = \text{diag}(1, i) = R_z(\pi/2) \cdot e^{i\pi/4}$  and the  $T$ -gate  $T = \text{diag}(1, e^{i\pi/4})$  are the canonical fixed rotations that, together with Hadamard and CNOT, generate a discrete universal gate set (the Clifford+T set).

**Example 7.2.** *A Hadamard gate followed by a  $Z$  gate followed by another Hadamard equals an  $X$  gate:  $HZH = X$ . Geometrically: rotate the sphere so the  $Z$  axis points along  $X$ , do a  $\pi$  rotation about the original  $Z$  direction (now  $X$ ), rotate back.*

## 7.5 Measurement and what it does

Measuring a qubit in the computational basis is described by the projectors  $P_0 = |0\rangle\langle 0|$  and  $P_1 = |1\rangle\langle 1|$ . The probability of outcome 0 is  $\langle\psi|P_0|\psi\rangle = |\alpha|^2$ ; conditional on this outcome the post-measurement state is  $|0\rangle$ . Equivalent for outcome 1.

Measurement in a different basis  $\{|\phi_0\rangle, |\phi_1\rangle\}$  can always be reduced to a basis rotation followed by a computational-basis measurement. Specifically, if  $U = |0\rangle\langle\phi_0| + |1\rangle\langle\phi_1|$  is the unitary mapping the alternative basis to the computational basis, then measuring  $|\psi\rangle$  in  $\{|\phi_0\rangle, |\phi_1\rangle\}$  is equivalent to applying  $U$  and then measuring in the computational basis. This is why current quantum hardware typically supports only computational-basis measurement: any other basis is obtained “for free” by a preceding gate.

**Remark 7.2.** *Measurement is irreversible. Once a qubit is measured, the superposition is destroyed; there is no recovering the pre-measurement state from the outcome. Algorithms therefore have a single climactic measurement at the end, with all the unitary cleverness preceding it.*

The no-cloning theorem [Nielsen and Chuang, 2010] is closely related: there is no unitary  $U$  such that  $U|\psi\rangle|0\rangle = |\psi\rangle|\psi\rangle$  for every  $|\psi\rangle$ . The proof is a one-liner: such a  $U$  would be linear, but the cloning operation  $|\psi\rangle \mapsto |\psi\rangle \otimes |\psi\rangle$  is nonlinear in  $|\psi\rangle$  (since  $(\alpha|\phi\rangle + \beta|\chi\rangle)^{\otimes 2} \neq \alpha|\phi\rangle^{\otimes 2} + \beta|\chi\rangle^{\otimes 2}$ ). No-cloning is the deep reason quantum signals cannot be amplified the

way classical ones can, and it underlies both quantum cryptography and the difficulty of quantum error correction.

## 7.6 Multi-qubit registers

An  $n$ -qubit register lives in the tensor-product space  $(\mathbb{C}^2)^{\otimes n} = \mathbb{C}^{2^n}$ . The computational basis  $\{|x\rangle : x \in \{0,1\}^n\}$  is indexed by classical bit-strings. A general state is

$$|\Psi\rangle = \sum_{x \in \{0,1\}^n} c_x |x\rangle, \quad \sum_x |c_x|^2 = 1.$$

The number of complex amplitudes is  $2^n$ . For  $n = 10$ , that is 1024; for  $n = 50$ , over a quadrillion; for  $n = 300$ , more than the number of atoms in the observable universe. Yet measuring the register returns only an  $n$ -bit string. The art of quantum algorithm design is, again, to manipulate  $2^n$  amplitudes so that the right bit-string falls out with high probability.

**Example 7.3.** *Two qubits prepared in  $|00\rangle$ , with a Hadamard on the first qubit and a CNOT to the second, produce*

$$\text{CNOT}(H \otimes I)|00\rangle = \text{CNOT} \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle) = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

*the canonical Bell state of Chapter on Entanglement.*

## 7.7 Physical realizations

A qubit is an abstract object; its physical realization can be almost anything with two well-isolated quantum states. The leading current platforms include:

- **Superconducting transmons.** An anharmonic LC oscillator etched on a chip, operated at millikelvin temperatures; the two lowest energy levels are the qubit [Koch et al., 2007]. IBM and Google use this platform.
- **Trapped ions.** The electronic states of an ion held in an electromagnetic trap; qubits are addressed and entangled with laser pulses [Cirac and Zoller, 1995]. IonQ and Quantinuum lead this technology.
- **Neutral atoms.** Hyperfine states of neutral atoms held in optical-tweezer arrays; entangled via Rydberg-state interactions. QuEra and Pasqal lead this technology.
- **Photonic qubits.** Polarisation or path of single photons, manipulated by linear optics; used by PsiQuantum and Xanadu.
- **Topological qubits.** Encoded in topologically protected states of matter (e.g., Majorana zero modes); the Microsoft Station Q vision, still pre-demonstration at meaningful scale.

The hardware chapter develops these in more detail. The point here is that the abstract qubit framework holds across platforms: a state vector in  $\mathbb{C}^2$ , gates as unitaries, measurements as projectors, with platform-specific gate sets and error characteristics.

**Remark 7.3.** DiVincenzo’s five criteria [DiVincenzo, 2000] for a viable physical qubit are: (1) scalable two-level systems, (2) initialization to a known state, (3) long coherence relative to

gate time, (4) a universal gate set, (5) qubit-specific measurement. Every platform must satisfy these, and the engineering competition is over how well.

## 7.8 A pocket reference

For the chapters that follow, the essential qubit facts are:

- State: unit vector in  $\mathbb{C}^2$ , parameterized as  $\cos(\theta/2)|0\rangle + e^{i\varphi}\sin(\theta/2)|1\rangle$ .
- Geometry: pure states on the Bloch sphere, mixed states inside the Bloch ball.
- Gates: unitaries in  $U(2)$ , geometrically rotations of the Bloch sphere; generated by  $\{H, T, \text{CNOT}\}$  together with one other qubit.
- Measurement: probabilistic projection onto basis states, irreversibly destroying superposition.
- Composition:  $n$  qubits live in  $\mathbb{C}^{2^n}$ , exponential in  $n$ .
- Cloning: forbidden by linearity of quantum mechanics.

With these in hand, we are ready for the three phenomena that make the qubit *quantum*: superposition, entanglement, and interference.



## Chapter 8

# Superposition

MANY THINGS AT ONCE, UNTIL YOU LOOK

Superposition is the headline quantum phenomenon—the one that makes for good popular science and bad popular science in equal measure. The mathematics is straightforward: it is just the linearity of the state space. The conceptual content is sharper than its popular versions suggest, and worth getting right.

### 8.1 *The principle, plainly*

The superposition principle states that if  $|\psi_1\rangle$  and  $|\psi_2\rangle$  are valid states of a quantum system, then so is any linear combination

$$|\psi\rangle = \alpha|\psi_1\rangle + \beta|\psi_2\rangle, \quad \alpha, \beta \in \mathbb{C},$$

subject to normalization  $\|\psi\|^2 = 1$ . The complex coefficients are amplitudes; the squared magnitudes are probabilities of obtaining each component upon measurement in the  $\{|\psi_1\rangle, |\psi_2\rangle\}$  basis (when those vectors are orthogonal).

A qubit prepared in  $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$  is in superposition of  $|0\rangle$  and  $|1\rangle$ . The often-repeated phrase “the qubit is in both states at once” is, strictly speaking, misleading. The qubit is in *one* state— $|+\rangle$ —which happens to be a non-trivial linear combination of  $|0\rangle$  and  $|1\rangle$ . The state  $|+\rangle$  is just as definite, in the quantum sense, as  $|0\rangle$  itself.

**Remark 8.1.** *A better intuition:  $|0\rangle$  and  $|1\rangle$  are two preferred directions, like “up” and “right.” The state  $|+\rangle$  is “upper-right.” Calling  $|+\rangle$  “both up and right at the same time” is technically true and conceptually confusing. It is one direction, equally tilted between two of our chosen labels.*

## 8.2 Where does superposition come from?

Three operational paths produce superpositions in a quantum computer:

- **Hadamard from a basis state.**  $H|0\rangle = |+\rangle$  and  $H|1\rangle = |-\rangle$ . A column of Hadamards on  $n$  qubits initialised in  $|0\rangle^{\otimes n}$  produces the uniform superposition

$$H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle.$$

This is the standard uniform superposition starting point for many quantum algorithms.

- **Continuous rotation.** A rotation  $R_y(\theta)$  applied to  $|0\rangle$  produces  $\cos(\theta/2)|0\rangle + \sin(\theta/2)|1\rangle$ , a superposition with controllable amplitude.
- **Phase encoding.** Applying  $R_z(\varphi)$  to  $|+\rangle$  produces  $\frac{1}{\sqrt{2}}(|0\rangle + e^{i\varphi}|1\rangle)$ , encoding a real-valued parameter  $\varphi$  in the relative phase of a superposition.

In a typical quantum algorithm, the first step is to put an input register into superposition over all bit-strings of interest; the body of the algorithm then evaluates a function in superposition and rearranges amplitudes; the final step is measurement.

**Example 8.1.** *The Deutsch–Jozsa algorithm [Deutsch, 1985] prepares  $n$  qubits in the uniform superposition  $H^{\otimes n}|0\rangle^{\otimes n}$ , then queries a function  $f : \{0,1\}^n \rightarrow \{0,1\}$  that is either constant or balanced. After a single quantum query and a final Hadamard, measurement returns  $0^n$  if and only if  $f$  is constant. Classical determination requires up to  $2^{n-1} + 1$  queries; the quantum advantage on this oracle is exponential.*

### 8.3 Quantum parallelism: powerful but tricky

A function  $f : \{0,1\}^n \rightarrow \{0,1\}$  can be implemented as a unitary  $U_f$  acting on  $n+1$  qubits as  $U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$ . If



we feed  $U_f$  the input register in uniform superposition,

$$U_f\left(\frac{1}{\sqrt{2^n}}\sum_x|x\rangle\right)|0\rangle=\frac{1}{\sqrt{2^n}}\sum_x|x\rangle|f(x)\rangle,$$

we have, in one circuit invocation, evaluated  $f$  on all  $2^n$  inputs simultaneously. This is quantum parallelism, and it is impressive on paper.

But—a crucial but—measuring the output register returns a single  $(x, f(x))$  pair, drawn uniformly. So the brute-force gain of parallelism is wiped out by the bottleneck of measurement: we cannot read  $2^n$  values from  $2^n$  amplitudes in a single shot. Quantum algorithms succeed only when they additionally rearrange amplitudes via interference so that the measurement is biased toward a useful outcome (Chapter on Interference).

**Remark 8.2.** *Quantum parallelism alone is not where the speedup lives. It is necessary but not sufficient. Every quantum algorithm with provable advantage uses parallelism to generate a structured superposition, then uses interference to concentrate amplitude on the answer. Without the second step, the algorithm is a costly random sampler.*

## 8.4 Examples in finance: data loading and amplitude encoding

In quantum algorithms for finance, the very first step is often the construction of a superposition that encodes a probability

distribution. To compute the expected payoff of an option under a risk-neutral price distribution  $p(x)$ , we want to prepare

$$|\psi_p\rangle = \sum_x \sqrt{p(x)} |x\rangle,$$

so that measuring in the computational basis reproduces samples from  $p(x)$ . This is amplitude encoding of a classical distribution; the bottleneck is constructing the unitary that produces  $|\psi_p\rangle$  from  $|0\rangle$ .

For specific distributions (uniform, Gaussian-like, log-normal) there are efficient circuit constructions. For arbitrary distributions, generic loading takes time exponential in the number of qubits—the well-known input problem of quantum algorithms [Biamonte et al., 2017]. Resolving the input problem efficiently is one of the active research fronts of quantum machine learning and quantum finance.

Stamatopoulos et al. [2020] construct circuits for option pricing under log-normal price distributions; the encoding circuit is the costly part of their pipeline. Subsequent quantum amplitude estimation reads the expected payoff with a quadratic speedup over classical Monte Carlo—an advantage that only materialises once the distribution is loaded.

## 8.5 *Measurement collapse and the basis choice*

A superposition state  $\alpha|0\rangle + \beta|1\rangle$  measured in the computational basis collapses to  $|0\rangle$  or  $|1\rangle$ . But it could also be

measured in the  $\{|+\rangle, |-\rangle\}$  basis, in which case it would collapse to  $|+\rangle$  or  $|-\rangle$  instead. The probabilities follow the same Born rule applied to the new basis.

What is the qubit “really” in superposition of? The honest answer is: the question presupposes a basis, and the basis is a choice of the experimenter. The state  $|+\rangle$  is a superposition of  $|0\rangle$  and  $|1\rangle$ , and simultaneously a single basis state in the  $\{|+\rangle, |-\rangle\}$  basis with no superposition at all. “Superposition” is basis-dependent; “state” is basis-independent.

**Example 8.2.** *Schrödinger’s cat is in superposition of “alive” and “dead” in one preferred basis. In a different (and physically unmotivated) basis, the same state might be a single basis vector. The cat doesn’t change; the description does.*

## 8.6 Coherent superposition versus statistical mixture

A frequent confusion: is a 50/50 superposition the same as a 50/50 classical mixture? The answer is no, and the distinction is central to everything quantum.

A coherent superposition  $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$  is a pure state. The density operator is

$$|+\rangle\langle+| = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}.$$

A statistical mixture of  $|0\rangle$  with probability  $\frac{1}{2}$  and  $|1\rangle$  with

probability  $\frac{1}{2}$  is the mixed state

$$\rho_{\text{mix}} = \frac{1}{2}|0\rangle\langle 0| + \frac{1}{2}|1\rangle\langle 1| = \frac{1}{2} \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \frac{I}{2}.$$

The diagonal entries are identical—both give 50/50 statistics for a computational-basis measurement. But the off-diagonal entries differ. Those off-diagonals are precisely the coherences, and they are what make interference possible. Apply a Hadamard:

$$H|+\rangle\langle +|H = |0\rangle\langle 0|, \quad H\rho_{\text{mix}}H = \rho_{\text{mix}}.$$

The coherent superposition produces  $|0\rangle$  with certainty after a Hadamard; the mixture remains 50/50. Decoherence (Chapter on Decoherence) is the process by which a coherent superposition turns into a statistical mixture through coupling to an environment.

## 8.7 *What superposition is, and what it isn't*

A short list of common misconceptions, and their corrections:

- “The qubit is in both states simultaneously.” Misleading. The qubit is in one state, which is a linear combination of basis states.
- “Superposition gives exponential parallelism for free.” Not by itself. Parallelism without interference is a random sampler.

- “A 50/50 superposition is a 50/50 random bit.” Not the same; the difference is the coherence, which interference exploits.
- “Superposition collapses when you look.” Yes, but only along the measurement basis you chose; in another basis the same outcome carries less information.

The cleaner picture: superposition is just the linearity of the quantum state space. Its power comes not from the linearity itself but from what unitary evolution and interference can do with it. The next chapter on entanglement and the chapter on interference fill in what those mechanisms look like.



## Chapter 9

# Entanglement

### CORRELATIONS WITHOUT A CAUSE—AND WHY ALGORITHMS NEED THEM

If superposition is quantum's most famous concept, entanglement is its most powerful. The word was coined by Schrödinger to describe the phenomenon that he considered “not one but rather the characteristic trait of quantum mechanics.” Einstein, who never accepted it, called it “spooky action at a distance.” Bell turned the philosophical disagreement into a falsifiable prediction [Bell, 1964], and Aspect's experiments [Aspect et al., 1982] settled the question in nature's favour: entanglement is real, and no local theory of hidden variables can reproduce its statistics.

## 9.1 Entanglement, defined

A pure state  $|\Psi\rangle \in \mathcal{H}_A \otimes \mathcal{H}_B$  of a bipartite system is separable (a product state) if it can be written as

$$|\Psi\rangle = |\phi\rangle_A \otimes |\chi\rangle_B$$

for some  $|\phi\rangle_A \in \mathcal{H}_A$  and  $|\chi\rangle_B \in \mathcal{H}_B$ . Otherwise it is entangled. For mixed states, separability means the density operator can be written as  $\rho_{AB} = \sum_i p_i \rho_A^{(i)} \otimes \rho_B^{(i)}$  with  $p_i \geq 0$ ; otherwise the state is entangled.

The mathematical definition is simple. What it implies is striking: an entangled state cannot be described by giving the state of  $A$  and the state of  $B$  separately. The whole carries information that the parts do not.

## 9.2 The Bell states

The four Bell states are an orthonormal basis of the two-qubit Hilbert space, consisting entirely of maximally entangled states:

$$\begin{aligned} |\Phi^+\rangle &= \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle), & |\Phi^-\rangle &= \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle), \\ |\Psi^+\rangle &= \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle), & |\Psi^-\rangle &= \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle). \end{aligned}$$

Each can be created from  $|00\rangle$  by a Hadamard on the first qubit followed by a CNOT with the first qubit as control and the second as target, possibly preceded by single-qubit operations to select among the four. The Bell states are the simplest

non-trivial entangled states, and the canonical resource for quantum communication tasks: superdense coding, quantum teleportation, and entanglement-based key distribution [Nielsen and Chuang, 2010; Bennett and Brassard, 2014].

**Example 9.1.** *Measuring  $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$  on the first qubit yields 0 or 1 with equal probability  $\frac{1}{2}$ . Conditional on outcome 0, the second qubit collapses to  $|0\rangle$ ; conditional on outcome 1, to  $|1\rangle$ . The two outcomes are perfectly correlated, regardless of the spatial separation of the qubits.*

### 9.3 Why Einstein objected

The EPR paradox [Einstein et al., 1935] argued that entanglement implied either faster-than-light influence or an incomplete description of physical reality. The argument: if Alice and Bob share  $|\Phi^+\rangle$  and Alice measures, she instantaneously “knows” Bob’s outcome regardless of distance. To Einstein, this was inadmissible, so quantum mechanics had to be incomplete; some additional “hidden variable” must determine the outcomes ahead of time.

Bell [Bell, 1964] translated this disagreement into a mathematical inequality satisfied by any local-hidden-variable theory, but violated by quantum mechanics. Experiments [Aspect et al., 1982] have repeatedly confirmed the quantum predictions to many standard deviations, ruling out local-hidden-variable theories. The 2022 Nobel Prize in Physics was awarded to Aspect, Clauser, and Zeilinger for this body



of work.

Entanglement does not transmit information faster than light. Alice's measurement outcomes, viewed alone, are random; she has no way to signal Bob through her choice of measurement basis. What she can do, via classical communication of her outcomes, is share *correlations* that have no classical explanation. This is the formal content of the no-signalling theorem.

**Remark 9.1.** *The cleanest framing: quantum correlations are stronger than classical, but weaker than the strongest correlations that would allow signalling. Entanglement is the precise way nature draws the line.*

## 9.4 Quantifying entanglement

For a bipartite pure state  $|\Psi\rangle_{AB}$ , the standard measure of entanglement is the entropy of entanglement: the von Neumann entropy of the reduced density operator

$$\rho_A = \text{Tr}_B |\Psi\rangle\langle\Psi|, \quad S(\rho_A) = -\text{Tr}(\rho_A \log_2 \rho_A).$$

For a separable pure state,  $\rho_A$  is itself pure and  $S(\rho_A) = 0$ . For a maximally entangled state of two  $d$ -dimensional systems,  $\rho_A = I/d$  is maximally mixed and  $S(\rho_A) = \log_2 d$ . For the Bell states,  $S = 1$  bit (or “ebit”).

The Schmidt decomposition sharpens this further: any bipar-

tite pure state can be written

$$|\Psi\rangle = \sum_i \sqrt{\lambda_i} |u_i\rangle_A \otimes |v_i\rangle_B,$$

where  $\{|u_i\rangle\}, \{|v_i\rangle\}$  are orthonormal bases of the two subsystems and  $\lambda_i \geq 0$  are the Schmidt coefficients with  $\sum_i \lambda_i = 1$ . The number of nonzero  $\lambda_i$ , called the Schmidt rank, is at least 2 if and only if the state is entangled.

**Example 9.2.**  $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|0\rangle|0\rangle + |1\rangle|1\rangle)$  already has Schmidt form, with Schmidt rank 2 and coefficients  $\lambda_1 = \lambda_2 = \frac{1}{2}$ . Tracing out one qubit gives  $\rho_A = I/2$ , a maximally mixed state with  $S(\rho_A) = 1$  bit.

## 9.5 Entanglement as a computational resource

In quantum algorithms, entanglement is not just an aesthetic curiosity—it is the structural feature that lets quantum computation outpace classical. A quantum circuit that produces only separable states is essentially a collection of independent single-qubit evolutions, simulable classically with linear overhead.

The technical statement is that *any* quantum algorithm with super-polynomial advantage over classical must generate entanglement [Nielsen and Chuang, 2010]. Shor’s algorithm leverages entanglement between the input register and a function register to perform period-finding. Grover’s algorithm uses entanglement implicitly when querying the oracle on

superpositions of all inputs. Quantum amplitude estimation builds an entangled state from the input register and an ancilla register, then extracts the amplitude via phase estimation.

Stabilizer states—the states produced by Clifford gates—can be highly entangled but are nonetheless classically simulable in polynomial time (the Gottesman–Knill theorem). So entanglement is necessary, but not sufficient, for quantum advantage. The full computational power of quantum mechanics requires non-Clifford gates (e.g., the  $T$ -gate) on top of entangling Cliffords.

**Remark 9.2.** *A useful slogan: entanglement is the substrate; non-Clifford gates are the activity on it. Take away either and quantum reduces to a classical machine in disguise.*

## 9.6 Entanglement in financial algorithms

In quantum finance, entanglement appears most directly in two places.

**Amplitude estimation.** The quantum Monte Carlo machinery underlying option pricing and risk [Egger et al., 2020; Stamatopoulos et al., 2020] entangles a function register (encoding payoff values) with an ancilla. The amplitude of a particular basis state of the ancilla equals the expected payoff. Phase estimation reads that amplitude with quadratic speedup over classical Monte Carlo.

**QAOA and VQE.** Variational algorithms for portfolio opti-

mization [Thomassin et al., 2025; Zaman et al., 2024; Morapakula et al., 2026] use parameterized circuits that include entangling layers (typically CNOTs in a defined connectivity pattern) interleaved with single-qubit rotations. The depth of entangling layers controls the expressive power of the ansatz; without them, the variational state cannot represent the correlations needed to encode realistic portfolio constraints.

A practical question in NISQ-era quantum finance is how much entanglement the hardware can sustain. Each two-qubit gate adds noise, and noise destroys entanglement; deep entangling circuits become unreliable on current devices. The art of NISQ algorithm design is to use just enough entanglement to capture the structure of the problem and no more—a constraint that has produced a literature on shallow ansatz circuits and error-aware quantum optimization [Cerezo et al., 2021].

## 9.7 *Entanglement in cryptography*

Entanglement underlies the security of entanglement-based quantum key distribution [Bennett and Brassard, 2014]. Two parties share Bell pairs over a quantum channel; eavesdropping necessarily disturbs the entanglement, which the legitimate parties can detect by checking statistics on a random sample of measurement outcomes. The protocol’s security follows from the no-signalling theorem and the monogamy of entanglement—the fact that a maximally entangled state of  $A$  with  $B$  cannot also be entangled with a third party.

This security is in striking contrast to RSA and elliptic-curve cryptography (Chapters on RSA and Elliptic Curves), whose security rests on the computational hardness of factoring and discrete logarithms—hardness assumptions that quantum computers can defeat (Chapter on Shor’s Algorithm). Entanglement-based protocols are *information-theoretically* secure, not just computationally secure.

## 9.8 What to remember

- Entanglement is the failure of a joint state to factorize as a product of subsystem states.
- Bell states are the canonical maximally entangled two-qubit states; they are the building blocks of quantum communication.
- Quantum correlations are stronger than classical (violating Bell inequalities) but cannot signal (no-signalling theorem).
- Entanglement is a necessary resource for super-polynomial quantum speedups, but it is not sufficient on its own.
- In finance, entanglement appears in amplitude estimation and variational ansatz construction; in cryptography, in QKD.

The next phenomenon—interference—is what turns entangled and superposed states into useful computation by amplifying correct answers and cancelling wrong ones.



## Chapter 10

# Interference

### WHERE THE QUANTUM SPEEDUP ACTUALLY LIVES

Superposition and entanglement are spectacular phenomena, but neither alone delivers algorithmic advantage. A uniform superposition is statistically indistinguishable from a random sampler; an entangled register holds correlations that classical post-processing must still extract. The third pillar—interference—is what turns these resources into computation. Interference is the engine. Without it, quantum mechanics is an expensive way to flip coins.

## 10.1 *What interference is*

Quantum amplitudes are complex numbers. When two computational paths arrive at the same final state, their amplitudes *add*, with possible cancellation due to phase. The probability

of the outcome is the squared modulus of the sum, not the sum of squared moduli:

$$P(\text{outcome}) = \left| \sum_{\text{paths}} \alpha_{\text{path}} \right|^2 \neq \sum_{\text{paths}} |\alpha_{\text{path}}|^2.$$

The latter is the classical formula for the probability of a sum of mutually exclusive paths. The former is the quantum formula and includes cross terms that classical probability cannot produce. Those cross terms are the source of interference, and they can be either constructive (positive) or destructive (negative).

**Example 10.1.** *The Mach–Zehnder interferometer is the canonical illustration. A photon enters a beam splitter, takes one of two paths to a second beam splitter, and is detected at one of two outputs. Each path contributes an amplitude. If the path lengths are equal, the amplitudes interfere constructively at one output and destructively at the other—the photon is always detected at the same output, despite having “probability  $\frac{1}{2}$ ” to take each path. Introducing a phase shift on one arm changes this; sweeping the phase produces the celebrated sinusoidal interference fringes.*

The Hadamard gate followed by a phase followed by another Hadamard implements exactly this interferometer on a qubit:

$$HR_z(\varphi)H|0\rangle = \cos(\varphi/2)|0\rangle - i\sin(\varphi/2)|1\rangle.$$



The probability of measuring  $|0\rangle$  is  $\cos^2(\varphi/2)$ , varying sinusoidally with  $\varphi$ . The qubit has, in effect, taken both arms of the interferometer and self-interfered.

## 10.2 *Constructive and destructive amplification*

Quantum algorithms can be understood as choreographies of interference. The starting configuration is a superposition over all candidate answers, with roughly equal amplitudes. The algorithm then applies a sequence of unitaries that adds positive phases to the desired answer and negative or imaginary phases to the wrong ones. When the final state is measured, the amplitudes of incorrect answers have largely cancelled, and the correct answer is overwhelmingly probable.

**Grover's algorithm** [Grover, 1996] provides the cleanest illustration. Starting from the uniform superposition over  $N$  items, Grover alternates two operations: an oracle that flips the sign of the amplitude of the target item, and a “diffusion” operation that reflects all amplitudes about their mean. Each pair of operations increases the amplitude of the target by approximately  $2/\sqrt{N}$ . After  $O(\sqrt{N})$  iterations, the target's amplitude is close to 1 and measurement returns it with high probability. The non-target amplitudes have systematically interfered destructively.

**The quantum Fourier transform (QFT)** underlies Shor's

algorithm and amplitude estimation. The QFT redistributes amplitudes across a register by applying a precise pattern of phases:

$$\text{QFT}|x\rangle = \frac{1}{\sqrt{N}} \sum_y e^{2\pi i xy/N} |y\rangle.$$

A function that has been evaluated in superposition—producing a register state whose amplitudes carry the function’s structure—can be passed through the QFT to convert that structure into a pattern of constructive and destructive interference, revealing the function’s period upon measurement.

**Remark 10.1.** *A simple way to think about it: classical algorithms compute the right answer. Quantum algorithms make the right answer the most probable measurement outcome. Both can be “correct,” but the mechanism is qualitatively different.*

### 10.3 Coherence: the prerequisite for interference

Interference requires that the amplitudes of competing paths remain coherent—meaning their relative phases are preserved during the computation. Any disturbance that randomizes the relative phases (heat, stray magnetic field, environmental coupling) destroys interference and reduces the quantum computer to a classical one.

This is why quantum hardware must be isolated, cooled, and operated within strict timing windows. A superconducting

transmon at IBM operates at 10 millikelvin in a dilution refrigerator, with electromagnetic shielding and cable filtering at multiple stages. A trapped ion is suspended in ultra-high vacuum and addressed by laser pulses with picosecond precision. Even with these heroic measures, current devices retain coherence for only tens to hundreds of microseconds—vastly shorter than the duration of a useful algorithm at scale [Preskill, 2018].

The chapter on decoherence treats the physics of coherence loss. The chapter on error correction treats how to combat it. Without those defences, interference patterns wash out, and the quantum advantage with them.

## 10.4 *The phase kickback trick*

A particular interference effect, phase kickback, appears in nearly every quantum algorithm. It works as follows. Suppose we have a function  $f$  implemented as a unitary  $U_f$  with

$$U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle.$$

Prepare the second register in  $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ . Then

$$U_f|x\rangle|-\rangle = (-1)^{f(x)}|x\rangle|-\rangle.$$

The phase  $(-1)^{f(x)}$ , which logically belongs to the second register, has been “kicked back” onto the first register. We have converted a function-value computation into a phase

imprinted on the input. If the first register is in superposition over many  $x$ , each  $|x\rangle$  component now carries a sign determined by  $f(x)$ .

This trick is at the heart of Deutsch–Jozsa, Bernstein–Vazirani, Simon’s algorithm, Grover’s algorithm, and Shor’s algorithm. It is the standard way to convert a function oracle into an interference-ready phase pattern.

**Example 10.2.** *For Deutsch’s problem with  $f : \{0, 1\} \rightarrow \{0, 1\}$ : prepare  $|+\rangle|-\rangle$ , apply  $U_f$ , then a final Hadamard. The result is*

$$H_1 U_f (H_1 \otimes I) |0\rangle |-\rangle = \begin{cases} |0\rangle |-\rangle & \text{if } f \text{ is constant,} \\ |1\rangle |-\rangle & \text{if } f \text{ is balanced.} \end{cases}$$

*A single quantum query distinguishes constant from balanced; classically, two queries are required.*

## 10.5 Interference in financial algorithms

In quantum amplitude estimation, the central tool of quantum Monte Carlo for finance, interference is doing two jobs at once. First, it converts the amplitude of a target state—which encodes the expected payoff—into a phase via the Grover-like “amplification operator.” Second, the quantum Fourier transform reads that phase off as a basis state of an output register, with interference concentrating amplitude on the basis state closest to the true phase [Stamatopoulos et al., 2020].

The result is that the amplitude is estimated with precision  $O(1/N)$  given  $N$  oracle queries, in contrast to the classical Monte Carlo precision  $O(1/\sqrt{N})$ . The square-root speedup is paid for entirely by interference: without coherent phases preserved across  $N$  oracle applications, the quantum estimator degrades to the classical one.

The chapter on quantum hardware will return to this point. NISQ-era amplitude estimation is limited by circuit depth, not by qubit count: the longer the coherent interference pattern, the better the speedup—but also the more likely a phase error will derail it. Hybrid shallow amplitude estimation variants [Cerezo et al., 2021; Temme et al., 2017] trade some of the asymptotic speedup for circuits short enough to execute reliably on current devices.

## 10.6 *The classical view of interference, contrasted*

Classical physics has interference too—water waves on a pond, sound waves in a concert hall, light waves at a diffraction grating. The mathematics looks identical: complex amplitudes, superposition, constructive and destructive sums. So what makes quantum interference different?

The answer is that classical interference occurs in a single physical wave occupying space. Many photons or molecules collectively realize the wave amplitudes. Quantum interference, by contrast, occurs at the level of *single particles*. A

single electron self-interferes in a double-slit experiment; the interference pattern builds up one electron at a time. The amplitudes are properties of the quantum state of one particle, not of an ensemble.

This single-particle interference is what makes quantum interference an algorithmic resource. A classical interferometer can compute one function of phase; a quantum computer with  $n$  qubits has  $2^n$  amplitudes available for interference patterns, far beyond what any classical wave system of comparable size can realize.

## 10.7 *Putting the three pillars together*

We can now restate the structure of quantum advantage in one line.

- **Superposition** provides a register state that simultaneously holds  $2^n$  basis components.
- **Entanglement** links those components into a structured joint state that classical systems cannot efficiently represent.
- **Interference** amplifies useful components and cancels useless ones, so that measurement returns an informative outcome.

Each pillar is necessary; none is sufficient on its own. The next chapter on the Schrödinger equation describes the underlying dynamics that produce all three, and from there we move to

the hardware that has to keep these dynamics intact long enough for the algorithm to finish.



## Chapter 11

# *Schrödinger's Equation*

### THE MASTER RULE OF UNITARY MOTION

If the qubit is the actor and Hilbert space is the stage, then the Schrödinger equation is the script. It tells you, given the state of a closed quantum system at one moment, what the state will be at any later moment. Every quantum gate, every algorithm, every superconducting qubit pulse—all of it is, at the lowest level, a particular solution of this single equation [[Schrödinger, 1926](#)].

#### 11.1 *The equation*

The time-dependent Schrödinger equation governs the evolution of the state  $|\psi(t)\rangle$  of a closed quantum system:

$$i\hbar \frac{d}{dt} |\psi(t)\rangle = \hat{H} |\psi(t)\rangle.$$



Here  $\hat{H}$  is the Hamiltonian, a Hermitian operator on the system's Hilbert space corresponding to the total energy of the system, and  $\hbar$  is the reduced Planck constant. The equation is linear in  $|\psi\rangle$ ; this linearity is the formal source of superposition.

For a time-independent Hamiltonian, the solution is

$$|\psi(t)\rangle = e^{-i\hat{H}t/\hbar}|\psi(0)\rangle = U(t)|\psi(0)\rangle.$$

The operator  $U(t) = e^{-i\hat{H}t/\hbar}$  is unitary because  $\hat{H}$  is Hermitian. So evolution under any closed-system Hamiltonian is automatically unitary, recovering the second postulate of Chapter on Quantum Computing. The Hamiltonian is the generator of time evolution.

**Remark 11.1.** *Engineering a quantum gate  $U$  means engineering a Hamiltonian whose evolution at the chosen interaction time equals  $U$ . “Programming” a quantum computer is, at the hardware level, choosing pulses that synthesize the right effective Hamiltonian over a precise duration.*

## 11.2 Eigenstates and stationary states

The time-independent Schrödinger equation is the eigenvalue problem

$$\hat{H}|E_n\rangle = E_n|E_n\rangle.$$

Its solutions are the energy eigenstates with definite energy  $E_n$ . Under time evolution they pick up only a global phase:

$$e^{-i\hat{H}t/\hbar}|E_n\rangle = e^{-iE_nt/\hbar}|E_n\rangle.$$

Because global phase is unobservable, the populations of energy eigenstates are conserved—which is why they are called stationary states.

A general state expands as  $|\psi(0)\rangle = \sum_n c_n |E_n\rangle$ , evolving to

$$|\psi(t)\rangle = \sum_n c_n e^{-iE_n t/\hbar} |E_n\rangle.$$

The amplitudes  $c_n$  are constant in modulus but acquire relative phases proportional to their energies. Interference (Chapter on Interference) arises directly from these relative phases.

**Example 11.1.** For a qubit with Hamiltonian  $\hat{H} = \frac{\hbar\omega}{2}\sigma_z$ , the energy eigenstates are  $|0\rangle$  and  $|1\rangle$  with energies  $\pm\hbar\omega/2$ . A superposition  $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$  evolves as

$$|\psi(t)\rangle = \frac{1}{\sqrt{2}}(e^{-i\omega t/2}|0\rangle + e^{i\omega t/2}|1\rangle) = \cos(\omega t/2)|+\rangle - i\sin(\omega t/2)|-\rangle.$$

The state oscillates between  $|+\rangle$  and  $|-\rangle$  at angular frequency  $\omega$ —the simplest example of Larmor precession, the rotational motion of a qubit's Bloch vector about the  $z$ -axis.

### 11.3 Continuous-variable Schrödinger: the wave function

For a particle moving on a line, the state is a wave function  $\psi(x, t) \in L^2(\mathbb{R})$  (Section 2.22). The Hamiltonian for a particle of mass  $m$  in potential  $V(x)$  is

$$\hat{H} = -\frac{\hbar^2}{2m} \frac{\partial^2}{\partial x^2} + V(x),$$

and the Schrödinger equation becomes the partial differential equation

$$i\hbar \frac{\partial \psi(x,t)}{\partial t} = -\frac{\hbar^2}{2m} \frac{\partial^2 \psi(x,t)}{\partial x^2} + V(x) \psi(x,t).$$

This is the form most physicists first encounter. The Born rule reads  $|\psi(x,t)|^2 dx$  as the probability of finding the particle in the interval  $[x, x + dx]$ .

For quantum computing on qubits, this continuous form rarely appears directly. But it underlies essentially all hardware physics: the energy levels of a transmon qubit, the trapping potential of a single ion, the photon modes of an optical cavity. The leap from continuous Schrödinger dynamics to discrete qubit gates is a leap of *effective theory*—we restrict attention to two energy levels, model their interactions, and let the underlying continuous physics handle itself.

## 11.4 Time-dependent Hamiltonians: pulses and gates

In practice, the Hamiltonian of a real quantum device depends on time, because experimenters control external fields to drive the qubits through desired evolutions. The general solution involves a time-ordered exponential:

$$U(t) = \mathcal{T} \exp\left(-\frac{i}{\hbar} \int_0^t \hat{H}(t') dt'\right),$$

where  $\mathcal{T}$  denotes time-ordering. For most practical gate synthesis, one designs a pulse  $\hat{H}(t)$  over an interval  $[0, T]$  such that  $U(T)$  equals a desired gate.

Two illustrations:

**Rabi oscillation.** A qubit driven on resonance experiences an effective Hamiltonian  $\hat{H} \propto \sigma_x$  in the rotating frame. Constant drive for time  $t$  produces  $U = \exp(-i\Omega t \sigma_x/2) = R_x(\Omega t)$ . A drive duration of  $t = \pi/\Omega$  gives an  $X$  gate; half that, an  $R_x(\pi/2)$  gate.

**Cross-resonance gate.** Two coupled superconducting qubits driven at the frequency of one of them produce a CNOT-like entangling gate via a carefully tuned effective Hamiltonian [Koch et al., 2007]. The pulse shape (DRAG, Gaussian, square) is optimized to minimize leakage to non-qubit states and to suppress unwanted phase errors.

**Remark 11.2.** *The pulse-level view is essential when designing error-mitigation strategies or when an algorithm pushes the limits of hardware. Qiskit Pulse and analogous frameworks expose this layer of control, allowing programmers to specify Hamiltonians and waveforms directly rather than only abstract gates.*

## 11.5 The path integral perspective

Feynman’s path-integral reformulation [Feynman, 1982] expresses the amplitude for a quantum system to go from  $|i\rangle$

at time 0 to  $|f\rangle$  at time  $T$  as a sum over all possible paths in configuration space, each weighted by  $e^{iS[\text{path}]/\hbar}$ , where  $S$  is the classical action along the path:

$$\langle f|U(T)|i\rangle = \int \mathcal{D}[\text{path}] e^{iS[\text{path}]/\hbar}.$$

For most paths, neighbouring paths have wildly different phases and interfere destructively. Paths near the classical extremum of  $S$  have stationary phase and interfere constructively. This is why classical mechanics emerges as the  $\hbar \rightarrow 0$  limit of quantum mechanics: only the classical path survives the interference.

For finance, the path-integral language is the bridge to Feynman–Kac formulas and stochastic calculus. The price of a European option, expressed as  $\mathbb{E}^Q[e^{-rT} \text{payoff}]$ , has a path-integral representation in which the integration runs over price paths weighted by the risk-neutral measure. Quantum amplitude estimation is, in a precise sense, the quantum mechanical analogue: an amplitude (rather than a probability density) encodes the expectation, and a quantum circuit (rather than a classical Monte Carlo) extracts it.

# 11.6    *What the Schrödinger equation forces on us*

The Schrödinger equation has three structural consequences that pervade everything in quantum computing.

**Unitarity.** Closed-system evolution is unitary, preserving inner products and norms. Quantum gates are thus reversible (Chapter on Classical vs Quantum), and information is conserved within a closed circuit.

**Linearity.** The equation is linear in  $|\psi\rangle$ . Two solutions superpose to a third solution. This is what makes superposition possible and gives quantum algorithms their parallelism.

**Energy as generator of time.** The Hamiltonian both encodes the system's energy spectrum and generates its time evolution. To build a gate, engineer a Hamiltonian. To simulate a physical system, engineer the same Hamiltonian on a quantum device—this is the original motivation of Feynman [Feynman, 1982] and the foundation of quantum simulation as a discipline.

**Remark 11.3.** *The structural lesson: every interesting quantum operation traces back to a Hamiltonian. Algorithm design, hardware engineering, and quantum simulation are all variations on the same theme of constructing or evolving under desired Hamiltonians.*

## 11.7 When the Schrödinger equation fails

The Schrödinger equation describes *closed* systems. Real quantum computers are not closed: they couple to environmental degrees of freedom (phonons in a chip substrate, blackbody photons, control electronics) that scramble the system's state. The closed-system equation is then no longer applicable, and

one must use the formalism of open quantum systems—master equations and Lindblad operators (Chapter on Decoherence).

A common open-system equation is the Lindblad master equation

$$\frac{d\rho}{dt} = -\frac{i}{\hbar}[\hat{H}, \rho] + \sum_k \left( L_k \rho L_k^\dagger - \frac{1}{2} \{L_k^\dagger L_k, \rho\} \right),$$

where  $L_k$  are the so-called jump operators encoding interactions with the environment. The first term is the unitary Schrödinger evolution; the second is the dissipative correction. When the  $L_k$  vanish, we recover unitary evolution; otherwise, populations leak between states, off-diagonal coherences decay, and the quantum advantage diminishes.

Quantum error correction (Chapter on Error Correction) and decoherence-aware algorithm design exist precisely because this open-system reality cannot be ignored. The clean Schrödinger picture is the goal; the open-system picture is the engineering challenge.

## 11.8 Take-aways

- Schrödinger's equation specifies the dynamics of a closed quantum system:  $i\hbar \partial_t |\psi\rangle = \hat{H} |\psi\rangle$ .
- Solutions are unitary:  $|\psi(t)\rangle = e^{-i\hat{H}t/\hbar} |\psi(0)\rangle$ .
- Eigenstates of  $\hat{H}$  are stationary; superpositions evolve via

phases accumulating proportional to energies.

- Quantum gates are pulses that synthesize chosen unitaries; the Hamiltonian is the engineer's design variable.
- Real devices require open-system corrections (Lindblad equation), which underlie decoherence and motivate error correction.

The next chapter takes the Schrödinger equation into the laboratory: the actual physical platforms on which quantum information is stored and manipulated, and the engineering trade-offs each one imposes.





## *Chapter 12*

# *Hardware in the Real World*

### THE PLATFORMS THAT HAVE TO MAKE QUBITS BEHAVE

A quantum algorithm is software; a quantum computer is physics. The abstract qubit of earlier chapters has to be realised as a real two-level quantum system, isolated well enough to preserve coherence (Chapter on Decoherence), addressable well enough to perform gates, and connectable well enough to entangle. No platform meets all these demands easily. This chapter surveys the leading hardware approaches, the trade-offs each makes, and the DiVincenzo criteria that any practical platform must satisfy.

## 12.1 *DiVincenzo's criteria*

DiVincenzo [2000] laid out five criteria that any viable quantum computing platform must satisfy, plus two more for quantum communication. The five for computation:

1. **Scalable physical qubits.** The platform must support a register that can be grown to many qubits without fundamental redesign.
2. **Initialization.** Qubits must be reliably prepared in a known state (typically  $|0\rangle^{\otimes n}$ ) before computation.
3. **Long coherence times.** Coherence must persist much longer than the duration of a single gate. The ratio  $T_{\text{coh}}/t_{\text{gate}}$  is the figure of merit; numbers like  $10^4$  or higher are needed for fault tolerance.
4. **Universal gate set.** Some efficiently implementable set of gates must span the unitary group, exactly or to arbitrary precision.
5. **Measurement.** Each qubit must be individually measurable in the computational basis with high fidelity.

Every leading platform now meets each criterion at small scale; the contest is over how well, and how much it costs.

## 12.2 *Superconducting qubits*

Superconducting circuits operated at millikelvin temperatures form the dominant quantum computing platform today. The qubit is encoded in the lowest two energy levels

of a nonlinear LC oscillator, where the nonlinearity comes from a Josephson junction—a sandwich of superconductor–insulator–superconductor through which Cooper pairs tunnel coherently. The transmon [Koch et al., 2007] is the dominant design, valued for its insensitivity to charge noise and its relatively long coherence times.

## Strengths.

- Manufacturable using semiconductor lithography techniques.
- Gate times are fast (10s to 100s of nanoseconds for single-qubit, hundreds of nanoseconds for two-qubit).
- Strong industrial pipeline (IBM, Google, Rigetti).

## Limitations.

- Requires dilution refrigeration to  $\sim 10$  mK.
- Coherence times still modest (100s of microseconds for  $T_1$ , similar for  $T_2$ ).
- Two-qubit gate errors around  $10^{-3}$ , far above the  $10^{-15}$  needed for unmitigated long algorithms.
- Connectivity is fixed by chip layout, typically a planar grid; non-adjacent gates require SWAP overhead.

Arute et al. [2019] reported the first claim of quantum supremacy (now sometimes called quantum advantage or

computational quantum advantage) on a 53-qubit superconducting processor, completing in 200 seconds a sampling task estimated to require 10000 years on the world's then-largest classical supercomputer. The claim has been contested by improved classical simulations but the underlying device technology has matured rapidly since.

## 12.3 *Trapped ions*

Cirac and Zoller [1995] proposed using cold trapped ions as a quantum computing platform. The qubit is encoded in two internal electronic states of an ion (hyperfine or optical levels), held in vacuum by an electromagnetic trap and laser-cooled to its motional ground state. Single-qubit gates are driven by laser pulses; two-qubit gates exploit the shared motional modes of the ion chain.

### **Strengths.**

- Long coherence times (seconds to minutes, far exceeding gate durations).
- Very high single- and two-qubit gate fidelities (above 99.9% on some platforms).
- All-to-all connectivity within a single trap—any pair of ions can interact directly.
- Identical qubits by virtue of identical atomic species.

## Limitations.

- Slower gates than superconducting (microseconds to tens of microseconds).
- Scaling beyond tens of ions per trap is hard; “modular” architectures (multiple traps, photonic interconnects) are an active research area.
- Complex laser systems and ultra-high vacuum requirements.

IonQ and Quantinuum lead the trapped-ion industry; their machines now offer high-fidelity demonstrations of small algorithms, including quantum chemistry and small-scale combinatorial optimization for finance.

## 12.4 *Neutral atoms*

Neutral-atom platforms hold individual atoms in arrays of optical tweezers and use Rydberg interactions to entangle them. The qubit is encoded in two hyperfine ground states or in a Rydberg state; gates are driven by laser pulses. The flexibility of the optical-tweezer array—rearrangeable, scalable to thousands of sites—has driven rapid growth in the platform.

## Strengths.

- Long coherence times for ground-state qubits (seconds).
- Rearrangeable connectivity; the optical tweezer pattern

can be programmed.

- Demonstrated to thousands of qubits in some recent experiments.

### **Limitations.**

- Two-qubit gate fidelities still catching up with leading trapped-ion numbers.
- Rydberg interaction times limit the practical gate speeds.
- Measurement is destructive (atoms must be re-loaded).

QuEra and Pasqal are the commercial leaders; [Harrow et al. \[2009\]](#)-style algorithms have been demonstrated at small scales on these platforms.

## *12.5 Photonic qubits*

Photonic quantum computing encodes qubits in the polarization, path, or time-bin of single photons. Gates are implemented via linear optics (beam splitters, phase shifters) and measurement-based feedforward; entanglement is generated by photon pair sources and post-selection on coincidence measurements.

### **Strengths.**

- Operates at room temperature.

- Photons travel naturally, making the platform ideal for quantum communication.
- Compatible with integrated photonic chip fabrication.

## **Limitations.**

- Probabilistic two-photon gates without strong nonlinearities; deterministic gates require sophisticated measurement-based schemes.
- Loss in optical components is the dominant error channel.
- Photon detection inefficiency.

PsiQuantum and Xanadu lead in photonic quantum computing, with Xanadu having demonstrated Gaussian boson sampling at large scale.

## *12.6 Other platforms*

Several additional platforms are at earlier stages of development but matter strategically.

**Silicon spin qubits.** Electron or nuclear spins in silicon, addressed and entangled via gate-defined quantum dots or donor atoms [Loss and DiVincenzo, 1998; Kane, 1998]. Coherence times are long; integration with standard CMOS manufacturing is attractive. Intel and several academic consortia are active.

**Topological qubits.** States of matter with non-Abelian anyon excitations would give intrinsically protected qubits [Kitaev, 2003]. Microsoft Station Q is pursuing this vision via Majorana zero modes; demonstration at scale remains elusive.

**Nitrogen-vacancy centres.** Spin states of nitrogen-vacancy defects in diamond, addressable via optical and microwave control. Used primarily for quantum sensing and small-scale quantum networking, less for computation.

**Quantum annealers.** D-Wave systems implement the special-purpose task of finding ground states of Ising Hamiltonians, used in QUBO portfolio optimization [Morapakula et al., 2026]. They are not universal quantum computers but offer practical access to optimization tasks at thousands of qubits.

## *12.7 Connectivity, qubit count, and effective scale*

Raw qubit count is the loudest marketing number but the least informative. Three other quantities matter at least as much.

**Connectivity.** A 100-qubit chip with only nearest-neighbour connectivity differs sharply from a 100-qubit trap with all-to-all coupling. SWAP gates can route information, but each SWAP adds depth and error.



**Gate fidelity.** A pair of qubits with 99.9% two-qubit fidelity can run a circuit of order 1000 gates before errors dominate. A pair with 99% fidelity is limited to about 100 gates. The difference is exponential in circuit depth.

**Quantum volume.** A scalar metric introduced by IBM combining qubit count, connectivity, fidelity, and circuit complexity. It captures the largest depth- $d$  random circuit a device can execute with reasonable fidelity on  $d$  qubits. Quantum volume doubles roughly each year on the leading platforms.

A more recent IBM measure is circuit layer operations per second (CLOPS), capturing throughput rather than capability. For financial applications that run many circuits, throughput matters as much as fidelity.

**Remark 12.1.** *A 1000-qubit device with poor connectivity and 1% errors is not, in practice, more useful than a 100-qubit device with good connectivity and 0.1% errors. The total effective computational power is what matters, and the leading benchmark is some variant of quantum volume.*

## 12.8 Control electronics and cryogenics

The qubits get the attention, but the supporting infrastructure is half the system. A superconducting quantum computer requires:

- A dilution refrigerator achieving  $\sim 10$  mK.

- Multi-stage filtering on all signal lines to prevent thermal noise from reaching the qubits.
- Arbitrary waveform generators producing nanosecond-resolution control pulses.
- Low-noise amplifiers (Josephson parametric amplifiers and HEMT amplifiers) for qubit readout.
- A classical FPGA control system orchestrating thousands of pulse sequences in real time.

Each layer is itself a major engineering enterprise. Cryogenic control electronics—moving classical control closer to the qubits, reducing cable counts—is one of the central scaling challenges of superconducting systems. Trapped-ion and neutral-atom platforms have analogous laser and vacuum infrastructure that scales nontrivially.

## 12.9 *The road ahead*

Leading vendors publish quantum computing roadmaps with steadily increasing qubit counts and decreasing error rates. IBM's roadmap, for example, targets thousands of physical qubits per chip in the next few years, with modular interconnections to scale further [IBM Quantum, 2023]. Quantinuum aims for fault-tolerant logical qubits by 2030; QuEra and Pasqal aim for 10,000+ neutral atoms. PsiQuantum aims for fault-tolerant photonic systems.

These roadmaps are aggressive but not implausible. The metric to watch is not raw qubit count but *logical* qubit count—the number of fault-tolerant, error-corrected qubits realized on top of the physical hardware. As of 2026, demonstrations of even a single logical qubit are recent and small-scale. The transition from NISQ to fault-tolerant computation will be the central hardware milestone of the next decade.

The chapter on quantum error correction explains the encoding of one logical qubit in many physical qubits, and the chapter on NISQ describes what we can do in the meantime. The remaining hardware question is whether the engineering follows the roadmap, or stalls at some plateau—a question only experiment can answer.



## Chapter 13

# *Decoherence*

### WHY THE QUANTUM WORLD FADES INTO THE CLASSICAL

A perfectly isolated qubit, evolving under a known Hamiltonian, holds its superposition indefinitely. A real qubit, embedded in a chip or a vacuum chamber or an optical cavity, interacts with countless environmental degrees of freedom—phonons, photons, fluctuating fields, neighbouring atoms. Those interactions destroy coherence on a timescale far shorter than we would like. Decoherence is the name of this destruction. Understanding it is the prerequisite to combating it.

## 13.1 *Two pictures of decoherence*

There are two complementary ways to think about decoherence.

**Information-theoretic.** A qubit becomes entangled with the environment. The joint qubit–environment state remains pure and unitary; but if we discard the environment (we have no access to it), the reduced state of the qubit becomes mixed. The off-diagonal entries of its density matrix decay, and with them go the interference effects that make quantum computing work.

**Physical.** The qubit, coupled to a heat bath, exchanges energy and acquires phase fluctuations. Energy exchange relaxes the qubit’s populations toward thermal equilibrium ( $T_1$  relaxation). Phase fluctuations randomize the relative phase between basis states ( $T_2$  dephasing). Both processes erode the qubit’s usefulness as a quantum bit.

Both pictures describe the same phenomenon [[Zurek, 2003](#)]. The information-theoretic view explains *why* environmental coupling is fatal; the physical view describes *how* it actually happens in a given device.

## 13.2 $T_1$ and $T_2$

The two canonical timescales of single-qubit decoherence are:

$T_1$  (**energy relaxation time**). The characteristic time for

a qubit in  $|1\rangle$  to decay to  $|0\rangle$  via energy dissipation into the environment. In Bloch-vector language,  $T_1$  governs the decay of the  $z$ -component of the Bloch vector toward its thermal equilibrium.

**$T_2$  (dephasing time).** The characteristic time for a qubit's superposition  $|+\rangle$  to lose its coherent off-diagonal entries. In Bloch-vector language,  $T_2$  governs the decay of the  $x$ - and  $y$ -components of the Bloch vector toward zero.

The two timescales are related by the inequality  $T_2 \leq 2T_1$ . The pure-dephasing time, defined by  $1/T_\varphi = 1/T_2 - 1/(2T_1)$ , isolates the dephasing process not caused by energy relaxation.

For current superconducting transmon qubits,  $T_1$  and  $T_2$  are typically 100–300 microseconds. Trapped-ion qubits routinely achieve  $T_2$  of seconds or longer. The figure of merit for an algorithm is  $T_{\text{coh}}/t_{\text{gate}}$ : how many gates fit inside one coherence time. Numbers in the few thousand are typical now; numbers above  $10^4$  are the threshold for serious error correction.

**Example 13.1.** *A two-qubit gate on a transmon takes around 300 ns. With  $T_2 = 100$  microseconds, about 300 such gates fit inside a coherence time. Allowing for single-qubit gates between them and for noise margins, a useful NISQ circuit is typically limited to depths of order 100 two-qubit gates.*

### 13.3 Quantum operations and noise channels

The general formalism for noise is the quantum operation (also called quantum channel), a linear map  $\mathcal{E}$  on density operators preserving positivity and trace. Every such map has a Kraus representation:

$$\mathcal{E}(\rho) = \sum_k K_k \rho K_k^\dagger, \quad \sum_k K_k^\dagger K_k = I.$$

The operators  $\{K_k\}$  are the Kraus operators of the channel. A unitary evolution has a single Kraus operator equal to the unitary; a noisy channel has multiple.

Two canonical single-qubit noise channels:

**Amplitude damping (energy relaxation,  $T_1$ ).** Kraus operators

$$K_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{pmatrix}, \quad K_1 = \begin{pmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{pmatrix},$$

with  $\gamma = 1 - e^{-t/T_1}$ . The diagonal  $K_0$  damps the  $|1\rangle$  population; the off-diagonal  $K_1$  irreversibly transfers  $|1\rangle$  to  $|0\rangle$  (emitting a quantum to the environment).

**Phase damping (pure dephasing,  $T_\varphi$ ).** Kraus operators

$$K_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\lambda} \end{pmatrix}, \quad K_1 = \begin{pmatrix} 0 & 0 \\ 0 & \sqrt{\lambda} \end{pmatrix},$$

with  $\lambda = 1 - e^{-t/T_\varphi}$ . The off-diagonals of  $\rho$  shrink by  $\sqrt{1-\lambda}$ ; populations are unchanged. This is the canonical destruction of coherence without energy loss.

## 13.4 The Lindblad master equation

For continuous-time noisy evolution under weak environmental coupling (the Born–Markov approximation), the density operator obeys the Lindblad master equation

$$\frac{d\rho}{dt} = -\frac{i}{\hbar}[\hat{H}, \rho] + \sum_k \left( L_k \rho L_k^\dagger - \frac{1}{2} \{ L_k^\dagger L_k, \rho \} \right).$$

The first term is the closed-system Schrödinger evolution; the second term is the dissipative correction with jump operators  $L_k$  encoding specific noise processes. For a transmon qubit with  $T_1$  relaxation,  $L_1 = \sqrt{1/T_1} \sigma_-$  (lowering operator); for pure dephasing,  $L_2 = \sqrt{1/(2T_\varphi)} \sigma_z$ .

The Lindblad equation has the great virtue of being a continuous, deterministic equation for  $\rho(t)$  that can be simulated numerically. It is the workhorse of quantum hardware modelling, control design, and noise characterization.

**Remark 13.1.** *The Born–Markov approximation assumes weak coupling to a memoryless environment. Real devices sometimes have non-Markovian dynamics (the environment remembers prior interactions), requiring more elaborate models. 1/f noise in superconducting qubits is a classic example.*

## 13.5 Sources of decoherence in real hardware

The dominant decoherence channels depend on the platform.

**Superconducting qubits.**



- Dielectric loss at the substrate–metal interface, the largest known  $T_1$  killer for transmons.
- Two-level defects in oxide layers, which exchange energy with the qubit.
- Quasiparticle poisoning from stray non-equilibrium quasiparticles.
- Cosmic rays and ionising radiation, recently identified as an unavoidable bulk source of correlated errors.

### **Trapped ions.**

- Spontaneous emission from optically driven states.
- Motional heating from electric field noise on trap electrodes.
- Magnetic field fluctuations dephasing hyperfine qubits.

### **Photonic systems.**

- Photon loss in waveguides, fibres, and detectors—the dominant error.
- Mode mismatch causing imperfect interference at beam splitters.

Each platform’s roadmap revolves around progressively eliminating these channels: better materials, cleaner fabrication, improved shielding, better control protocols.

## 13.6 *Decoherence as the loss of interference*

For finance algorithms, the practical effect of decoherence is the degradation of interference patterns. Quantum amplitude estimation relies on coherent phases accumulating over many oracle queries. Each application of the oracle has some probability of an uncorrected error; as the circuit depth grows, the surviving coherent component shrinks exponentially.

The result is a soft ceiling on algorithm performance. A noisy quantum amplitude estimation, even with the asymptotic  $1/N$  scaling preserved in principle, achieves only  $1/\sqrt{N}$  in practice once  $N$  exceeds a noise-set crossover. Error mitigation techniques [Temme et al., 2017; Kim et al., 2023]—zero-noise extrapolation, probabilistic error cancellation, virtual distillation—push this crossover further out but do not remove it. Only full error correction (Chapter on Error Correction) breaks the ceiling.

## 13.7 *The classical limit as decoherence-driven*

Decoherence has a deep conceptual role beyond engineering. It explains why the macroscopic world appears classical, even though its constituents are quantum. A coffee cup interacting with  $10^{23}$  air molecules, photons, and phonons decoheres on a timescale of about  $10^{-20}$  seconds—essentially instantaneously. Its position state is then a statistical mixture in the preferred “pointer basis” (here, position-eigenstate-like states), with no observable quantum coherence [Zurek, 2003].

In the language of Chapter on Superposition: macroscopic objects are not in superposition because they cannot maintain coherence in the face of constant environmental coupling. Quantum mechanics does not vanish at large scales; it is just that the coherences needed to demonstrate it are washed out before any observer notices. The boundary between quantum and classical is not a sharp line; it is a coherence-time scale, which is itself a function of size, isolation, and the number of environmental degrees of freedom.

**Remark 13.2.** *This is why building a quantum computer is engineering at the very edge of nature. The default state of any macroscopic system is rapid decoherence; to compute we must hold the system off the slide into classicality for the duration of the algorithm. The harder the computation, the longer the hold.*

## 13.8 *What it all means for algorithm design*

Three rules of thumb follow from the structure of decoherence.

**Shallower is better.** Circuit depth, not qubit count, is the binding constraint on most NISQ-era algorithms. Algorithms that work with low-depth circuits and many short runs (variational methods, hybrid loops) generally outperform deep one-shot circuits.

**Error-aware design.** Algorithms benefit from being expressed in primitives whose noise behaviour is well-characterized: standard gate sets, restricted connectivity pat-

terns, dynamical decoupling sequences. Custom pulse sequences may be more efficient in principle but harder to characterize.

**Mitigation, then correction.** Error mitigation can extract useful results from noisy circuits when error correction is not yet possible. The mitigated estimator is biased and noisy, but at moderate depths it provides accurate expectations that the bare noisy estimator does not.

The next chapter on quantum error correction describes the long-term fix: encoding logical qubits in many physical qubits so the noise can be detected and corrected before it accumulates fatally.



## Chapter 14

# *Saving Qubits from Themselves*

### QUANTUM ERROR CORRECTION AND THE ROAD TO FAULT TOLERANCE

Classical computers are robust: a stray cosmic ray flips a bit, ECC memory catches it, the user never knows. Quantum computers are fragile: noise corrupts the very superposition that the computation depends on, and you cannot simply read the qubit to check it, because reading collapses the state. Quantum error correction (QEC) solves this seemingly impossible problem by encoding a single logical qubit in many physical qubits and using clever syndrome measurements that detect errors without disturbing the encoded information. This chapter sketches the central ideas [[Nielsen and](#)

## 14.1 *Why classical strategies don't work*

The obvious classical strategy for error correction is repetition: store each bit as three copies, decode by majority vote. For qubits this fails for three reasons.

**No cloning.** The no-cloning theorem (Chapter on the Qubit) forbids copying an unknown qubit. We cannot make three identical replicas of  $|\psi\rangle$  to vote on.

**Continuous errors.** A classical bit flips or doesn't. A qubit can experience small rotations through angle  $\epsilon$ , errors of any continuous magnitude. We must somehow turn continuous error processes into a discrete error to correct.

**Measurement collapse.** Measuring a qubit to check its value destroys the superposition. We need a way to detect errors without learning the encoded information.

Each of these obstacles has a quantum solution, and together they yield the QEC framework.

## 14.2 *The three-qubit codes*

The simplest illustrative codes correct only bit-flip ( $X$ ) or only phase-flip ( $Z$ ) errors. The three-qubit bit-flip code encodes a

logical qubit as

$$|0_L\rangle = |000\rangle, \quad |1_L\rangle = |111\rangle.$$

A logical state  $\alpha|0_L\rangle + \beta|1_L\rangle = \alpha|000\rangle + \beta|111\rangle$  is created from  $\alpha|0\rangle + \beta|1\rangle$  by two CNOTs (cloning the basis but not the amplitudes). If one of the three qubits suffers a bit-flip, syndrome measurements of  $Z_1Z_2$  and  $Z_2Z_3$  identify which qubit flipped without measuring the logical state, and an  $X$  gate on that qubit restores the code. Similar logic, with Hadamards inserted, gives the three-qubit phase-flip code.

These codes are pedagogically essential but not practical: they correct only one type of error. The first code that corrects an arbitrary single-qubit error—bit-flip, phase-flip, both, or any small continuous rotation thereof—was Shor’s nine-qubit code [Shor, 1995].

### 14.3 *Shor’s nine-qubit code*

Shor [1995] constructed the first code that corrects an arbitrary single-qubit error by concatenating the phase-flip and bit-flip codes:

$$|0_L\rangle = \frac{1}{\sqrt{8}}(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)(|000\rangle + |111\rangle),$$

$$|1_L\rangle = \frac{1}{\sqrt{8}}(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)(|000\rangle - |111\rangle).$$

Nine physical qubits encode one logical qubit. Syndrome measurements identify which qubit was struck by which error type, allowing correction.

The crucial structural insight is discretization of continuous errors. A small rotation  $e^{i\epsilon X}|\psi\rangle = \cos\epsilon|\psi\rangle + i\sin\epsilon X|\psi\rangle$  is a superposition of “no error” and “bit flip on this qubit.” The syndrome measurement collapses this superposition into one of the two outcomes; if it collapses to “bit flip,” the correction is applied, and if to “no error,” nothing is done. Either way the state is restored. The continuous error has been converted to a discrete one by the measurement.

## 14.4 Stabilizer formalism and CSS codes

The general framework that made QEC tractable is the stabilizer formalism. A stabilizer code is defined by a set of commuting Pauli operators (the stabilizers), whose common  $+1$  eigenspace forms the code subspace. Errors are detected by measuring the stabilizers and looking for syndromes (which stabilizers report  $-1$ ).

The CSS construction [[Calderbank and Shor, 1996](#); [Steane, 1996](#)] produces quantum codes from pairs of classical linear codes satisfying a duality condition. The Steane  $[[7, 1, 3]]$  code encodes one logical qubit in seven physical qubits and corrects any single-qubit error; it is one of the smallest practical CSS codes and remains a standard pedagogical example.

CSS codes have the operational advantage that logical Clifford gates can be implemented *transversally*: a Clifford gate on the logical qubit is obtained by applying the same gate to each physical qubit independently. Transversal gates do not



propagate errors within an encoded block, which is essential for fault tolerance.

**Remark 14.1.** *Not every desired logical gate is transversal. The  $T$ -gate, needed for universality, is not transversal in CSS codes; it must be implemented via magic state distillation, an expensive procedure that prepares high-fidelity ancillary states and consumes them to implement  $T$ -gates. Magic state overhead is a significant practical concern in fault-tolerant cost estimates.*

## 14.5 The surface code

The dominant code for near-term fault-tolerant proposals is the surface code [Kitaev, 2003; Fowler et al., 2012]. It is a CSS code defined on a 2D grid of physical qubits, with stabilizers measured by local four-qubit syndrome checks involving only nearest-neighbour qubits. The surface code's strengths:

- **Locality.** All operations are nearest-neighbour, well-matched to chip layouts.
- **High threshold.** The error correction threshold—the physical error rate below which logical errors can be made arbitrarily small by increasing code distance—is around 1% for the surface code, the highest of any practical code family.
- **Scalability.** The code distance  $d$  scales with the linear size of the grid; the logical error rate decays exponentially as  $\sim p^{d/2}$  where  $p$  is the physical error rate.

The cost is that one logical qubit requires  $O(d^2)$  physical qubits. For a target logical error rate  $10^{-12}$  at physical error rate  $10^{-3}$ , the code distance might be  $d = 15$  or higher, demanding several hundred physical qubits per logical qubit. For Shor's algorithm on a 2048-bit RSA key, current estimates put the resource requirement at millions of physical qubits running for hours [Nielsen and Chuang, 2010].

**Example 14.1.** *A distance-3 surface code encodes one logical qubit in 17 physical qubits (9 data, 8 ancilla). A distance-5 code uses 49 physical qubits. The footprint grows quadratically with distance. Current hardware has produced distance-3 and distance-5 demonstrations, with logical error rates approaching but not yet beating the threshold.*

## 14.6 The threshold theorem and fault tolerance

The threshold theorem [Aharonov and Ben-Or, 2008; Shor, 1996] states that if the physical error rate per gate is below some threshold  $p_{\text{th}}$ , then arbitrarily long quantum computations can be performed with arbitrarily high reliability by encoding qubits in a sufficiently large code. The theorem's existence makes fault-tolerant quantum computing in principle possible. The practical question is what  $p_{\text{th}}$  is in real hardware and how much overhead the encoding costs.

For surface codes, the threshold is  $\sim 1\%$ . Current superconducting two-qubit gate fidelities are  $\sim 99.5\%$  or better on the

best platforms, putting hardware just below threshold. This is what makes fault tolerance suddenly feel near at hand rather than decades away.

The threshold theorem includes important fine print:

- Errors must be local and approximately uncorrelated (correlated bursts violate the assumptions).
- The overhead grows polynomially in the inverse logical error rate (modest, but not free).
- The threshold value depends on the code, decoder, and noise model.

**Remark 14.2.** *A subtle point: “below threshold” is not a yes/no statement. Different operations (state preparation, gates, measurement) may have different effective error rates. Demonstrating that a logical qubit has lower error rate than a physical qubit at the same noise level—genuine sub-threshold behavior—is the milestone the field has been chasing. Recent surface code demonstrations are achieving it for the first time.*

## 14.7 *Where the field stands in 2026*

As of 2026, several groups have demonstrated logical qubits with surface codes at distance 3 and 5, with logical error rates that just beat the corresponding physical error rates. These are first-of-their-kind demonstrations: small numbers of logical qubits, short execution times, but conceptually full

QEC operating below threshold. Roadmaps from IBM, Google, Quantinuum, and others target tens to hundreds of logical qubits in the second half of the 2020s.

What this means for finance:

- Cryptographically relevant Shor's algorithm runs (Chapter on Shor) still require thousands of logical qubits and trillions of  $T$ -gates—likely a decade or more away.
- Smaller logical-qubit demonstrations of useful financial algorithms (e.g., 20 logical qubits running amplitude estimation) may arrive within five years.
- In the meantime, the NISQ era continues, and error mitigation (Chapter on NISQ) is the bridge to fault tolerance.

## *14.8 Beyond surface codes: high-rate codes and biased noise*

The surface code's  $O(d^2)$  overhead is expensive. Research on alternative codes seeks to reduce this overhead while preserving the surface code's locality and high threshold. Some promising directions:

**Color codes.** Three-dimensional CSS codes with all single-qubit Clifford gates transversal, including  $T$ -gates in three dimensions. The 2D version has the same overhead as the surface code; the 3D version gains some transversality at the cost of harder fabrication.

**Quantum LDPC codes.** Low-density parity-check codes with constant rate (linear in the number of physical qubits), substantially more efficient than the surface code's vanishing rate. Long-range syndrome connections are required, which is an obstacle for planar architectures.

**Biased-noise codes.** Codes optimized for noise channels where one error type dominates (e.g.,  $T_1$ -dominated relaxation in superconducting qubits). The Kitaev cat-state code and the dual-rail encoding can achieve much lower thresholds for biased noise.

The bottom line: surface codes are the leading near-term choice, but the long-run code landscape is more diverse, and the choice of code may eventually be platform-dependent.

## 14.9 *The QEC takeaway*

Quantum error correction is the bridge from NISQ to scale. The conceptual content is profound: encode information so that no measurement reveals the encoded state, then measure only *differences* between code configurations to detect errors. The mathematical machinery is intricate but mature, and the engineering challenge is now squarely about reducing physical error rates below threshold and managing the overhead of code distance.

For finance applications, QEC is not a near-term concern but a long-term enabler. The transformative quantum-finance

use cases—factoring RSA at scale, computing exact derivative prices on a high-dimensional model—are gated on fault-tolerant computation. The chapters on RSA, Elliptic Curves, and Shor’s algorithm assume fault tolerance has arrived. The NISQ chapter, by contrast, assumes it hasn’t.



## Chapter 15

# *Living in the NISQ Era*

### DOING USEFUL WORK WITH NOISY, INTERMEDIATE-SCALE QUANTUM MACHINES

In 2018 John Preskill coined a term that has shaped expectations of the entire field: NISQ, for *Noisy Intermediate-Scale Quantum* computing [[Preskill, 2018](#)]. The phrase captures the awkward middle ground we now inhabit: machines large enough that classical simulation is hard, but small and noisy enough that fault-tolerant error correction (Chapter on Error Correction) is not yet running at scale. The NISQ era is where every demonstration of quantum advantage to date has lived, and where the next several years of quantum finance research will continue to live.

## 15.1 *What NISQ means, precisely*

A NISQ device, in Preskill's original sense, has these features:

- **Noisy.** Gate error rates are too high (roughly  $10^{-3}$  for two-qubit gates) to run long algorithms without error correction.
- **Intermediate-scale.** Qubit count is large enough that classical exact simulation is expensive but small enough that the device cannot host the millions of physical qubits needed for fault-tolerant algorithms of consequence. The original range was 50–several hundred; in 2026, leading machines have reached 1000+ physical qubits, still nowhere near fault-tolerant scale.
- **Pre-fault-tolerant.** Quantum error correction may be demonstrated on small numbers of logical qubits but is not yet the routine substrate of computation.

The defining tension of NISQ is this: every additional gate adds noise that compounds without correction. Useful NISQ algorithms must therefore be either *short* (low circuit depth) or *noise-tolerant* (designed to extract information despite errors). The bulk of NISQ-era algorithm research is about meeting one of those criteria.

## 15.2 *Variational algorithms: the workhorse*

The dominant NISQ algorithmic paradigm is the variational quantum algorithm [Cerezo et al., 2021]. A parameterized



quantum circuit  $U(\boldsymbol{\theta})$  is evaluated on the device to compute a cost function  $C(\boldsymbol{\theta}) = \langle \psi(\boldsymbol{\theta}) | H | \psi(\boldsymbol{\theta}) \rangle$  for some Hamiltonian  $H$ . A classical optimizer adjusts  $\boldsymbol{\theta}$  to minimize  $C$ . The pattern combines a shallow, error-tolerant quantum subroutine with a powerful classical outer loop.

Two canonical variational algorithms:

**Variational Quantum Eigensolver (VQE)** [Peruzzo et al., 2014]. Find the ground state of a Hamiltonian  $H$ . The cost function is the expectation  $\langle H \rangle$ ; the optimizer drives it down. Used for quantum chemistry and condensed-matter problems; also adaptable to finance via Hamiltonians encoding portfolio constraints.

**Quantum Approximate Optimization Algorithm (QAOA)** [Farhi et al., 2014]. Approximate the ground state of an Ising or QUBO Hamiltonian via a layered ansatz of cost and mixer unitaries. Each layer adds expressive power; the optimal depth depends on the problem and the device. Used for combinatorial optimization, including portfolio selection [Zaman et al., 2024; Thomassin et al., 2025].

Both algorithms share three features that make them NISQ-suitable: shallow circuits, intrinsic error tolerance via the variational principle, and clean integration with classical optimizers. They also share two weaknesses: cost-function evaluations are noisy and require many shots; the optimization landscape has barren plateaus and local minima that grow worse with qubit count.

**Example 15.1.** *A QAOA portfolio optimization with 20 assets uses 20 qubits in superposition over portfolio inclusion patterns, with two-qubit gates between coupled assets and single-qubit rotations capturing risk-return trade-offs. A few layers of QAOA can produce competitive portfolios on small instances even in noisy execution.*

### 15.3 Error mitigation, not correction

NISQ devices cannot run quantum error correction at full scale. They can run error mitigation, a different idea: instead of correcting errors during the computation, accept them, but extract less biased expectation values by post-processing many noisy runs.

Three widely used techniques:

**Zero-noise extrapolation (ZNE)** [Temme et al., 2017]. Run the algorithm at several deliberately increased noise levels, fit a model of how the expectation behaves as a function of noise, and extrapolate to zero noise. Linear or exponential extrapolation suffices in many cases. ZNE is conceptually simple and has been demonstrated to give meaningful corrections at moderate depths.

**Probabilistic error cancellation (PEC)** [Temme et al., 2017]. Decompose the desired noiseless gate as a quasi-probabilistic mixture of noisy gates, then sample from this mixture and apply correcting sign flips. PEC is exact in principle but

expensive: the sampling overhead grows exponentially with circuit depth.

**Virtual distillation / error suppression.** Use a virtual purification procedure with multiple copies of the noisy state to produce a less noisy estimate. Particularly effective for state-tomography-style tasks.

[Kim et al. \[2023\]](#) demonstrated that error-mitigated quantum computations on a 127-qubit IBM device produced accurate expectation values for a problem (Ising-like dynamics) where classical exact simulation is infeasible—suggestive evidence that NISQ devices can be useful, not just toy demonstrations.

## 15.4 *Where NISQ wins, where it doesn't*

A useful framing: NISQ wins where the algorithmic structure is intrinsically tolerant of noise. This includes:

- **Variational ground state finding** where the variational principle guarantees that the cost is bounded below the true ground energy and noisy improvements are still improvements.
- **Quantum sampling** for problems where the output distribution itself is the answer; some kinds of noise still produce hard-to-simulate distributions.
- **Hybrid feature extraction** as in [Ciceri et al. \[2025\]](#)'s bond trading results, where quantum hardware noise plausibly acts

as a regularizer that improves classical model generalization.

- **Quantum simulation of mildly noisy physical systems**, where the simulated system itself is noisy and the device's noise is part of the model rather than an obstacle.

NISQ loses where the algorithm requires deep coherent circuits with high-precision interference. The most prominent loser is Shor's algorithm (Chapter on Shor): the precision of phase estimation required to factor large integers translates to circuits beyond NISQ reach. Quantum amplitude estimation runs at the same border: shallow variants exist but trade speedup for circuit depth, and true asymptotic speedup awaits fault tolerance.

**Remark 15.1.** *The blunt summary: NISQ is good for exploration and possibly for narrow advantage on small instances; it is not good for breakthrough applications at scale. Both kinds of work matter, but the marketing temptation is always to confuse them.*

## 15.5 The benchmarking landscape

How do we measure NISQ progress? Several benchmark frameworks have emerged.

**Quantum volume.** A scalar metric introduced by IBM combining qubit count, connectivity, and fidelity in a single number, based on the largest random circuit a device can successfully execute.

**CLOPS.** Circuit Layer Operations Per Second, capturing throughput. For variational algorithms, where many circuit evaluations occur in the optimization loop, CLOPS may matter more than peak fidelity.

**Application-specific benchmarks.** The Quantum Economic Development Consortium (QED-C) and others have proposed standardised benchmarks for specific applications (chemistry, optimization, finance) that measure end-to-end accuracy and runtime on the actual problems users care about.

**Randomized benchmarking.** A protocol that measures average gate fidelity by running random Clifford sequences. The decay rate of the success probability is a clean noise metric independent of state preparation and measurement (SPAM) errors.

Each benchmark has weaknesses; none captures every dimension of performance. The mature view is that benchmarks are tools for comparing apples to apples, not universal scores.

## 15.6 *The practical NISQ recipe for finance*

For a finance team contemplating a NISQ project, a working recipe is:

1. **Find a problem that fits NISQ.** Variational optimization or quantum-enhanced machine learning, on instances small enough to fit on current hardware (typically tens of variables/features).

2. **Build the classical baseline first.** Establish what classical algorithms achieve on the same instance. Quantum claims of advantage require this baseline to compare against.
3. **Choose hardware.** Match the algorithm's structure to the device's strengths: trapped ions for high fidelity and connectivity, superconducting for throughput and qubit count.
4. **Apply error mitigation.** ZNE or PEC for expectation-style quantities. Without mitigation, NISQ results are typically too biased to interpret.
5. **Iterate.** The optimization loop, hyperparameter tuning, and noise characterization run many times. Compute budget is dominated by classical-quantum loop overhead, not by single-shot quantum cost.

[Morapakula et al. \[2026\]](#) and [Ciceri et al. \[2025\]](#) are good examples of this discipline in action in the finance literature.

## 15.7 *What ends the NISQ era?*

The NISQ era will end—either by gradual transition into the fault-tolerant era, as logical qubit counts grow and error rates drop below threshold, or by some unexpected hybrid regime where logical and physical operations co-exist. Current vendor roadmaps point toward the first scenario, with the late 2020s as a likely transition window for early fault-tolerant computations and the 2030s for genuinely deep logical-qubit algorithms.

For finance, the key question is when an end-to-end advantage on a production-scale problem becomes possible. Several conditions must be met simultaneously:

- Physical error rates below code threshold (approached now).
- Logical qubit counts in the hundreds (mid-to-late 2020s on aggressive roadmaps).
- Efficient classical data loading or problem encoding (an algorithmic obstacle, not just a hardware one).
- Total runtime competitive with classical solutions (a system-level question).

We are not there yet. The NISQ era is therefore best understood as a long apprenticeship: building intuition, identifying the structures where quantum techniques will eventually pay off, and developing the engineering discipline of hybrid classical-quantum computing. Finance applications have been at the leading edge of this apprenticeship and will likely be at the leading edge of the transition to fault tolerance as well.

The next chapter introduces the software framework—Qiskit—through which most of this work is done in practice.



## Chapter 16

# *Programming Quanta with Qiskit*

### THE OPEN-SOURCE TOOLKIT THAT BRIDGES ALGORITHM AND HARDWARE

Quantum algorithms live in mathematical notation; quantum hardware lives in pulses and cryogenic chambers. Software frameworks bridge the two. The dominant such framework is Qiskit, the open-source quantum computing toolkit developed primarily by IBM. Qiskit is the language in which most of the algorithms cited throughout this book were implemented and tested, and the path through which a researcher's quantum-finance code reaches actual hardware [[Pathak et al., 2025](#)].



## 16.1 *What Qiskit is*

Qiskit is a Python library for designing, simulating, optimizing, and executing quantum circuits. Its architecture has matured through several major revisions since the original 2017 release. The current stack (Qiskit 1.x and the IBM Quantum cloud) provides:

- **Circuit construction.** A `QuantumCircuit` object with gates, measurements, classical control, and parameterised expressions.
- **Transpilation.** A pipeline that compiles abstract circuits to the gate set and connectivity of a specific target backend, optimizing depth and gate count.
- **Simulators.** Classical simulators (statevector, density-matrix, stabilizer, MPS) for testing and debugging.
- **Hardware access.** Programmatic access to IBM Quantum hardware via the Qiskit Runtime service, with primitives for sampling and expectation estimation.
- **Pulse control.** Lower-level access to the analog pulse schedules that implement gates on superconducting hardware.
- **Application modules.** Domain-specific helpers, including Qiskit Finance (option pricing, portfolio optimization, credit risk).

Other frameworks exist—Google’s Cirq, Microsoft’s Q#, Xanadu’s PennyLane, AWS Braket—and each has strengths. Qiskit’s dominant position comes from IBM’s hardware reach, its substantial documentation, and the size of its user community. Most quantum-finance papers since 2018 use Qiskit as the implementation language.

## 16.2 *A minimal Qiskit circuit*

The shortest non-trivial Qiskit program creates a Bell state and measures it. In current Qiskit:

```
from qiskit import QuantumCircuit

qc = QuantumCircuit(2, 2)
qc.h(0)                # Hadamard on qubit
qc.cx(0, 1)            # CNOT with control
qc.measure([0, 1], [0, 1])
```

Three lines of quantum code produce  $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$  and measure both qubits. Running this on a simulator yields outcomes 00 and 11 each with probability  $\frac{1}{2}$ , and never 01 or 10. Running it on hardware yields the same distribution plus a few percent of 01 and 10 from gate and readout errors.

**Example 16.1.** *A Bell-state circuit is the “hello world” of quantum computing. The fact that this three-line program demonstrates an essentially non-classical correlation (Chapter*

on Entanglement)—one that violates Bell inequalities when measured along non-trivial bases—is the single sharpest illustration of what Qiskit makes routine.

## 16.3 Transpilation: from algorithm to chip

A user writes circuits in terms of abstract gates ( $H$ ,  $X$ ,  $T$ , CNOT, parameterised rotations) and abstract qubits. A real chip implements only a specific gate set on a specific connectivity graph. Transpilation is the translation step.

The Qiskit transpiler performs several passes:

- **Gate decomposition.** Each abstract gate is rewritten in the target gate set. A Toffoli becomes a chain of single-qubit and CNOT gates; a parameterized rotation may decompose into a sequence of native gates.
- **Qubit routing.** Logical qubits are mapped to physical qubits, and SWAP gates are inserted where two-qubit gates are needed between non-adjacent qubits.
- **Optimization.** Adjacent gates are combined, redundant pairs are cancelled, and circuit depth is minimized via several heuristic passes.
- **Layout selection.** The transpiler chooses which physical qubits to use, weighing connectivity and per-qubit fidelity.

For an algorithm to run efficiently on a particular backend,

transpilation typically reduces circuit depth by a factor of 2–5x compared to a naive rewrite. The transpilation strategy can be customized via optimization level (0 to 3 in Qiskit), with higher levels producing shorter but more compute-intensive transpilations.

**Remark 16.1.** *A common surprise to newcomers: a circuit with 100 gates as written may transpile into 500 gates on hardware once routing and decomposition costs are included. Algorithm complexity claims should be made on transpiled, not abstract, circuit depth.*

## 16.4 Primitives: sampling and estimation

Recent Qiskit Runtime introduced two high-level primitives that capture the two main things quantum users want:

**Sampler.** Given a parameterised circuit, return samples from the measurement distribution. Useful for circuit-style algorithms (Grover, QFT, sampling tasks). Returns counts of bit-strings.

**Estimator.** Given a parameterised circuit and an observable, return the expectation  $\langle \psi | H | \psi \rangle$ . Useful for variational algorithms (VQE, QAOA) and amplitude estimation. Returns a real number (with error bars).

These primitives hide the low-level details of shot counts, measurement-basis rotations, error mitigation, and post-processing. They are designed so that the same code works

against simulators, noiseless backends, and noisy hardware with mitigation applied—changing one of these is a configuration choice rather than a rewrite.

For finance, the Estimator primitive is the workhorse: portfolio cost functions, expected payoffs, and risk metrics all reduce to expectation values of suitable observables.

## 16.5 *Qiskit Finance*

The Qiskit Finance module [noa] bundles common quantum-finance algorithms behind clean Python APIs:

- **Option pricing** via quantum amplitude estimation [Stamatopoulos et al., 2020]. Constructs the encoding circuit for log-normal price distributions and applies iterative amplitude estimation to compute expected payoffs.
- **Portfolio optimization** via QAOA or VQE [Thomassin et al., 2025; Zaman et al., 2024]. Encodes Markowitz mean-variance optimization (with cardinality and budget constraints) as a QUBO Hamiltonian and minimizes via variational circuits.
- **Credit risk** via quantum amplitude estimation on default-probability models [Dri et al., 2022]. Constructs Bernoulli mixtures for systemic risk factors and estimates portfolio loss distributions.
- **Quantum data loaders** that prepare quantum states en-

coding empirical or analytic distributions, the bottleneck for many finance algorithms.

Each module is a thin layer above the core Qiskit primitives, exposing the financial problem at a level natural for practitioners while delegating quantum circuit construction to the framework. A typical workflow: import the relevant module, construct the financial problem as a Python object, hand it to a Qiskit algorithm class, run on a simulator or hardware, post-process.

**Example 16.2.** *Pricing a European call option in Qiskit Finance is, schematically: build a `LogNormalDistribution` object with desired parameters, build a `EuropeanCallPricing` object referencing the distribution and strike, hand it to `IterativeAmplitudeEstimation`, call `estimate()`. The result is the option price with a confidence interval. The amplitude-estimation queries to the encoding circuit happen behind the scenes.*

## 16.6 Hybrid programs: classical-quantum loops

For variational algorithms, the canonical loop is:

1. Choose initial parameters  $\theta_0$ .
2. Submit the parameterised circuit  $U(\theta)$  with current  $\theta$  to the Estimator primitive.

3. Receive the cost  $C(\theta)$ .
4. Update parameters  $\theta$  using a classical optimizer (SPSA, COBYLA, Adam, etc.).
5. Repeat until convergence.

Qiskit Runtime's Session API allows this loop to run with minimal latency between iterations, keeping the quantum hardware reserved across iterations rather than re-queueing each time. For NISQ workloads, this kind of latency reduction is itself a significant performance gain: a long optimization with hundreds of iterations would otherwise spend most of its time in queueing rather than computation.

## 16.7 *Simulators: development before deployment*

Most algorithm development happens on simulators, not on hardware. The Qiskit Aer simulators provide:

- **Statevector simulator.** Exact pure-state simulation. Memory grows exponentially: 30 qubits requires  $2^{30} \cdot 16$  bytes  $\approx 16$  GB.
- **Density-matrix simulator.** Includes mixed states; can model arbitrary noise channels. Memory grows as  $4^n$ , far worse than statevector.
- **Stabilizer simulator.** Efficient for Clifford-only circuits (any number of qubits in polynomial time), useful for bench-

marking error correction.

- **Matrix-product-state simulator.** Efficient for states with limited entanglement, common in NISQ algorithms.
- **Noisy hardware simulator.** Reproduces the noise model of a specific real backend, allowing realistic testing before queueing.

A practical development workflow is to debug on the statevector simulator, validate on the noisy simulator using a real backend's noise model, and only then deploy to hardware. The cost difference is large: hardware time is metered and queued; simulators run on the user's laptop or cluster.

## 16.8 *Where Qiskit is going*

The Qiskit project's trajectory points to several directions over the next few years.

**Dynamic circuits.** Mid-circuit measurement and classical feedforward, enabling teleportation-based gates, measurement-based variational algorithms, and error-correction primitives. Recent IBM hardware supports these, and Qiskit Runtime exposes them.

**Error mitigation built-in.** Future Estimator primitive versions will provide error mitigation (ZNE, PEC) as a configuration option, no longer requiring user-side implementation.



**Quantum-centric supercomputing.** Workflows that combine HPC clusters with quantum hardware over the cloud, with Qiskit serving as the orchestration layer. This is positioned as the architecture for utility-scale quantum applications.

**Fault-tolerant primitives.** As logical qubits arrive, Qiskit's abstraction will need to expose them while hiding the encoding details. Early work in this direction is already in progress in Qiskit and competing frameworks.

## 16.9 *Why this matters for the financial reader*

A finance professional or researcher entering quantum computing should expect to spend most of their early effort in Qiskit, not in algorithm derivation from scratch. The papers cited throughout this book—[Stamatopoulos et al. \[2020\]](#), [Morapakula et al. \[2026\]](#), [Ciceri et al. \[2025\]](#), [Dri et al. \[2022\]](#), and many others—have Qiskit implementations available, and the fastest way to build intuition is to run them, modify them, and observe the results.

Qiskit is not the entire story. PennyLane, Cirq, Q# and others provide overlapping capabilities with different philosophies and hardware backends. But by sheer momentum, Qiskit is where the quantum-finance literature mostly lives, and where the next several years of practical experimentation will continue to be done.

The remaining three chapters of this book turn to the negative side of the quantum advantage: cryptography. The same algorithms Qiskit makes accessible to researchers can, when run on fault-tolerant hardware, break the public-key cryptography that secures the financial system. We turn next to that threat.



## Chapter 17

# RSA

### THE PADLOCK THAT SECURES THE FINANCIAL INTERNET—AND THE ONE QUANTUM WILL PICK

In 1977 Rivest, Shamir, and Adleman published the cryptographic protocol now universally called RSA [[Rivest et al., 1978](#)]. Half a century later, RSA is the most widely deployed public-key cryptosystem on earth: TLS handshakes, code signing, smartcards, S/MIME email, IPsec VPNs, financial settlement protocols, government identity documents. Almost every encrypted message you have ever sent has, at some point in its life, depended on RSA. This chapter explains how RSA works, why it has held up so well against classical cryptanalysis, and why Shor's algorithm (Chapter on Shor) is its decisive enemy.

## 17.1 Public-key cryptography in one paragraph

Until 1976, all known cryptosystems required sender and receiver to share a secret key in advance. [Diffie and Hellman \[1976\]](#) proposed the radical alternative: each party has a *public* key that anyone can use to encrypt, and a separate *private* key that only the owner can use to decrypt. Encryption is easy; decryption without the private key is computationally hopeless. This is public-key or asymmetric cryptography. Diffie–Hellman provided the first such protocol (for key exchange); RSA [[Rivest et al., 1978](#)], a year later, provided one for both encryption and digital signatures.

The breakthrough rested on the idea of a trapdoor function: easy to compute in one direction, hard to invert without secret information. RSA’s trapdoor is modular exponentiation:  $x \mapsto x^e \bmod N$  is fast;  $y \mapsto y^{1/e} \bmod N$  is, without the private key, equivalent in difficulty to factoring  $N$ .

## 17.2 The RSA construction

### Key generation.

1. Choose two large primes  $p$  and  $q$ . Compute  $N = pq$ .  $N$  becomes part of the public key.
2. Compute Euler’s totient  $\varphi(N) = (p-1)(q-1)$ .
3. Choose a public exponent  $e$  with  $\gcd(e, \varphi(N)) = 1$ .  
Common choices:  $e = 65537 = 2^{16} + 1$  (efficient for

verification).

4. Compute the private exponent  $d \equiv e^{-1} \pmod{\varphi(N)}$  via the extended Euclidean algorithm.
5. Public key:  $(N, e)$ . Private key:  $d$  (and possibly  $p, q$ ).

**Encryption.** For a message  $m$  with  $0 \leq m < N$ , compute ciphertext  $c = m^e \bmod N$ .

**Decryption.** Compute  $m = c^d \bmod N$ .

The correctness of decryption rests on Fermat's little theorem (generalised via Euler): for any  $m$  coprime to  $N$ ,  $m^{\varphi(N)} \equiv 1 \pmod{N}$ . Since  $ed \equiv 1 \pmod{\varphi(N)}$ , we have  $ed = 1 + k\varphi(N)$  for some integer  $k$ , so

$$c^d = (m^e)^d = m^{ed} = m^{1+k\varphi(N)} = m \cdot (m^{\varphi(N)})^k \equiv m \pmod{N}$$

The encryption and decryption operations cancel.

**Example 17.1.** Pick  $p = 61, q = 53$ . Then  $N = 3233, \varphi(N) = 60 \cdot 52 = 3120$ . Choose  $e = 17$ ; then  $d = e^{-1} \bmod 3120 = 2753$ . To encrypt  $m = 65$ :  $c = 65^{17} \bmod 3233 = 2790$ . To decrypt:  $m = 2790^{2753} \bmod 3233 = 65$ . Tiny example; real RSA uses primes hundreds of digits long.

### 17.3 Where the security comes from

RSA's security rests on the difficulty of integer factorization: given  $N = pq$  with  $p, q$  unknown large primes, recover  $p$  and  $q$ . If an adversary could factor  $N$ , they could compute  $\varphi(N) = (p-1)(q-1)$ , then  $d = e^{-1} \bmod \varphi(N)$ , and decrypt at will.

The reduction goes only one way (factoring is sufficient to break RSA). It is an open question whether breaking RSA *requires* factoring, or whether a separate algorithm could break RSA without recovering  $p$  and  $q$ . For practical purposes, factoring is the assumed hardness.

Factoring is hard *classically*. The best general-purpose algorithm is the number field sieve (NFS) [Lenstra et al., 1993], with sub-exponential complexity

$$L_N \left[ \frac{1}{3}, \sqrt[3]{\frac{64}{9}} \right] = \exp \left( (c + o(1)) (\log N)^{1/3} (\log \log N)^{2/3} \right), \quad c$$

This is faster than fully exponential time but much slower than polynomial. For a 2048-bit RSA key, NFS requires somewhere in the neighbourhood of  $10^{30}$  operations; the largest factored RSA modulus to date is RSA-250 (829 bits), taking 2700 core-years of compute in 2020. A 2048-bit modulus is, by every classical projection, secure for decades.

**Remark 17.1.** *Note “classically secure for decades.” The same modulus on a sufficiently large quantum computer running Shor’s algorithm is broken in hours. The asymmetry between classical sub-exponential and quantum polynomial complexity is what makes the post-quantum cryptography problem urgent.*

## 17.4 RSA in the financial system

A non-exhaustive list of the places RSA secures the financial infrastructure:

- **TLS.** Web browsers and banking apps use RSA in the TLS handshake to establish symmetric session keys. Every online banking session begins with an RSA-protected key exchange.
- **Card networks.** EMV chip cards use RSA-based digital signatures to authenticate themselves to point-of-sale terminals; Visa and Mastercard issuer authentication relies on RSA.
- **SWIFT.** The interbank messaging network uses RSA-based public-key infrastructure for message authentication and key distribution.
- **Digital signatures on contracts.** S/MIME and PKCS#7 signatures on legal documents, securities filings, regulatory submissions almost always use RSA.
- **Code signing.** Software updates to bank applications and trading systems are signed with RSA to prevent tampering.

The pervasiveness is the problem. Replacing RSA throughout this infrastructure requires coordinated migration across thousands of organizations, decades of legacy code, embedded hardware with fixed cryptography, and protocols that pre-date the post-quantum cryptography standards. [Auer et al. \[2024\]](#) estimate that a financial-sector-wide migration will take 10 years or more even with concerted effort.

## 17.5 *Variants and accelerators*

Several variants and accelerators have been developed for RSA over the years.

**RSA-OAEP.** Plain “textbook” RSA is malleable and deterministic, and is therefore not used in production. Real RSA encryption uses Optimal Asymmetric Encryption Padding (OAEP), which adds randomness and structure to make the ciphertext indistinguishable from random.

**RSA-PSS.** The probabilistic signature scheme adds a random salt to RSA signatures, providing stronger security proofs.

**CRT decryption.** Decryption can be sped up by a factor of  $\sim 4$  using the Chinese Remainder Theorem to compute  $c^d \bmod p$  and  $c^d \bmod q$  separately and combine.

**Multi-prime RSA.** Using three or more primes can speed decryption further at a modest security cost.

None of these variants change the fundamental quantum vulnerability: all rely on the hardness of factoring or related number-theoretic problems that quantum computers can solve in polynomial time.



## 17.6 Pollard rho and other classical factoring techniques

For completeness, a few classical factoring techniques worth knowing:

**Trial division.** Test small primes as factors. Works for  $N$  with small prime factors; fails immediately for products of two large primes (the RSA case).

**Pollard rho** [Pollard, 1975]. A clever random-walk approach that runs in  $O(N^{1/4})$  time. Practical for moderately sized  $N$  but quickly outpaced by the number field sieve.

**Pollard  $p-1$ .** Effective when  $p-1$  has only small prime factors; mitigated by choosing  $p$  such that  $p-1$  has at least one large prime factor.

**Quadratic sieve.** Faster than Pollard for  $N$  above  $\sim 10^{50}$ , slower than the number field sieve above  $\sim 10^{100}$ .

**Number field sieve.** The current champion for large  $N$ , the algorithm that has factored the largest RSA challenge moduli to date.

The classical state of the art improves slowly. Each algorithmic advance shifts the secure key size somewhat, but the asymptotic complexity has not changed since NFS in the early 1990s. By contrast, quantum factoring (Shor) is asymptotically polynomial—a category change, not an incremental

improvement.

## 17.7 *The harvest-now–decrypt-later threat*

A practical concern often overlooked: an adversary does not need a quantum computer *today* to attack RSA-encrypted data *today*. They need only to record the encrypted traffic now and decrypt it later when a quantum computer becomes available. This is the harvest now, decrypt later threat.

For information that loses value quickly—one-time passwords, ephemeral chat—this is not a concern. For information that retains value for years or decades—strategic positions, intellectual property, classified intelligence, long-term financial records, identity-document signatures—it is an active threat right now. State-level adversaries are widely believed to be archiving encrypted internet traffic precisely for this purpose.

[Auer et al. \[2024\]](#) highlight this point for the financial sector: client information, contract clauses, regulatory filings, and trade secrets routinely require multi-year or multi-decade confidentiality. Any such data encrypted with RSA today is at risk if quantum computers become powerful enough during its required confidentiality window.

## 17.8 *The post-quantum migration*

The cryptographic community has spent fifteen years developing public-key schemes that resist both classical and

quantum cryptanalysis: post-quantum cryptography (PQC). NIST ran a multi-round PQC competition from 2016 to 2024, culminating in the standardisation of:

- **ML-KEM** (Module-Lattice Key Encapsulation Mechanism), based on the Module-LWE problem [[National Institute of Standards and Technology, 2024](#)]. Replaces RSA for key exchange.
- **ML-DSA** (Module-Lattice Digital Signature Algorithm). Replaces RSA for digital signatures.
- **SLH-DSA** (Stateless Hash-Based Digital Signature Algorithm). A hash-based signature alternative.

These are now FIPS standards (FIPS 203, 204, 205). Major browser vendors, cloud providers, and financial standards bodies are rolling them out. Financial institutions are advised to follow NIST's roadmap: inventory cryptographic dependencies, prioritize high-value long-confidentiality data, deploy hybrid (classical + post-quantum) protocols during transition, fully retire RSA in production over the next 5–10 years.

The migration is large and complex but tractable. The chapters on elliptic curves and Shor's algorithm complete this thread: ECC, the other major public-key family, faces the same fate as RSA, and Shor's algorithm is the single quantum reason why.

## Chapter 18

# *Elliptic Curves*

### CRYPTOGRAPHY'S QUIET WORKHORSE—AND ANOTHER CASUALTY OF SHOR

If RSA is the famous lock, elliptic-curve cryptography (ECC) is the small, efficient one that does most of the actual work. Introduced independently by [Koblitz \[1987\]](#) and [Miller \[1986\]](#), ECC provides the same security as RSA at much smaller key sizes, and has largely displaced RSA wherever bandwidth and CPU matter: TLS in modern browsers, mobile authentication, Bitcoin and most other cryptocurrencies, modern signing protocols (Ed25519, ECDSA). And, like RSA, it falls to Shor's algorithm.

## 18.1 What an elliptic curve is

An elliptic curve over a field  $\mathbb{F}$  is the set of points  $(x, y) \in \mathbb{F}^2$  satisfying a Weierstrass equation

$$y^2 = x^3 + ax + b,$$

together with a special point at infinity  $\mathcal{O}$ . For cryptography, we work over a finite field  $\mathbb{F}_p$  (the integers mod a large prime  $p$ ) or over a binary field  $\mathbb{F}_{2^m}$ . The condition  $4a^3 + 27b^2 \neq 0$  (non-vanishing discriminant) ensures the curve is non-singular.

The points of an elliptic curve form an abelian group under a geometric “chord-and-tangent” addition law. Given two points  $P, Q$  on the curve:

- Draw the line through  $P$  and  $Q$  (or the tangent at  $P$  if  $P = Q$ ).
- Find the third intersection point  $R'$  of this line with the curve.
- Reflect  $R'$  across the  $x$ -axis. The reflected point is  $P + Q$ .

This rule, made explicit in coordinates, gives algebraic formulas for  $P + Q$  and  $2P$  in terms of the coordinates of  $P$  and  $Q$ . The point  $\mathcal{O}$  acts as the identity, and  $-P$  is the reflection of  $P$  across the  $x$ -axis. The group law is commutative and associative, making the points of the curve into an abelian group.

**Example 18.1.** On the curve  $y^2 = x^3 - x + 1$  over  $\mathbb{R}$ , take  $P = (0, 1)$  and  $Q = (1, 1)$ . The line through them is  $y = 1$ . Substituting:  $1 = x^3 - x + 1$ , so  $x^3 - x = x(x - 1)(x + 1) = 0$ . The third intersection is  $R' = (-1, 1)$ . Reflecting:  $R = P + Q = (-1, -1)$ .

## 18.2 Scalar multiplication and the discrete log problem

For an integer  $k$  and a point  $P$  on the curve, scalar multiplication is

$$[k]P = \underbrace{P + P + \cdots + P}_{k \text{ times}}.$$

This is fast to compute: by the standard “double-and-add” procedure,  $[k]P$  requires  $O(\log k)$  point doublings and additions, each of which is fast over  $\mathbb{F}_p$ .

The reverse problem, the elliptic curve discrete logarithm problem (ECDLP), is hard:

**Problem.** Given a point  $P$  on the curve and a point  $Q = [k]P$  for some unknown integer  $k$ , recover  $k$ .

The fastest classical algorithm for the ECDLP on a well-chosen curve is Pollard rho, with complexity  $O(\sqrt{n})$  where  $n$  is the order of the subgroup generated by  $P$ . For a 256-bit curve order, this is  $2^{128}$  operations—infeasible on any classical computer, now or in the foreseeable future.

The hardness of the ECDLP is, by every classical metric, more robust than the hardness of factoring. Factoring has the number-field sieve (sub-exponential); ECDLP on standard curves has only Pollard rho (fully exponential in the bit-length). This is why ECC keys can be much shorter than RSA keys for the same security level: a 256-bit ECC key delivers  $\sim 128$  bits of security, comparable to a 3072-bit RSA key.

### 18.3 *ECDH key exchange and ECDSA signatures*

The two dominant ECC applications are key exchange and digital signatures.

#### **ECDH (Elliptic Curve Diffie–Hellman).**

1. Alice and Bob agree on a curve  $E$  over  $\mathbb{F}_p$  and a base point  $G$ .
2. Alice picks random private key  $a$ , computes public key  $A = [a]G$ .
3. Bob picks random private key  $b$ , computes public key  $B = [b]G$ .
4. Each exchanges their public key over an insecure channel.
5. Both compute  $[a][b]G = [ab]G$ . This is their shared secret.

An eavesdropper sees  $A$  and  $B$  but cannot extract  $a$  or  $b$

without solving the ECDLP. ECDH is the basis of the key exchange in modern TLS.

**ECDSA (Elliptic Curve Digital Signature Algorithm).** A digital signature scheme. To sign a message  $m$  with private key  $a$ :

1. Choose random nonce  $k$ , compute  $R = [k]G = (x_R, y_R)$ . Set  $r = x_R \bmod n$  (where  $n$  is the order of  $G$ ).
2. Compute  $s = k^{-1}(h(m) + ar) \bmod n$ , where  $h$  is a hash function.
3. Signature is  $(r, s)$ .

Verification recovers  $r$  from the signature and the public key  $A$ . A unique-nonce flaw (re-using  $k$  for two different messages) allows recovery of the private key; this caused notorious failures (the PlayStation 3, early Bitcoin wallets).

ECDSA and its modern variant Ed25519 (using Edwards curves) are the dominant signature schemes for software signing, web certificates, SSH keys, and cryptocurrency wallets.

## 18.4 *Curve choices and standards*

Not all elliptic curves are equally secure or efficient. Standardised curves include:

- **NIST P-256 (secp256r1).** A 256-bit prime-field curve published by NIST. Widely deployed in TLS, EMV, government



systems. Suspect to some cryptographers for opaque parameter generation; the NSA's role in its design has been debated.

- **secp256k1.** The curve used by Bitcoin and many cryptocurrencies. Chosen for performance reasons (efficient endomorphism). Mathematically sound but not in original NIST suite.
- **Curve25519 / Ed25519.** A modern curve designed by Daniel Bernstein, providing high performance, “rigid” parameter generation, and resistance to side-channel attacks. Increasingly the default in new protocols.
- **Brainpool curves.** European-standardised curves with transparent parameter generation. Used in some European financial PKIs.

For finance applications, NIST P-256 dominates legacy systems (EMV cards, financial PKIs), with Ed25519 increasingly the choice for new protocols. The migration to post-quantum cryptography will affect both equally—no elliptic curve, however carefully chosen, resists Shor's algorithm.

## 18.5 *Why ECC is the modern default*

Compared to RSA, ECC offers concrete advantages at every security level:

- **Smaller keys.** 256-bit ECC matches the security of 3072-bit RSA. For 192-bit security, ECC uses 384-bit keys vs RSA's

7680-bit keys.

- **Faster operations.** ECC point multiplication is much faster than RSA modular exponentiation at equivalent security.
- **Smaller signatures and ciphertexts.** An ECDSA signature is around 64 bytes; an RSA signature at equivalent security is 256+ bytes.
- **Lower power consumption.** Critical for smart cards, IoT, and mobile devices.

These advantages have driven ECC's dominance in new deployments since around 2010. TLS 1.3 essentially defaults to ECC for key exchange. Mobile banking apps use ECC for authentication. Bitcoin and most other cryptocurrencies use ECDSA on secp256k1. Smart cards in modern EMV chip and mobile-wallet deployments are ECC-based.

**Remark 18.1.** *The story of ECC adoption in financial systems is a slow tide rather than a sudden flip. EMV chip cards transitioned from RSA-only to ECC-supporting over more than a decade. Many legacy systems still use RSA in parallel. The PQC migration will be an equally long process—perhaps another decade—and the financial industry will likely run all three (RSA, ECC, PQC) in parallel for years.*

## 18.6 *Pairings and identity-based cryptography*

A more exotic family of elliptic curves admit bilinear pairings, mappings  $e : E_1 \times E_2 \rightarrow \mathbb{G}_T$  satisfying  $e([a]P, [b]Q) = e(P, Q)^{ab}$ . Pairings enable cryptographic primitives unavailable with vanilla ECC:

- **Identity-based encryption** (IBE), where a user's identity (e.g., email address) serves directly as their public key.
- **Short signatures** (BLS), used by Ethereum and other blockchain protocols.
- **Aggregate signatures**, where many signatures can be combined into one short signature.

Pairing-based schemes have been used in some financial blockchain protocols and in research-stage privacy systems. They too rely on the hardness of the ECDLP (or the related Bilinear Diffie–Hellman Problem) and are vulnerable to the same quantum attacks.

## 18.7 *Shor's algorithm hits ECC at least as hard as RSA*

Shor's algorithm (Chapter on Shor) is most famous for factoring, but it solves the discrete logarithm problem in any finite abelian group in polynomial time. The ECDLP is a discrete log in the group of points on an elliptic curve over  $\mathbb{F}_p$ , so Shor breaks it.

The resource estimates for breaking ECC with Shor are, in fact, smaller than for breaking RSA at equivalent classical security. A 256-bit ECC key (128-bit classical security) requires roughly 2500 logical qubits and  $\sim 10^{10}$  Toffoli gates under recent optimised resource estimates—less than the  $\sim 6000$  logical qubits and  $\sim 10^{11}$  Toffoli gates for 2048-bit RSA. So ECC is, paradoxically, slightly more vulnerable to early fault-tolerant quantum computers than RSA at equivalent classical security.

This has practical consequences. An adversary with a moderately-sized fault-tolerant quantum computer would target ECC-secured systems first. Since ECC is now the dominant key-exchange algorithm in TLS, mobile banking, and cryptocurrencies, the financial sector's quantum exposure is concentrated in ECC just as much as in RSA.

**Example 18.2.** *A Bitcoin transaction is signed with ECDSA on secp256k1. Once a transaction's public key is revealed on the blockchain (which happens at the moment of spending), a sufficiently powerful quantum computer could recover the private key and forge subsequent transactions. The community is actively researching post-quantum replacements (lattice-based and hash-based signature schemes) and migration paths.*

## 18.8 *The post-quantum landscape for ECC replacements*

The same NIST post-quantum standards (ML-KEM, ML-DSA, SLH-DSA, FALCON) that replace RSA also replace ECC. For finance, the migration considerations are similar: inventory dependencies, prioritize long-confidentiality data, deploy hybrid protocols during transition, eventually retire ECC in production.

One difference matters. ECC has smaller key sizes and signatures than RSA. The post-quantum replacements typically have *larger* keys and signatures than ECC—ML-KEM-768 has 1184-byte public keys, ML-DSA-65 has 1952-byte public keys, far above the 32–64 byte ECC analogues. For bandwidth-constrained protocols (mobile banking, smart cards), this presents engineering challenges: protocols must be updated to accommodate larger keys and signatures, and embedded systems may need hardware upgrades.

The migration from ECC to PQC will therefore be more disruptive in some sectors than the migration from RSA, despite ECC's modern reputation. Financial systems that already use ECC for performance reasons (smart cards, mobile devices) will feel this cost most.

## 18.9 The unifying lesson

RSA and ECC are mathematically different but share a common structural weakness: both rely on the difficulty of an abelian-group discrete logarithm or factoring problem, and both fall to Shor's algorithm. The next chapter explains how.

The broader cryptographic lesson is that *algebraic structure is dangerous*. Any cryptosystem whose security can be reduced to a hidden-subgroup problem over an abelian group is vulnerable to Shor. Post-quantum cryptosystems must therefore base their hardness on different mathematical problems—lattice problems, code-based problems, hash-based constructions, multivariate polynomial systems—that resist the quantum techniques of Shor and his successors. The diversity of the NIST PQC finalists reflects this strategic shift: not putting all post-quantum eggs in one mathematical basket.



## Chapter 19

# *Shor's Algorithm*

### THE FACTORING BOMBSHELL THAT STARTED THE QUANTUM GOLD RUSH

In 1994 Peter Shor, then at Bell Labs, published an algorithm that factors an  $n$ -bit integer in time polynomial in  $n$  on a quantum computer [Shor, 1997]. By the same technique, it solves the discrete logarithm problem. RSA, ECC, ECDH, ECDSA—the public-key infrastructure that secures essentially every encrypted transaction in the modern financial system—is broken by Shor's algorithm running on a sufficiently large fault-tolerant quantum computer. Shor's result transformed quantum computing from an academic curiosity into a strategic priority for governments, banks, and intelligence agencies worldwide. This chapter explains how it works.

## 19.1 Factoring as period finding

Shor's central insight is that factoring an integer  $N$  can be reduced to finding the period of a particular function.

Pick a random  $a$  coprime to  $N$ . Consider the function

$$f(x) = a^x \bmod N.$$

This function is periodic: there exists a smallest positive integer  $r$  (the order of  $a$  modulo  $N$ ) such that  $f(x+r) = f(x)$  for all  $x$ . The order  $r$  is the period of  $f$ .

If we know  $r$ , and  $r$  happens to be even, and  $a^{r/2} \not\equiv -1 \pmod{N}$ , then

$$a^r - 1 = (a^{r/2} - 1)(a^{r/2} + 1) \equiv 0 \pmod{N},$$

so  $N$  divides  $(a^{r/2} - 1)(a^{r/2} + 1)$  but not either factor. Computing  $\gcd(a^{r/2} \pm 1, N)$  via the Euclidean algorithm yields a non-trivial factor of  $N$ . The conditions ( $r$  even,  $a^{r/2} \not\equiv -1$ ) hold for at least half of randomly chosen  $a$ , so a few attempts suffice.

The reduction from factoring to period finding is purely classical. What Shor's algorithm contributes is the quantum subroutine that finds the period  $r$  exponentially faster than any classical algorithm can.

**Example 19.1.** Take  $N = 15$ ,  $a = 7$ . The powers of 7 mod 15 are 7, 4, 13, 1, 7, 4, ..., so  $r = 4$ . Since  $r$  is even and



$7^{r/2} = 49 \equiv 4 \pmod{15}$ , *neither  $-1$  nor problematic*, we compute  $\gcd(7^2 - 1, 15) = \gcd(48, 15) = 3$  and  $\gcd(7^2 + 1, 15) = \gcd(50, 15) = 5$ . The factors are 3 and 5.

## 19.2 The quantum period-finding subroutine

The quantum part of Shor's algorithm uses two registers and three steps: create a uniform superposition, evaluate  $f$ , and apply the quantum Fourier transform.

**Setup.** Let  $Q = 2^q$  for some  $q$  with  $Q \geq N^2$ . Prepare two registers: the first of size  $q$  initialized to  $|0\rangle$ , the second of size  $\lceil \log_2 N \rceil$  also initialized to  $|0\rangle$ .

**Step 1: superposition.** Apply Hadamard gates to each qubit of the first register:

$$\frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle |0\rangle.$$

**Step 2: function evaluation.** Apply the unitary  $U_f$  implementing  $|x\rangle |0\rangle \mapsto |x\rangle |f(x)\rangle = |x\rangle |a^x \bmod N\rangle$ . The state becomes

$$\frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle |a^x \bmod N\rangle.$$

**Step 3: measure the second register.** Suppose the outcome is  $f_0$ . The first register collapses to the superposition over

all  $x$  with  $a^x \equiv f_0 \pmod{N}$ —namely the residues  $x_0, x_0 + r, x_0 + 2r, \dots$  for some offset  $x_0$ :

$$\frac{1}{\sqrt{Q/r}} \sum_{j=0}^{Q/r-1} |x_0 + jr\rangle.$$

This is a comb of  $\delta$ -functions spaced by  $r$ —a state whose Fourier transform reveals  $r$  directly.

**Step 4: quantum Fourier transform on the first register.**

$$\text{QFT}_Q |x\rangle = \frac{1}{\sqrt{Q}} \sum_{y=0}^{Q-1} e^{2\pi i xy/Q} |y\rangle.$$

The Fourier transform of a comb is a comb. After QFT, the amplitudes concentrate near multiples of  $Q/r$ . Measuring the first register yields a value  $y$  close to  $kQ/r$  for some  $k$ .

**Step 5: classical post-processing.** From the measured  $y$ , recover  $r$  by computing the continued-fraction expansion of  $y/Q$  and extracting the rational approximation  $k/r$  in lowest terms. With probability  $O(1/\log\log r)$ , the measured  $y$  allows recovery of  $r$ . Repeat  $O(\log\log N)$  times to amplify success probability close to 1.

The total quantum cost is dominated by the modular exponentiation  $U_f$ , which requires  $O(n^3)$  gates (using the standard schoolbook implementation) or  $O(n^2 \log n \log \log n)$  with faster modular arithmetic. The QFT itself costs  $O(n^2)$  gates. The whole algorithm runs in  $O(n^3)$  time on a quantum computer.

**Remark 19.1.** *The structural magic is in Step 3 and Step 4. The function evaluation creates an entanglement between the input register and the function register; measuring the function register collapses the input register to a periodic state with the desired period. The QFT extracts that period via interference: amplitudes at non-period frequencies cancel, amplitudes at period-related frequencies add constructively.*

### 19.3 Why classical algorithms cannot match this

The classical complexity of period finding for  $f(x) = a^x \bmod N$  is the same as factoring  $N$ : sub-exponential via the number field sieve [Lenstra et al., 1993]. There is no known classical algorithm that exploits the periodic structure to achieve polynomial time. The reason—roughly—is that classical algorithms can examine  $f$  only at one input at a time, and the period  $r$  can be as large as  $N$ . Sampling  $f$  uniformly and looking for repetitions takes  $O(\sqrt{r}) = O(\sqrt{N}) = O(2^{n/2})$  queries (birthday paradox), exponential in  $n$ .

The quantum algorithm escapes this by evaluating  $f$  in *superposition* on all  $Q$  inputs simultaneously, then using the QFT to read off the period directly. The QFT is the crucial primitive: it converts a periodic state into a sharply peaked state at the reciprocal frequencies, with all the interference happening coherently inside the quantum register.

This is the same QFT-based interference pattern that drives

quantum phase estimation, the algorithm at the heart of quantum amplitude estimation for finance [Stamatopoulos et al., 2020]. The factoring application is the most famous use; the underlying primitive (period/phase estimation via QFT) is shared with much of the rest of quantum algorithmics.

## 19.4 *From factoring to discrete logarithms*

Shor's algorithm extends to the discrete logarithm problem (DLP) in any abelian group, with a structurally similar construction. For the DLP in  $\mathbb{Z}_p^*$  (find  $x$  such that  $g^x \equiv h \pmod{p}$ ), one constructs a function  $f(a, b) = g^a h^{-b} \pmod{p}$  that is periodic in  $(a, b)$  in a way that encodes  $x$ , then uses the QFT to extract the period.

The same template applies to the ECDLP (find  $k$  such that  $Q = [k]P$  on an elliptic curve): the function  $f(a, b) = [a]P - [b]Q$  has a period in  $(a, b)$  that encodes  $k$ . Resource estimates suggest a 256-bit ECDLP requires  $\sim 2500$  logical qubits and  $\sim 10^{10}$  Toffoli gates, fewer than 2048-bit RSA factoring.

Both forms of Shor are polynomial-time in the bit-length of the problem. This is the algorithmic content that has driven the post-quantum cryptography migration discussed in the RSA and ECC chapters.

## 19.5 *Has Shor's algorithm been run?*

Small-scale demonstrations of Shor's algorithm have been published since 2001.

**NMR (Nuclear Magnetic Resonance).** [Vandersypen et al. \[2001\]](#) factored  $N = 15$  on a 7-qubit liquid-state NMR system, identifying the factors 3 and 5. The demonstration was historic but controversial—NMR systems are not scalable quantum computers, and the “qubits” were ensemble averages rather than individual two-level systems.

**Photonic systems.** Several photonic demonstrations have factored small integers (15, 21) using compiled, optimized versions of Shor's circuit. These too rely on substantial classical pre-computation and do not represent “full” executions of the algorithm.

**Superconducting.** Demonstrations on IBM and other platforms have factored similarly small integers. Recent work (2024–2025) has factored 35 and 91 on noisy superconducting hardware with extensive error mitigation, again with significant classical assistance.

The largest integer factored by a genuine “end-to-end” quantum execution of Shor's algorithm remains in the low double digits. The reason is that factoring a  $n$ -bit integer requires controlled modular exponentiation on a register of  $\sim 2n$  qubits, with circuit depth in the tens of thousands of two-qubit gates—vastly beyond what current NISQ hardware

can execute coherently. The actual quantum threat to RSA awaits fault-tolerant quantum computing.

**Remark 19.2.** *News stories occasionally claim that a small-scale quantum factoring demonstration “breaks RSA.” These claims are misleading. Factoring 35 or 91 is no threat to 2048-bit RSA. The scaling problem is severe: the resources required grow polynomially, but the polynomial is large enough that current hardware is many orders of magnitude short.*

## 19.6 Resource estimates for cryptographically relevant Shor

For RSA-2048, the most-cited modern resource estimate is roughly:

- $\sim 4000$  logical qubits (after compilation and ancilla overhead).
- $\sim 3 \times 10^9$  Toffoli gates.
- $\sim 8$  hours of computation time on hardware with appropriate clock rates.
- Implemented on surface-code-encoded logical qubits requiring physical error rates  $\sim 10^{-3}$ , code distance  $\sim 25$ , and thus  $\sim 20$  million physical qubits.

These estimates have been steadily declining as algorithmic improvements and better resource analyses appear. The 2025

estimates are around half the qubit count and Toffoli count of those from 2019; another 5–10x reduction is plausible. Even so, the requirements are far beyond current devices. The most aggressive vendor roadmaps target millions of physical qubits by the early-to-mid 2030s; cryptographically relevant Shor seems achievable in that decade, with significant uncertainty.

For ECC-256, the resource estimates are smaller—around 2500 logical qubits and  $10^{10}$  Toffolis, with corresponding physical qubit counts in the low millions. ECC may fall first.

## 19.7 *The strategic implications*

Three strategic conclusions flow from the existence of Shor’s algorithm:

**Migrate now, not later.** The harvest-now-decrypt-later threat is operational today. Any data that must remain confidential beyond, say, 2035 should be encrypted with post-quantum schemes today. NIST’s standardisation of ML-KEM and ML-DSA [[National Institute of Standards and Technology, 2024](#)] gives financial institutions a clear migration target.

**Hybrid first.** For the next several years, hybrid classical-PQC protocols are the prudent default. They preserve classical security guarantees while gaining post-quantum resistance, at the cost of some bandwidth and CPU overhead.

**Crypto-agility.** The PQC landscape will continue to evolve. Cryptanalysis of lattice-based schemes is an active research

area, and breaks of NIST candidates have occurred during standardisation. Financial systems should be designed to swap cryptographic primitives easily—an architectural principle called crypto-agility—rather than to depend on any single algorithm.

Auer et al. [2024] make these points in the central-bank context. Gong et al. [2026] elaborate them for the broader financial sector. The migration is, in practice, the most actionable consequence of quantum computing for finance professionals today—more so than any of the speedup opportunities, which require quantum hardware advances that have not yet arrived.

## 19.8 *The book in one circuit*

Shor’s algorithm ties together every conceptual thread of this book.

- It begins with the qubit register in superposition, generated by a Hadamard layer.
- It evaluates  $f(x) = a^x \bmod N$  via a quantum circuit, entangling the input and function registers (entanglement).
- It measures the function register, collapsing the input register to a periodic comb (measurement).
- It applies the quantum Fourier transform, exploiting interference to convert period in the time domain to peaks in the



frequency domain.

- It runs on a fault-tolerant architecture (Chapter on Error Correction) because the circuit depth far exceeds NISQ coherence.
- It is implemented in Qiskit or similar frameworks (Chapter on Qiskit), albeit at small instance sizes.
- Its consequences for RSA and ECC drive the entire post-quantum cryptography agenda for the financial sector.

Shor's algorithm is the canonical demonstration of why quantum computing matters. It is also a sobering reminder that the same technology that could transform Monte Carlo simulation, portfolio optimization, and machine learning in finance also threatens to dismantle the cryptographic substrate on which the entire financial system runs. Building the first capability and defending against the second are two sides of the same project, and both demand the attention of the financial industry over the next decade.

The remaining chapters of the book, and the surveys cited throughout, return to those positive applications: where quantum computing helps finance, how to think about advantage on real hardware, and what the discipline looks like as it matures from a research curiosity into a practical tool.



## *Chapter 20*

# *Where Now?*

### HOPES, FEARS, AND A WORKING AGENDA FOR QUANTUM FINANCE

A book is a snapshot. The reader who closes this one in 2026 will, by 2030, be working in a different quantum-finance landscape than the one we have just described. New hardware milestones will land, new algorithmic ideas will percolate, the post-quantum cryptography migration will be partway through. This concluding chapter tries to do two things: lay out a working agenda for the practitioner, researcher, or student wondering what to do next; and acknowledge openly the hopes and fears that surround this discipline. Both matter. Neither belongs in isolation.

## 20.1 *What we know, and where the limits are*

After 17 chapters, a few statements have crystallised.

**The mathematics is settled.** Quantum mechanics, as a computational model, is mature. Hilbert space, unitary evolution, the Born rule—these are not in dispute. Algorithms with provable speedups (Shor, Grover, quantum amplitude estimation, HHL under conditions) are well-characterised. The theoretical case for quantum advantage on specific financial bottlenecks is solid [Egger et al., 2020; Herman et al., 2023; Gong et al., 2026].

**The hardware is improving, slowly.** NISQ devices have grown from tens to thousands of physical qubits over a decade. Gate fidelities have improved by an order of magnitude. The transition to fault-tolerant computation has begun in the form of small logical-qubit demonstrations. Vendor roadmaps point to hundreds of logical qubits within five to ten years, with the cryptographically relevant scale—millions of physical qubits—a longer reach [IBM Quantum, 2023; Preskill, 2018].

**The financial use cases are real but bounded.** Quantum advantages in derivative pricing [Stamatopoulos et al., 2020; Chen et al., 2025], portfolio optimization [Morapakula et al., 2026; Thomassin et al., 2025], risk [Matsakos and Nield, 2024; Dri et al., 2022], and machine learning [Ciceri et al., 2025] are credible at the algorithmic level, sometimes demonstrable at the hardware level. None of them constitute production-scale industrial advantage today.

**The cryptographic threat is real and immediate.** Shor's algorithm on a sufficient quantum computer breaks RSA and ECC. The hardware is not yet there. The harvest-now-decrypt-later attack on confidential financial data is operational today. Post-quantum migration is therefore underway, regardless of any optimization or pricing upside [[Auer et al., 2024](#); [National Institute of Standards and Technology, 2024](#)].

These four statements are, in my reading, the durable conclusions of the field. The rest is execution.

## *20.2 An agenda for the next five years*

For different audiences, the most productive next steps differ.

### **For the finance practitioner.**

- Inventory cryptographic dependencies in your firm's stack. RSA, ECC, ECDSA usage in TLS, signing, settlement, identity, archives.
- Begin hybrid post-quantum deployments for high-value, long-confidentiality data. NIST's ML-KEM and ML-DSA are the standards to target.
- Run small-scale pilot studies with quantum hardware vendors (IBM, IonQ, Quantinuum, QuEra) on a representative optimization or pricing problem. The goal is intuition, not production.

- Build internal expertise. One or two people learning Qiskit and reading the primary literature is far more valuable than waiting for a quantum solution to be “ready.”

### **For the researcher.**

- Bridge the input problem: efficient quantum data loading from classical financial data sets remains an open and consequential bottleneck.
- Develop hardware-aware variational ansatz designs for portfolio and risk problems, taking real connectivity and noise into account [[Cerezo et al., 2021](#)].
- Pursue error-mitigated quantum amplitude estimation at scales just beyond classical reach—following the template of [Kim et al. \[2023\]](#).
- Study quantum-inspired classical algorithms [[Chou et al., 2026](#)]; they capture some of the upside without hardware risk and can deliver value today.

### **For the student.**

- Read the foundational papers, not just the surveys. Shor 1994, Grover 1996, Deutsch–Jozsa, the original Bell paper, the original RSA paper. The clarity of those papers is itself instructive.
- Implement a quantum amplitude estimation circuit in Qiskit from scratch. It will teach you more than any review

paper.

- Take a serious classical finance course in parallel. Quantum advantage in finance only matters relative to a well-understood classical baseline; without the baseline, you cannot tell what is advance and what is hype.
- Stay close to the hardware. Devices change quickly; intuitions about what is possible should change with them.

### **For the policymaker.**

- Treat post-quantum migration as critical infrastructure. The financial system's cryptographic transition will not happen voluntarily on the timeline that adversaries' quantum capabilities may dictate.
- Support open benchmarking. Application-level benchmarks measured on real hardware are what distinguish progress from press release, and the public-good case for funding them is strong.
- Track the export and dual-use implications of quantum capability. A cryptographically relevant quantum computer is a strategic asset.

## *20.3 The hopes*

Quantum computing genuinely offers things that classical computing cannot. The honest hopes for quantum finance

are these.

**Quadratic speedup on Monte Carlo, at scale.** If amplitude estimation can be run on a fault-tolerant quantum computer with enough qubits and short enough cycle time, the financial industry's Monte Carlo bill—which today consumes substantial fractions of the compute budgets at major banks—could be cut substantially. Pricing of exotic derivatives, value-at-risk on large portfolios, regulatory stress testing: all would benefit. The advantage is not exponential; it is quadratic. But on workloads measured in compute-years, quadratic matters.

**Portfolio optimization with realistic constraints.** Markowitz mean-variance optimization is easy classically. Markowitz with cardinality constraints, transaction-cost penalties, regulatory bounds, dynamic rebalancing, and risk-budget overlays is hard, and current methods rely on relaxations or heuristics. A fault-tolerant quantum optimizer that handles such combinatorial structures natively could produce better portfolios at the institutional scale. Hybrid approaches [[Morapakula et al., 2026](#)] are early evidence in this direction.

**Quantum-enhanced machine learning.** The most interesting and least-understood frontier. [Ciceri et al. \[2025\]](#)'s bond-trading result, suggesting that quantum hardware noise itself can improve generalization, is the kind of surprise that points to genuinely new territory. If this generalises, ML on quantum-transformed financial features could give modest

but real edge in prediction tasks where modest edge translates into significant profit.

**Modelling truly quantum financial structure.** Speculative but worth saying. Some researchers [[Schaden, 2002](#)] argue that quantum formalism is not just a computational tool but a more faithful model of certain market phenomena—non-commuting observables capturing trader behaviour, path integrals capturing price dynamics. If even a small fraction of this proves productive, quantum computing in finance would be more than a faster classical engine; it would be a new modelling language.

**Security beyond computational hardness.** Quantum key distribution and entanglement-based protocols offer information-theoretic security, not just computational security. For the most sensitive financial communications—inter-central-bank settlement, sovereign-debt operations—this is a meaningful upgrade, and one whose deployment is technically feasible now via quantum-network testbeds.

These hopes are not guarantees. They are directions where the structural case for advantage is strong and where, if hardware delivers, the financial impact would be substantial.

## 20.4 *The fears*

A book that lists only the upside is not honest. The fears are equally real.



**The cryptographic transition fails or stalls.** Post-quantum migration is technically tractable but operationally enormous. If the financial sector underestimates the timeline—if regulators delay, vendors lag, or legacy systems prove unmigratable—we may arrive at the era of cryptographically relevant quantum computers with critical infrastructure still vulnerable. The harvest-now–decrypt-later threat compounds this: data encrypted today with RSA or ECC may be decryptable in a decade, and the damage retroactive.

**Quantum advantage on production-scale financial problems may take much longer than vendor roadmaps suggest.** The path from current NISQ hardware to fault-tolerant systems large enough to factor RSA-2048 or run amplitude estimation at industrial scale involves several still-unsolved engineering problems: cryogenic control electronics, low-noise interconnects, magic-state distillation overhead, classical co-processing latency. Any of these could slip by years. A decade of hype followed by underwhelming delivery would damage the field's credibility and divert resources from problems that classical methods could have solved.

**Concentration of quantum capability.** Quantum hardware is built and operated by a small number of vendors and nation-states. If cryptographically relevant capability materialises asymmetrically—available to some actors but not others—the implications for financial markets are severe. A single firm with a working factoring engine could decrypt competitors' communications, forge digital signatures, and

manipulate markets at scale. The non-proliferation problem here is harder than for many other strategic technologies because the relevant artefact is software running on rare hardware, not a physical object.

**Quantum-washing.** A version of greenwashing for quantum. Firms claiming quantum advantage on problems where classical methods would do as well, or running “quantum” algorithms that are actually quantum-inspired classical ones. Researchers chasing publishable demos on toy instances rather than honest scaling studies. Investors deploying capital on the basis of incremental hardware press releases. This pattern is already visible and is likely to accelerate as more hype-cycle dollars flow into the field. The discipline of asking “what would the classical baseline produce?” is the only protection.

**The fundamental physics may not cooperate.** A more existential concern. Several proposed paths to fault-tolerant quantum computing rest on assumptions about noise correlations, materials properties, and the absence of unknown decoherence channels that may not hold at scale. If the noise turns out to be more correlated than assumed, or if some new physics intervenes at large qubit numbers, the threshold theorem’s promises may be harder to redeem than current optimism suggests. This is unlikely but not impossible.

**Quantum finance crowds out better questions.** The optimization, pricing, and machine-learning problems quantum

computing might accelerate are not necessarily the most important problems in finance. Climate-related financial risk, systemic resilience to non-stationarity, equitable access to credit and capital—these are first-order problems for which quantum computing has nothing to say. A research culture too captivated by quantum’s appeal could spend a generation of talent on second-order problems.

## 20.5 *A balanced posture*

The right posture toward quantum computing in finance is neither evangelism nor dismissal. It is engaged scepticism: take the technology seriously enough to learn it, run pilots, train people, migrate cryptography on the timeline that the threat model demands; remain sceptical enough to compare against classical baselines, to distinguish demonstration from production, and to track the field’s claims against its delivery.

The mature quantum-finance team in 2030 will look different from today’s. It will include cryptographers running the post-quantum transition, quantum-algorithm researchers wired into the academic literature, classical quants benchmarking quantum candidates against state-of-the-art alternatives, and software engineers operating hybrid classical-quantum workflows on cloud infrastructure. The team will treat quantum hardware as one accelerator among several—GPUs, TPUs, FPGAs, quantum—chosen for the problem at hand on technical merits rather than novelty.

That is the team this book has tried to equip. The mathematics, physics, algorithms, hardware realities, and cryptographic implications are all in scope; the financial applications motivate the choices; the open questions are flagged honestly rather than papered over. The remaining work is the actual work—running the algorithms, migrating the cryptography, building the intuition, comparing the baselines.

## 20.6 *Closing*

The most exciting thing about quantum computing for finance is that we genuinely do not know how the next decade will unfold. We have credible algorithms, improving hardware, an industry willing to invest, and a cryptographic deadline that focuses minds. We also have real engineering obstacles, a long history of overpromising, and a tendency for hype to outpace delivery. Both forces are operating simultaneously, and the outcome will be shaped by the technical and operational choices made over the next several years.

If this book has done its job, the reader leaves with three things: enough mathematics to read the primary literature; enough physics and hardware intuition to evaluate claims of advantage; and enough financial context to know which problems are worth attacking. The rest—which problems you attack, on which hardware, with which baselines, on what timeline—is the work itself.

Go do it. And when the inevitable updates to this book are

written, they will tell the story of what you, collectively, made of the moment.



# Bibliography

- Quantum Portfolio Optimizer - A Qiskit Function by Global Data Quantum. [27](#), [173](#)
- D. Aharonov and M. Ben-Or. Fault-Tolerant Quantum Computation with Constant Error Rate. *SIAM Journal on Computing*, 38(4):1207–1282, 2008. doi: 10.1137/S0097539799359385. [154](#)
- F. Arute, K. Arya, R. Babbush, et al. Quantum Supremacy Using a Programmable Superconducting Processor. *Nature*, 574(7779):505–510, 2019. doi: 10.1038/s41586-019-1666-5. [53](#), [131](#)
- A. Aspect, J. Dalibard, and G. Roger. Experimental Test of Bell’s Inequalities Using Time-Varying Analyzers. *Physical Review Letters*, 49(25):1804–1807, 1982. doi: 10.1103/PhysRevLett.49.1804. [56](#), [102](#), [104](#)
- R. Auer, A. Dupont, L. Gambacorta, et al. Quantum computing and the financial system: opportunities and risks. *BIS Papers*, 2024. [15](#), [16](#), [25](#), [183](#), [186](#), [208](#), [212](#)

- A. Barenco, C. H. Bennett, R. Cleve, et al. Elementary Gates for Quantum Computation. *Physical Review A*, 52(5):3457–3467, 1995. doi: 10.1103/PhysRevA.52.3457. [55](#), [61](#), [87](#)
- J. S. Bell. On the Einstein Podolsky Rosen Paradox. *Physica Physique Fizika*, 1(3):195–200, 1964. doi: 10.1103/PhysicsPhysiqueFizika.1.195. [56](#), [102](#), [104](#)
- C. H. Bennett. Logical Reversibility of Computation. *IBM Journal of Research and Development*, 17(6):525–532, 1973. doi: 10.1147/rd.176.0525. [51](#)
- C. H. Bennett and G. Brassard. Quantum Cryptography: Public Key Distribution and Coin Tossing. *Theoretical Computer Science*, 560:7–11, 2014. doi: 10.1016/j.tcs.2014.05.025. Originally in Proc. IEEE Conf. on Computers, Systems, and Signal Processing (Bangalore, 1984). [74](#), [104](#), [108](#)
- J. Biamonte, P. Wittek, N. Pancotti, et al. Quantum machine learning. *Nature*, 549(7671):195–202, September 2017. ISSN 1476-4687. doi: 10.1038/nature23474. Bandiera\_abtest: a Cg\_type: Nature Research Journals Number: 7671 Primary\_atype: Reviews Subject\_term: Computer science;Quantum information;Quantum simulation Subject\_term\_id: computer-science;quantum-information;quantum-simulation. [22](#), [23](#), [65](#), [98](#)
- J. Blanchet, M. S. Squillante, M. Szegedy, and G. Wang. Connecting Quantum Computing with Classical Stochastic Simulation, September 2025. arXiv:2509.18614 [quant-ph]. [18](#)

- M. Born. Zur Quantenmechanik der Stoßvorgänge. *Zeitschrift für Physik*, 37(12):863–867, 1926. doi: 10.1007/BF01397477. [52](#), [71](#)
- L. Bunesu and A. M. Vartei. Modern finance through quantum computing—A systematic literature review. *PLOS ONE*, 19(7):e0304317, July 2024. ISSN 1932-6203. doi: 10.1371/journal.pone.0304317. [15](#)
- A. R. Calderbank and P. W. Shor. Good Quantum Error-Correcting Codes Exist. *Physical Review A*, 54(2):1098–1105, 1996. doi: 10.1103/PhysRevA.54.1098. [152](#)
- M. Cerezo, A. Arrasmith, R. Babbush, et al. Variational Quantum Algorithms. *Nature Reviews Physics*, 3(9):625–644, 2021. doi: 10.1038/s42254-021-00348-9. [57](#), [64](#), [108](#), [117](#), [160](#), [213](#)
- J. Chen, Y. Li, and A. Neufeld. Quantum Monte Carlo algorithm for solving Black-Scholes PDEs for high-dimensional option pricing in finance and its complexity analysis, November 2025. arXiv:2301.09241 [quant-ph]. [18](#), [211](#)
- Y. H. Chou, Y. T. Lai, Y. C. Jiang, et al. Quantum-inspired Financial Optimization for Dynamic G7 Portfolio Management [Feature]. *IEEE Nanotechnology Magazine*, 20(1):22–31, 2026. ISSN 1932-4510. doi: 10.1109/MNANO.2025.3638485. [20](#), [213](#)
- A. Ciceri, A. Cottrell, J. Freeland, et al. Enhanced fill probability estimates in institutional algorithmic bond trading



using statistical learning algorithms with quantum computers, September 2025. arXiv:2509.17715 [quant-ph] version:  
1. [24](#), [29](#), [57](#), [163](#), [166](#), [177](#), [211](#), [215](#)

J. I. Cirac and P. Zoller. Quantum Computations with Cold Trapped Ions. *Physical Review Letters*, 74(20):4091–4094, 1995. doi: 10.1103/PhysRevLett.74.4091. [92](#), [132](#)

D. Deutsch. Quantum Theory, the Church–Turing Principle and the Universal Quantum Computer. *Proceedings of the Royal Society of London. A*, 400(1818):97–117, 1985. doi: 10.1098/rspa.1985.0070. [59](#), [63](#), [96](#)

W. Diffie and M. E. Hellman. New Directions in Cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976. doi: 10.1109/TIT.1976.1055638. [180](#)

P. A. M. Dirac. *The Principles of Quantum Mechanics*. Oxford University Press, 4th edition, 1958. [69](#), [77](#)

D. P. DiVincenzo. The Physical Implementation of Quantum Computation. *Fortschritte der Physik*, 48(9-11):771–783, 2000. doi: 10.1002/1521-3978(200009)48:9/11<771::AID-PROP771>3.0.CO;2-E. [92](#), [130](#)

E. Dri, E. Giusto, A. Aita, and B. Montrucchio. Towards practical Quantum Credit Risk Analysis. *Journal of Physics: Conference Series*, 2416(1):012002, December 2022. ISSN 1742-6588, 1742-6596. doi: 10.1088/1742-6596/2416/1/012002. arXiv:2212.07125 [cs]. [21](#), [173](#), [177](#), [211](#)

- D. J. Egger, C. Gambella, J. Marecek, et al. Quantum Computing for Finance: State of the Art and Future Prospects. *IEEE Transactions on Quantum Engineering*, 1: 1–24, 2020. ISSN 2689-1808. doi: 10.1109/TQE.2020.3030314. arXiv:2006.14510 [quant-ph]. [14](#), [15](#), [17](#), [19](#), [29](#), [55](#), [64](#), [83](#), [107](#), [211](#)
- A. Einstein, B. Podolsky, and N. Rosen. Can Quantum-Mechanical Description of Physical Reality Be Considered Complete? *Physical Review*, 47(10):777–780, 1935. doi: 10.1103/PhysRev.47.777. [104](#)
- E. Farhi, J. Goldstone, and S. Gutmann. A Quantum Approximate Optimization Algorithm, 2014. [64](#), [161](#)
- R. P. Feynman. Simulating Physics with Computers. *International Journal of Theoretical Physics*, 21(6–7):467–488, 1982. doi: 10.1007/BF02650179. [54](#), [59](#), [64](#), [124](#), [126](#)
- A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland. Surface Codes: Towards Practical Large-Scale Quantum Computation. *Physical Review A*, 86(3):032324, 2012. doi: 10.1103/PhysRevA.86.032324. [65](#), [150](#), [153](#)
- H. Gong, A. Sedai, T. Schroeder, and F. Medda. Quantum Computing for Financial Transformation: A Review of Optimisation, Pricing, Risk, Machine Learning, and Post-Quantum Security, April 2026. arXiv:2604.08180 [q-fin]. [15](#), [16](#), [26](#), [27](#), [29](#), [208](#), [211](#)

- L. K. Grover. A Fast Quantum Mechanical Algorithm for Database Search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (STOC '96)*, pages 212–219, 1996. doi: 10.1145/237814.237866. 54, 64, 113
- A. W. Harrow, A. Hassidim, and S. Lloyd. Quantum Algorithm for Linear Systems of Equations. *Physical Review Letters*, 103(15):150502, 2009. doi: 10.1103/PhysRevLett.103.150502. 64, 134
- D. Herman, C. Googin, X. Liu, et al. A Survey of Quantum Computing for Finance. January 2022. \_eprint: 2201.02773. 15
- D. Herman, C. Googin, X. Liu, et al. Quantum computing for finance. *Nature Reviews Physics*, 5(8), July 2023. ISSN 2522-5820. doi: 10.1038/s42254-023-00603-1. 14, 15, 16, 23, 29, 211
- J. Huang. NUS-Frontiers-in-Fintech-and-Quantum-Computing-2022/Part 2 Quantum Finance/Part 2 Quantum Finance - Hands-on.ipynb at main · HuangJunye/NUS-Frontiers-in-Fintech-and-Quantum-Computing-2022. 24
- IBM Quantum. The IBM Quantum Development Roadmap. *IBM Research Blog*, 2023. 138, 211
- B. E. Kane. A Silicon-Based Nuclear Spin Quantum Computer. *Nature*, 393(6681):133–137, 1998. doi: 10.1038/30156. 135

- Y. Kim, A. Eddins, S. Anand, et al. Evidence for the utility of quantum computing before fault tolerance. *Nature*, 618 (7965):500–505, 2023. doi: 10.1038/s41586-023-06096-3. [146](#), [163](#), [213](#)
- A. Y. Kitaev. Fault-Tolerant Quantum Computation by Anyons. *Annals of Physics*, 303(1):2–30, 2003. doi: 10.1016/S0003-4916(02)00018-0. [65](#), [136](#), [153](#)
- N. Koblitz. Elliptic Curve Cryptosystems. *Mathematics of Computation*, 48(177):203–209, 1987. doi: 10.1090/S0025-5718-1987-0866109-5. [188](#)
- J. Koch, T. M. Yu, J. Gambetta, et al. Charge-Insensitive Qubit Design Derived from the Cooper Pair Box. *Physical Review A*, 76(4):042319, 2007. doi: 10.1103/PhysRevA.76.042319. [87](#), [92](#), [124](#), [131](#)
- R. Landauer. Irreversibility and Heat Generation in the Computing Process. *IBM Journal of Research and Development*, 5(3):183–191, 1961. doi: 10.1147/rd.53.0183. [51](#)
- A. K. Lenstra, H. W. Lenstra, M. S. Manasse, and J. M. Pollard. The Number Field Sieve. pages 11–42, 1993. doi: 10.1007/BFb0091537. [54](#), [182](#), [203](#)
- D. Loss and D. P. DiVincenzo. Quantum Computation with Quantum Dots. *Physical Review A*, 57(1):120–126, 1998. doi: 10.1103/PhysRevA.57.120. [135](#)

- A. Martin, B. Candelas, A. Rodriguez-Rozas, et al. Toward pricing financial derivatives with an IBM quantum computer. *Physical Review Research*, 3(1):013167, February 2021. doi: 10.1103/PhysRevResearch.3.013167. 17
- T. Matsakos and S. Nield. Quantum Monte Carlo simulations for financial risk analytics: scenario generation for equity, rate, and credit risk factors. *Quantum*, 8:1306, April 2024. doi: 10.22331/q-2024-04-04-1306. 18, 22, 64, 211
- V. S. Miller. Use of Elliptic Curves in Cryptography. In *Advances in Cryptology — CRYPTO ’85 Proceedings*, pages 417–426. Springer, 1986. doi: 10.1007/3-540-39799-X\_31. 188
- S. N. Morapakula, S. Deshpande, R. Yata, et al. End-to-End Portfolio Optimization with Hybrid Quantum Annealing. *Advanced Quantum Technologies*, 9(4):e00753, 2026. doi: <https://doi.org/10.1002/qute.202500753>. \_eprint: <https://advanced.onlinelibrary.wiley.com/doi/pdf/10.1002/qute.202500753>. 19, 27, 57, 108, 136, 166, 177, 211, 215
- National Institute of Standards and Technology. Module-Lattice-Based Key-Encapsulation Mechanism Standard. NIST FIPS 203, 2024. 187, 207, 212
- M. A. Nielsen and I. L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010. ISBN 978-1-107-00217-3. 59, 65, 90, 104, 106, 149, 154

- R. Orús, S. Mugel, and E. Lizaso. Quantum computing for finance: Overview and prospects. *Reviews in Physics*, 4: 100028, November 2019. ISSN 24054283. doi: 10.1016/j.revip.2019.100028. 15, 19
- P. Pathak, K. Tarakeshwar, S. S. Ali, et al. The Evolution of IBM's Quantum Information Software Kit (Qiskit): A Review of its Applications, August 2025. arXiv:2508.12245 [quant-ph]. 27, 168
- A. Peruzzo, J. McClean, P. Shadbolt, et al. A Variational Eigenvalue Solver on a Photonic Quantum Processor. *Nature Communications*, 5(1):4213, 2014. doi: 10.1038/ncomms5213. 64, 161
- J. M. Pollard. A Monte Carlo Method for Factorization. *BIT Numerical Mathematics*, 15(3):331–334, 1975. doi: 10.1007/BF01933667. 185
- J. Preskill. Quantum Computing in the NISQ era and beyond. *Quantum*, 2:79, August 2018. ISSN 2521-327X. doi: 10.22331/q-2018-08-06-79. arXiv: 1801.00862. 15, 27, 57, 65, 66, 115, 159, 211
- R. L. Rivest, A. Shamir, and L. Adleman. A Method for Obtaining Digital Signatures and Public-Key Cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978. doi: 10.1145/359340.359342. 179, 180
- B. V. Samal. Quantum Computing for Finance: In-Depth Overview and Bayesian Prospects, October 2024. 23

- M. Schaden. Quantum finance. *Physica A: Statistical Mechanics and its Applications*, 316(1):511–538, December 2002. ISSN 0378-4371. doi: 10.1016/S0378-4371(02)01200-1. [28](#), [216](#)
- E. Schrödinger. An Undulatory Theory of the Mechanics of Atoms and Molecules. *Physical Review*, 28(6):1049–1070, 1926. doi: 10.1103/PhysRev.28.1049. [69](#), [120](#)
- C. E. Shannon. A Mathematical Theory of Communication. *Bell System Technical Journal*, 27(3):379–423, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x. [51](#)
- P. W. Shor. Scheme for Reducing Decoherence in Quantum Computer Memory. *Physical Review A*, 52(4):R2493–R2496, 1995. doi: 10.1103/PhysRevA.52.R2493. [151](#)
- P. W. Shor. Fault-tolerant quantum computation. pages 56–65, 1996. doi: 10.1109/SFCS.1996.548464. [154](#)
- P. W. Shor. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997. doi: 10.1137/S0097539795293172. [54](#), [63](#), [199](#)
- H. Soller. Quantum computing in finance: Redefining banking | McKinsey. [15](#)
- H. Soller, S. Smit, and A. Heid. Quantum computing business applications and benefits | McKinsey. [15](#)

- N. Stamatopoulos, D. J. Egger, Y. Sun, et al. Option Pricing using Quantum Computers. *Quantum*, 4:291, July 2020. doi: 10.22331/q-2020-07-06-291. [17](#), [64](#), [98](#), [107](#), [116](#), [173](#), [177](#), [204](#), [211](#)
- A. M. Steane. Error Correcting Codes in Quantum Theory. *Physical Review Letters*, 77(5):793–797, 1996. doi: 10.1103/PhysRevLett.77.793. [152](#)
- K. Temme, S. Bravyi, and J. M. Gambetta. Error Mitigation for Short-Depth Quantum Circuits. *Physical Review Letters*, 119(18):180509, 2017. doi: 10.1103/PhysRevLett.119.180509. [117](#), [146](#), [162](#)
- P. Thomassin, G. Guerard, S. Djebali, and V. M. Lambert. A Quantum Model for Constrained Markowitz Modern Portfolio Using Slack Variables to Process Mixed-Binary Optimization under QAOA. *Quantum Machine Intelligence*, 7(2):102, December 2025. ISSN 2524-4906, 2524-4914. doi: 10.1007/s42484-025-00330-z. arXiv:2601.03278 [math]. [20](#), [108](#), [161](#), [173](#), [211](#)
- L. M. K. Vandersypen, M. Steffen, G. Breyta, et al. Experimental Realization of Shor’s Quantum Factoring Algorithm Using Nuclear Magnetic Resonance. *Nature*, 414(6866):883–887, 2001. doi: 10.1038/414883a. [205](#)
- K. Zaman, A. Marchisio, M. Kashif, and M. Shafique. PO-QA: A Framework for Portfolio Optimization using Quantum Algorithms, July 2024. arXiv:2407.19857 [quant-ph]. [20](#), [108](#), [161](#), [173](#)



- A. Zecevic. *Quantum and Parallel Computing: A Tale of Two Paradigms*. Independently published, 2022. ISBN 979-8-7885-0405-6. [31](#)
- J. Zhou. Quantum Finance: Exploring the Implications of Quantum Computing on Financial Models. *Computational Economics*, 67(2):1043–1072, February 2026. ISSN 1572-9974. doi: 10.1007/s10614-025-10894-4. [15](#), [22](#)
- W. H. Zurek. Decoherence, Einselection, and the Quantum Origins of the Classical. *Reviews of Modern Physics*, 75(3): 715–775, 2003. doi: 10.1103/RevModPhys.75.715. [141](#), [146](#)

# *Index*

- 1/f noise, [144](#)
- amplitude encoding, [98](#)
- amplitudes, [52](#), [86](#)
- AND, [51](#)
- asymmetric, [180](#)
- Bell states, [103](#)
- bilinear pairings, [195](#)
- Bloch sphere, [88](#)
- Born rule, [71](#)
- bra, [78](#)
- Bra-ket, [77](#)
- bra-ket, [76](#)
- Cauchy-Schwarz inequality, [79](#)
- Church-Turing-Deutsch, [63](#)
- circuit layer operations per second, [137](#)
- classical bit, [51](#)
- Clifford+T, [89](#)
- CNOT, [61](#)
- coherences, [100](#)

- coherent, 114
- coherent superposition, 99
- combinatorial optimization, 19
- completeness relation, 81
- computational basis, 86
- cross terms, 112
- crypto-agility, 208
- CSS, 152
- Decoherence, 140
- decoherence, 115
- density operator, 73
- discrete logarithm problem, 204
- discretization of continuous errors, 152
- ECC, 209
- elliptic curve, 189
- elliptic curve discrete logarithm problem, 190
- elliptic curves, 187
- elliptic-curve cryptography, 188
- entangled, 72, 103
- entanglement, 14, 101, 102, 208
- entanglement-based, 108
- entropy of entanglement, 105
- EPR paradox, 104
- error correction, 115
- Error mitigation, 146
- error mitigation, 162
- expectation value, 71

fault-tolerant, [209](#)  
Feynman–Kac, [125](#)  
fidelity, [74](#)  
  
Hamiltonian, [121](#)  
harvest now, decrypt later, [16](#), [186](#)  
hybrid quantum–classical, [57](#)  
hybrid quantum-classical, [27](#)  
  
input problem, [23](#), [98](#)  
integer factorization, [181](#)  
interference, [14](#), [97](#), [101](#), [109](#), [111](#), [208](#)  
  
jump operators, [127](#), [144](#)  
  
ket, [78](#)  
Kraus representation, [143](#)  
  
Larmor precession, [122](#)  
Lindblad master equation, [144](#)  
loss given default, [21](#)  
  
machine learning, [15](#)  
magic state distillation, [153](#)  
measurement, [208](#)  
  
NAND, [51](#)  
NISQ, [159](#)  
NISQ era, [15](#)  
no-cloning theorem, [74](#), [90](#), [150](#)  
no-signalling theorem, [105](#)

NOT, 51

number field sieve, 182

open quantum systems, 127

optimization, 15

OR, 51

order, 200

outer product, 80

period, 200

phase kickback, 115

post-quantum cryptography, 187

primitives, 172

projector, 80

public-key, 180

QAOA, 19

Qiskit, 167, 168, 209

Quadratic Unconstrained Binary Optimization, 19

Quantum amplitude estimation, 17

quantum amplitude estimation, 98

quantum annealing, 19

quantum channel, 143

quantum circuit, 61

Quantum error correction, 149

quantum error correction, 65, 148

quantum gate, 61

quantum machine learning, 22

quantum operation, 143

quantum parallelism, 97

quantum supremacy, 131  
quantum-inspired, 21  
qubit, 52, 60, 85, 208  
qubits, 14  
  
ray, 70  
resolution of the identity, 81  
RSA, 179, 209  
Rydberg interactions, 133  
  
scalar multiplication, 190  
Schmidt decomposition, 105  
Schmidt rank, 106  
Schrödinger equation, 120  
separable, 103  
shallow amplitude estimation, 117  
shallow ansatz, 108  
Shor's algorithm, 164, 187  
simulation and sampling, 15  
slack variables, 20  
stabilizer formalism, 152  
stationary states, 122  
statistical mixture, 99  
superposition, 69, 208  
superposition principle, 94  
surface code, 153  
  
tensor product, 72  
three-qubit bit-flip code, 150  
threshold theorem, 154

time-independent Schrödinger equation, [121](#)  
Toffoli gate, [55](#)  
trace distance, [74](#)  
transmon, [131](#)  
Transpilation, [171](#)  
trapdoor function, [180](#)  
  
uniform superposition, [96](#)  
unitary, [55](#)  
universal gate set, [61](#)  
  
variational quantum algorithm, [160](#)